

SPADE ♠: Self-Play in Adaptive Synthetic Executable Environments

Bo Liu^{1,2,*}, Simon Yu^{3,*}, Yiding Jiang⁴, Ao Qu⁵, Andrew Zhao, Zichen Liu⁶, Junsu Kim⁷, Zijian Zhou⁶, Seungone Kim⁴, Tongzheng Ren, Mickel Liu¹, Hanfei Yu⁸, Zhaorun Chen⁹, Weiyan Shi³, Paul Pu Liang⁵, Luke Zettlemoyer¹, Yejin Choi², Natasha Jaques¹

¹University of Washington, ²Stanford University, ³Northeastern University, ⁴Carnegie Mellon University, ⁵Massachusetts Institute of Technology, ⁶National University of Singapore, ⁷Seoul National University, ⁸Stevens Institute of Technology, ⁹University of Chicago

*Equal contribution

Continuous self-improvement requires an ever-expanding pool of self-generated, diverse, adaptive goals. For language agents, existing training environment pools (hand-curated, statically synthesized, or frozen-verifier) keep the goal distribution fixed as the learner scales. We introduce SPADE (Self-Play in Adaptive Synthetic Executable Environments), a self-play RL framework in which a single LLM plays two roles: an **Environment Designer** that writes complete, long-horizon training environments as executable code with an OpenAI Gym-style `reset()/step()` interface, and a **Reasoning Agent** that learns to act in them. Each is a stateful, multi-turn environment (state transitions, reward functions, and verification code), so one interface spans reasoning problems and multi-step agentic tool use. The **Reasoning Agent**’s regret is estimated using the gap between its reward with and without privileged hints; in optimizing this regret signal the **Environment Designer** learns to target environments at the edge of the agent’s capabilities while keeping them feasible. Through extensive experimentation, we find several components critical to success: grounding the **Environment Designer** on documents sampled from a large pretraining corpus, and giving it an accumulated environment memory. Scaling to 30B-parameter models, SPADE improves over the strongest fixed-environment baseline by +5.3 on average across eight held-out math, science, code, and reasoning benchmarks, and lifts the tool-use setting by +5.7 on BFCL v4 multi-turn and +13.9 on ACEBench-Agent; on the games setting, the margin over the strongest baseline grows with model scale. By making environment design itself a learnable component, SPADE takes a concrete step toward open-ended self-improvement.

Correspondence: benjaminliu.eecs@gmail.com, nj@cs.washington.edu

Code: <https://github.com/spade-rl/spade>

Project: <https://spade-rl.github.io>

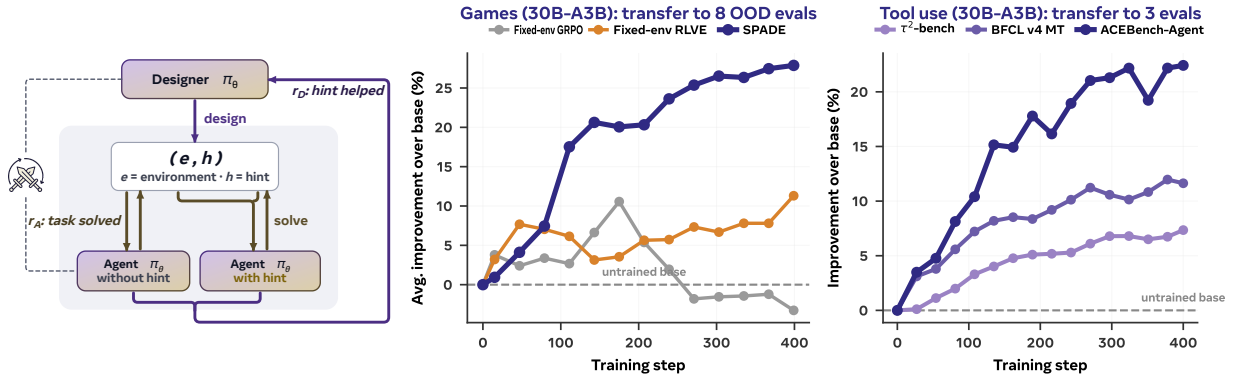


Figure 1 SPADE designs and solves its own training environments in both settings. **Left:** a single LLM π_θ plays both roles, an **Environment Designer** that writes an executable environment e with a privileged hint h , and a **Reasoning Agent** that solves e with and without h ; the return gap rewards the **Environment Designer** (hint-based regret), task completion rewards the **Reasoning Agent**, and both update the same weights. **Middle:** average *relative* improvement over the untrained base across the eight games-setting evals ((score – base)/base), versus Fixed-env RLVE (orange) and Fixed-env GRPO (gray). **Right:** per-benchmark relative improvement of SPADE-30B-A3B on the three tool-use evals (τ^2 -bench, BFCL v4 multi-turn, and ACEBench-Agent; Table 2).

1 Introduction

Agentic AI has become broadly capable: language models now reason over long horizons, use tools, search the web, and operate computers (OpenAI, 2024; DeepSeek-AI, 2025; Wang et al., 2025c; Zhang et al., 2025c). These gains increasingly come not from pretraining alone but from learning through experience, where a model improves from its own trajectories of interacting with an environment and the rewards they return (Silver and Sutton, 2025). As high-quality human text is a finite resource (Villalobos et al., 2024), the bottleneck for further agentic progress is increasingly the supply of training environments: interactive tasks with verifiable rewards. Building them has become a central industry investment, with major labs weighing environment budgets exceeding \$1 billion a year and a new class of startups raising nine figures to supply them (Zeff, 2025; PYMNTS, 2026). Yet whether hand-built or synthesized, these environments form a fixed pool that does not adapt as the learner improves, so an agent stops improving once it exhausts them.

Several lines of work try to keep agents improving, each with its own limitation. *Harness engineering* improves how an agent acts within a given environment at inference time, through tool orchestration and self-correction (Yao et al., 2023; Shinn et al., 2023), but it changes no weights, so its gains are tied to each hand-built harness, rather than improving general capabilities the model carries to new tasks. *Human-curated scaling* (Zhang et al., 2025a; Guertler et al., 2025; Liu et al., 2025c) builds diverse environments to train across, but scales only as fast as people can write them. *Synthetic generation* (Wang et al., 2026b; Tu et al., 2026; Zhu et al., 2026; Gandhi et al., 2026; Zeng et al., 2025) produces environments programmatically, but the generators are fixed, so the environment space does not grow with the agent and the model soon exhausts it (Song et al., 2024). *Self-play* (Zhao et al., 2025; Huang et al., 2025; Liu et al., 2025a) lets a model improve through dual roles, but ungrounded self-play is bounded by information symmetry, unable to pose challenges beyond its own knowledge and prone to amplifying its errors (Chae et al., 2025); corpus grounding mitigates this (Liu et al., 2025b), yet most methods still generate *tasks* (a problem with a sparse terminal reward) rather than complete *multi-turn environments* (state transitions, reward functions, and verification code). None produces a self-improving system whose environments keep growing in complexity as the agent improves.

We introduce SPADE (Self-Play in Adaptive Synthetic Executable Environments), a framework where a single LLM plays two roles: an **Environment Designer** that produces complete training environments as executable Python code, and a **Reasoning Agent** that learns from them. Each environment implements a Gym-style interface (the standard `reset()/step()` API used in reinforcement learning) (Brockman et al., 2016), representing a full Markov decision process (MDP) with state transitions and reward functions. This *code-as-environment* representation unifies single-turn settings (one step to terminal reward) and multi-turn agentic tasks (sequential interaction with transition dynamics) under a single interface. Because any computable MDP can be written as a program, the **Environment Designer** can express any such environment in code, rather than only those a hand-designed parameterization allows. Unlike prior works where the environment generator is frozen, SPADE’s **Environment Designer** is itself trained via RL with a *hint-based regret* signal that targets solvable environments precisely at the frontier of the **Reasoning Agent**’s capability, creating co-evolution where the environment distribution shifts as the **Reasoning Agent** improves (Figure 2).

Overall, our work makes the following contributions:

1. We introduce a *general framework for co-evolving environments synthesis and agentic capability through self-play*, where the same LLM both generates executable environments as code and learns to solve them. By representing environments as Python programs with a Gym-style interface, the framework unifies single-turn reasoning and multi-turn agentic tasks, and turns environment design into a learnable, RL-trained component of post-training, enabling continual open-ended self-improvement.
2. We introduce a *hint-based regret reward* for the **Environment Designer**. The **Environment Designer** is rewarded by the gap between **Reasoning Agent** performance with and without privileged hints, a lightweight estimate of minimax regret (Dennis et al., 2020) that targets environments at the frontier of the **Reasoning Agent**’s capability. Unlike a pure adversary, which is free to make environments unsolvable, or a cooperative designer, which can inflate **Reasoning Agent** reward without teaching it anything, hint-based regret yields constrained competitive dynamics that reward environments which are both solvable and at the learning frontier, with an equilibrium analysis in Appendix B.
3. We design a *corpus-grounded and memory-augmented design pipeline* spanning cognitive-skill games and

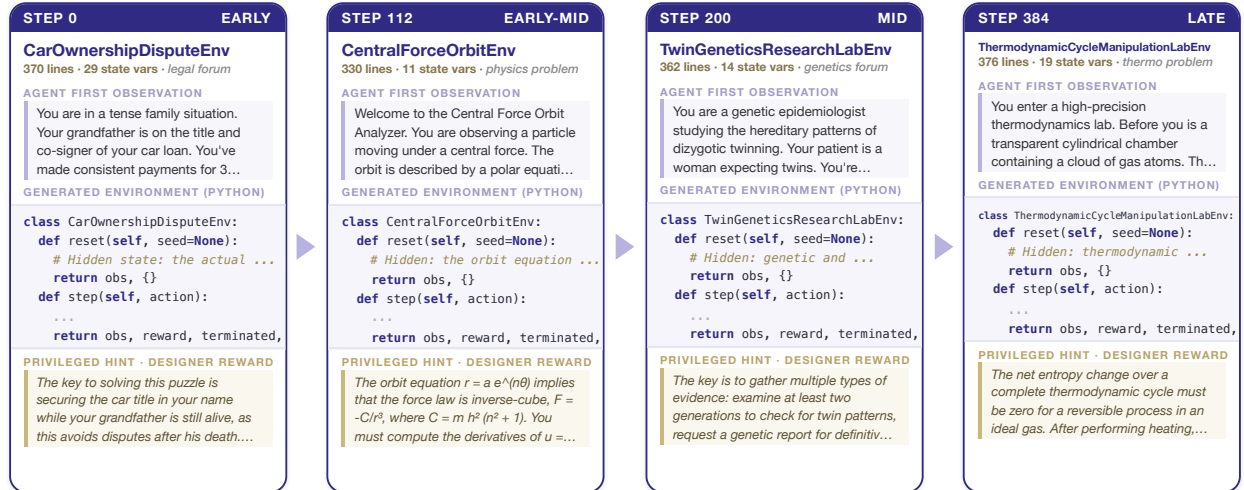


Figure 2 SPADE generates an adaptive, multi-turn curriculum. Four environments the Environment Designer produces over one 30B-A3B run, from step 0 (early) to step 384 (late); each card shows the agent’s first observation, the generated Python environment, and the designer-written hint. Every environment is a complete MDP with a `reset()`/`step()` interface, and the tasks shift toward state-gated, multi-turn interaction as the Reasoning Agent improves. Unlike a fixed human-curated pool or a frozen synthetic generator, this curriculum keeps moving with the learner.

tool-use tasks. Where prior environment-generation work targets a single domain (math, code, or tool use), SPADE demonstrates improvements in both settings, with environments grounded in pretraining corpus knowledge and accumulated experience.

4. We validate *self-play working at 30B+ scale* with practical recipe provided, demonstrating that the approach works beyond the small-model with a complete training recipe including environment validation, reward hacking avoidance, and curriculum design.

On the games setting, SPADE outperforms fixed-environment baselines on all three backbones, by +5.3 points on average and up to +7.5 on individual benchmarks over the strongest fixed-environment baseline at 30B-A3B, with gains retained after static distributions begin to saturate. In the tool-use setting, SPADE lifts BFCL v4 multi-turn (Patil et al., 2025) by +10.3 at 4B and +5.7 at 30B-A3B, and ACEBench-Agent (Chen et al., 2025a) by +13.9 at 30B-A3B. Ablations show the advantage of hint-based regret over an exponential-moving-average (EMA) learning-potential signal (Kanitscheider et al., 2021; Zhang et al., 2023), and of the full adaptive, corpus-grounded, memory-augmented configuration over the partial and frozen-designer controls. Qualitative analysis reveals emergent curricula: SPADE progresses from simple single-skill tasks to complex multi-constraint environments requiring long-horizon interaction.

In SPADE, the Environment Designer writes a new, self-contained environment and the Reasoning Agent learns by acting in it. Making design a learnable role lets the two co-evolve: as the agent improves, so do the environments it trains on (Figure 2). A single model thus improves not only how it reasons and acts, but the worlds it builds to learn in, a concrete step toward open-ended, continual self-improvement.

2 Related Work

For an extended background and comparison to the literature see Appendix E, summarized here:

Self-Play for LLMs Self-play has driven capability growth from TD-Gammon (Tesauro et al., 1995) through AlphaZero (Silver et al., 2017, 2018) to modern game AI (Vinyals et al., 2019; , FAIR). PowerPlay (Schmidhuber, 2013) proposed a single self-modifying system that continually invents tasks at its own capability frontier, prefiguring the dual-role single-LLM pattern adopted below. Asymmetric self-play (Sukhbaatar et al., 2017) later formalized a proposer-solver paradigm in which one agent sets challenges while another solves them. Applying self-play to LLMs is harder than in single-task game-playing systems: the goal is general capability

across open-ended problems, and self-play training at LLM scale is often unstable. Earlier LLM self-play methods rely on curated seed data or evaluation sets, either for self-distillation (Chen et al., 2024; Yuan et al., 2024; Singh et al., 2023), for adversarial language games over fixed vocabularies (Cheng et al., 2024), or for bootstrapping a learned proposer (Fang et al., 2025b; Sundaram et al., 2026). A more recent line removes this dependency, training proposer and solver from minimal seeds with proxy rewards that target the learnable frontier. AZR (Zhao et al., 2025), SQLM (Chen et al., 2025c), and Language Self-Play (Kuba et al., 2025) have a single LLM play both roles with shared parameters; R-Zero (Huang et al., 2025) and Tool-R0 (Acikgoz et al., 2026) instead train two separate models initialized from the same base, using uncertainty- or solve-rate-based rewards. The closest predecessors to our work are SPIRAL (Liu et al., 2025a), which trains LLMs through self-play on multi-turn zero-sum language games, and SPICE (Liu et al., 2025b), which extends self-play to corpus-grounded question generation. However, both generate *tasks* (a single problem with only a terminal reward) and can collapse when the model’s capacity bounds the reachable solution distribution (Chae et al., 2025) or when proxy rewards induce degenerate outputs (Shafayat et al., 2025); a recent theorem-proving method adds a frozen Guide to score the proposer’s conjectures and curb collapse (SGS, Bailey et al., 2026); in SPADE the verifier is instead part of each generated environment, co-evolving with the **Environment Designer** rather than fixed. By contrast, SPADE generates *full multi-turn MDP environments* (state transitions, reward functions, and verification code) as executable code, trains the **Environment Designer** itself via RL with hint-based regret grounded in measured **Reasoning Agent** return, and spans a broader range of tasks (games and tool use) than prior methods confined to a single domain.

Unsupervised Environment Design and Open-Endedness Curriculum learning and learnability-driven adaptation (Bengio et al., 2009; Schmidhuber, 2013) laid the groundwork for what Dennis et al. (2020) termed unsupervised environment design (UED). POET (Wang et al., 2019) pioneered paired open-ended environment-agent co-evolution via evolutionary search over parameterized terrain, while PAIRED (Dennis et al., 2020) introduced adversarial environment design with minimax regret; subsequent work improved level curation via prioritized replay (Jiang et al., 2021b,a), evolutionary complexity (Parker-Holder et al., 2022), stability (Mediratta et al., 2023), regret approximations (Rutherford et al., 2024), and theoretical foundations (Monette et al., 2025). Güzel et al. (2025) extends prioritized level replay to imagined trajectories from a learned diffusion world model. Multi-agent autotricula provide a complementary route to emergent complexity (Baker et al., 2019), and open-endedness has been argued essential for superhuman AI (Hughes et al., 2024). OMNI-EPIC (Faldor et al., 2024) represents environments as executable code generated by foundation models, with a frozen FM judge of interestingness gating task acceptance. PAPRIKA (Tajwar et al., 2025) brings curriculum-style training to LLM agents across diverse synthetic task groups. However, classical UED searches *parameterized* environments (maze dimensions, grid layouts) within small, fixed design spaces, and curriculum-based LLM training draws from fixed task pools without a learned generator. SPADE carries the regret principle into an unbounded, code-defined environment space: the **Environment Designer** is itself the learning agent, writing full MDPs as programs and co-evolving with the **Reasoning Agent** online, rather than an adversary selecting from a hand-designed parameterization. This turns environment design from selection within a fixed space into open-ended generation.

Environment Synthesis and Scaling Following the shift from single-turn reinforcement learning with verifiable rewards (RLVR) (OpenAI, 2024; DeepSeek-AI, 2025) to multi-turn agentic RL (Wang et al., 2025c; Zhang et al., 2025c), the env-pool bottleneck has motivated work that synthesizes training environments for LLM agents at scales unreachable by hand-curation. Recent systems generate agentic worlds and toolsets (Wang et al., 2026b; Tu et al., 2026; Dong et al., 2026), terminal and computer-use sandboxes (Zhu et al., 2026; Gandhi et al., 2026; Fan et al., 2026; Pi et al., 2026; Xue et al., 2026a,b; Zhang et al., 2026e), and dynamic RL pipelines (Wang et al., 2026a; Guo et al., 2025; Shi et al., 2026); AgentScaler (Fang et al., 2025a) clusters thousands of real APIs into per-domain Python tool environments and trains agents via filtered supervised fine-tuning (SFT), and Eurekaverse (Liang et al., 2024) provides a robotics analogue. Complementary task-level synthesis grounds generation in agent exploration (Ramrakhya et al., 2025), converts unverifiable text into RLVR data (Lu et al., 2026), or expands seed task spaces (Jiang et al., 2025b; Chen et al., 2025e; Li et al., 2025; Lu et al., 2025). Scaling studies show that distributions of verifiable environments lift generalization: WebScale-RL (Cen et al., 2025) mines RL tasks from web pretraining data, AgentRL (Zhang et al., 2025a) unifies multi-turn multi-task training, and RLVE (Zeng et al., 2025) hand-engineers 400 verifiable environments with adaptive difficulty levels. Curriculum selection over fixed pools provides difficulty adaptation without changing the underlying environments (Chen et al., 2025d; Xu et al., 2026a). Across these systems the environment generator is

typically frozen, hand-engineered, or trained on a separate signal. SPADE instead trains the **Environment Designer** online, together with the **Reasoning Agent**, via RL with hint-based regret, produces *full multi-turn MDP environments* as executable Python rather than tasks with sparse terminal rewards, and co-evolves the environment distribution with the **Reasoning Agent**’s capability frontier in a single training loop.

3 Preliminaries

SPADE builds on two standard components, which we review before presenting the method: the Markov decision process with a Gym-style `reset()/step()` interface, used to represent environments, and reinforcement learning from verifiable rewards optimized with GRPO, used to train the policy.

3.1 Markov Decision Processes and Gym-Style Environments

A Markov Decision Process (MDP) is a tuple $(\mathcal{S}, \mathcal{A}, T, R, \rho_0)$ with state space $\mathcal{S}$, action space $\mathcal{A}$, transition function $T(s' | s, a)$, reward function $R(s, a)$, and initial state distribution ρ_0 . The standard programmatic interface for an MDP, popularized by OpenAI Gym (Brockman et al., 2016) and standardized in its Gymnasium fork (Towers et al., 2026), exposes two functions: `reset()` returns an initial observation $s_0 \sim \rho_0$ (with an info dictionary), and `step(a)` advances the state via T and returns $(s', r, \text{terminated}, \text{truncated}, \text{info})$, separating natural termination from time-limit truncation. Our generated environments are fully observed: the observation returned at each step is the state itself. Listing 1 shows a minimal instantiation.

Listing 1 Minimal Gym-style MDP. A Wordle-flavored multi-turn deduction game as a single Python class, in the format SPADE’s **Environment Designer** emits (full generated environments appear in Appendix G.3.2). The episode is stateful (`self.target`, `self.turns_left`), ends either on a correct guess (terminated, reward 1) or at the 6-turn limit (truncated, reward 0); each guess returns per-letter feedback to guide deduction.

```
import random

class WordleEnv:
    WORDS = ["spade", "trace", "lemon", "graph"]  # truncated

    def reset(self, seed=None):  # initial state
        self.target = random.Random(seed).choice(self.WORDS)
        self.turns_left = 6
        return "Guess a 5-letter word in 6 tries.", {}

    def step(self, guess):  # (s', r, term, trunc, info)
        self.turns_left -= 1
        left = [t for g, t in zip(guess, self.target) if g != t]
        fb = ""
        for g, t in zip(guess, self.target):
            if g == t: fb += "G"
            elif g in left: fb += "Y"; left.remove(g)
            else: fb += "_"
        if fb == "GGGGG":
            return fb, 1.0, True, False, {}
        return fb, 0.0, False, self.turns_left == 0, {}
```

3.2 RLVR and GRPO

We train the policy π_θ , an LLM with parameters θ , via reinforcement learning with verifiable rewards (RLVR) using Group Relative Policy Optimization (GRPO) (Shao et al., 2024). For each input prompt x , GRPO samples a group of G responses $\{y_1, \dots, y_G\} \sim \pi_\theta(\cdot | x)$ and computes group-normalized advantages:

$$\hat{A}^i = \frac{r^i - \text{mean}(\{r^j\}_{j=1}^G)}{\text{std}(\{r^j\}_{j=1}^G)} \quad (1)$$

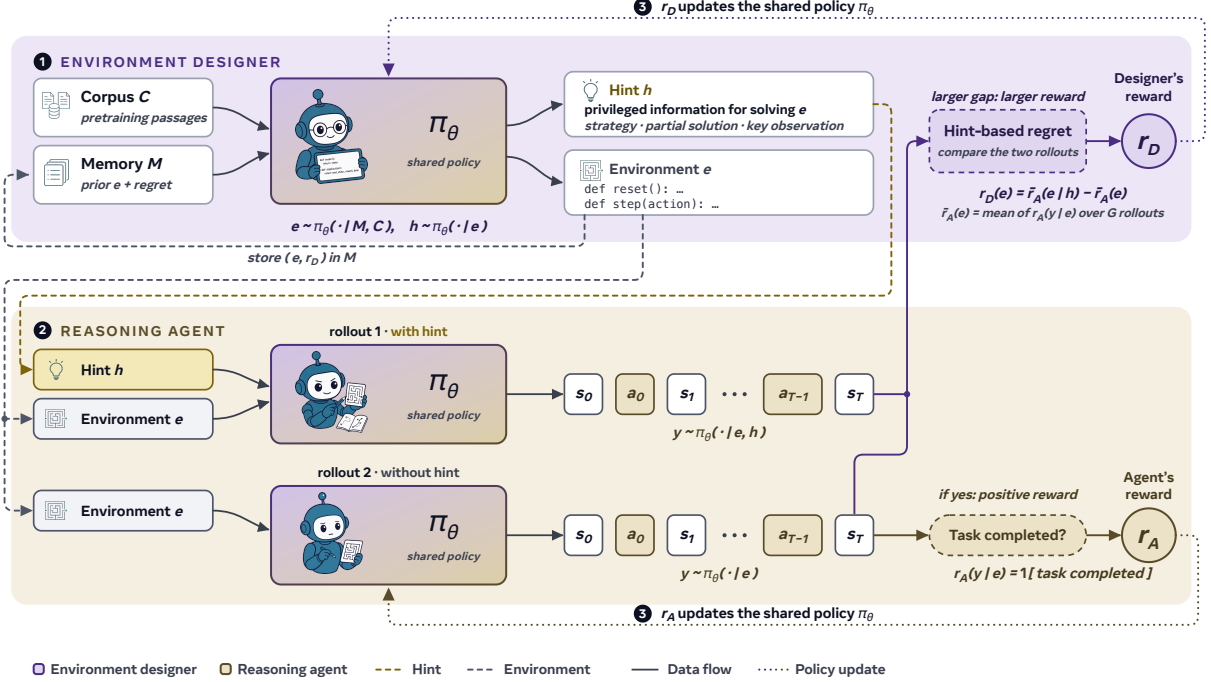


Figure 3 The SPADE framework. Top: the **Environment Designer** conditions on the environment memory M and pretraining corpus C to emit an executable environment e and a privileged hint h . Bottom: the **Reasoning Agent** plays e with and without h ; the return gap is the **Environment Designer**’s hint-based regret $r_D(e)$ (Eq. 3) and task correctness is the **Reasoning Agent** reward. Both rewards update the shared policy π_θ via GRPO.

The policy is updated via clipped policy gradient with KL regularization:

$$\mathcal{L}(\theta) = -\frac{1}{G} \sum_{i=1}^G \min \left(\frac{\pi_\theta(y_i | x)}{\pi_{\text{old}}(y_i | x)} \hat{A}^i, \text{clip} \left(\frac{\pi_\theta(y_i | x)}{\pi_{\text{old}}(y_i | x)}, 1 - \varepsilon_{\text{low}}, 1 + \varepsilon_{\text{high}} \right) \hat{A}^i \right) + \beta_{\text{KL}} \cdot \text{KL}[\pi_\theta \| \pi_{\text{ref}}] \quad (2)$$

4 SPADE: Self-Play in Adaptive Synthetic Executable Environments

SPADE is an end-to-end self-play framework where a single LLM, π_θ , alternates between generating executable training environments and learning to solve them (Figure 3; Algorithm 1). Its three components create an adaptive curriculum: § 4.1 details how dual-role self-play couples environment generation with **Reasoning Agent** (RA) learning; § 4.2 explains how hint-based regret steers the **Environment Designer** (ED) toward environments at the **Reasoning Agent**’s learning frontier; and § 4.3 outlines the pipeline that grounds and validates these executable environments.

4.1 Dual-Role Self-Play and Code-as-Environment

SPADE formulates adaptive environment self-play as a game where a single model π_θ acts in two roles via role-specific system prompts: designing environments (role= D) or solving them (role= A). Let $\mathcal{E}$ denote the space of all valid environments. In the **Environment Designer** role (π_D), π_θ produces an executable environment $e \in \mathcal{E}$ as a Python program implementing the Gym-style `reset()`/`step()` API (Section 3.1; a minimal example is shown in Appendix G.3.1), where the transition function $T(s' | s, a)$ and reward function $R(s, a)$ are both encoded in the `step()` implementation. The **Environment Designer** also emits a privileged hint for each environment. A hint h is task-relevant information that the **Environment Designer** attaches to an environment (for example, a partial solution sketch or a key structural observation); revealing h to the **Reasoning Agent** makes the environment easier to solve, and the gap in **Reasoning Agent** return with versus without h defines the **Environment Designer**’s hint-based regret (Section 4.2).

Algorithm 1 SPADE ♠: Self-Play in Adaptive Synthetic Executable Environments

Require: Pretrained LLM π_θ ; domain prompts $\{p_d\}$; batch size B ; group size G ; iterations N

```
1: for  $n \leftarrow 1$  to  $N$  do
2:   Environment Designer role (role= $D$ ): ▷ generate environments
3:   for  $b \leftarrow 1$  to  $B$  do
4:     Sample domain prompt  $p_d$ ; generate  $e_b \sim \pi_\theta(\cdot \mid p_d, \text{role}=D)$ 
5:     Validate  $e_b$ : syntax, executability; discard invalid
6:     Generate hint  $h_b \sim \pi_\theta(\cdot \mid e_b, \text{role}=D)$  ▷ privileged info
7:   end for
8:   Shared Reasoning Agent rollouts (role= $A$ ): ▷ trained on and scored for regret
9:   for each valid environment  $e_b$  do
10:     $\{y_i\}_{i=1}^G \sim \pi_\theta(\cdot \mid e_b, \text{role}=A)$ ;  $r_A(y_i \mid e_b) \in [-1, 1]$  ▷ no-hint plays
11:     $\{y'_i\}_{i=1}^G \sim \pi_\theta(\cdot \mid e_b, h_b, \text{role}=A)$ ;  $\bar{r}_A(e_b \mid h_b) \leftarrow \frac{1}{G} \sum_{i=1}^G r_A(y'_i \mid e_b, h_b)$  ▷ hint plays
12:   end for
13:   Rewards and joint GRPO update (for each valid  $e_b$  and its rollouts):
14:     $r_D(e_b) \leftarrow \bar{r}_A(e_b \mid h_b) - \frac{1}{G} \sum_{i=1}^G r_A(y_i \mid e_b)$  ▷  $\max(0, \cdot)$  in training
15:     $\hat{A}_D^b \leftarrow r_D(e_b) - \text{mean}_{\text{same skill}}\{r_D(e_c)\}$ ;  $\hat{A}_A^i \leftarrow \frac{r_A(y_i \mid e_b) - \text{mean}(\{r_A(y_j \mid e_b)\}_{j=1}^G)}{\text{std}(\{r_A(y_j \mid e_b)\}_{j=1}^G)}$  ▷ per-role advantages
16:    Update  $\pi_\theta$  by clipped policy gradient on  $\{\hat{A}_D^b\} \cup \{\hat{A}_A^i\}$ 
17:   end for
18: return trained model  $\pi_\theta$ 
```

In the **Reasoning Agent** role (π_A), π_θ interacts with e via sequential actions, receiving observations and rewards. This *code-as-environment* representation unifies single-turn settings (**reset()** $\rightarrow$ **step(answer)** $\rightarrow$ terminal reward) and multi-turn agentic settings (**reset()** $\rightarrow$ **step()** $\rightarrow \dots \rightarrow$ **done**) under a single interface and training pipeline. This representation provides the vast task space that AI-GAs (Clune, 2019) argue is necessary for open-ended self-improvement: any computable MDP can be expressed as a Python program, so the **Environment Designer** can express any such environment in code rather than being confined to a hand-designed environment parameterization. Both roles share parameters θ , so updates for either role affect the same policy. Each self-play cycle first collects **Environment Designer** trajectories, and then **Reasoning Agent** trajectories from those generated environments. The **Environment Designer** receives hint-based regret (Section 4.2), whereas the **Reasoning Agent** receives task correctness from the environment’s reward function. Each candidate environment is validated for syntactic correctness and executability before entering the training pool; validation details are in Appendix C.2.

Stabilizing joint two-role training. Optimizing a single policy for two coupled objectives requires several stabilizing techniques (Liu et al., 2026a). Following SPIRAL (Liu et al., 2025a), we normalize advantages independently for each role. We standardize **Reasoning Agent** returns within each environment’s rollouts and mean-center **Environment Designer** rewards within each skill. To ensure both roles contribute comparably, we upweight the less frequent **Environment Designer** trajectories. Finally, we delay the **Environment Designer** update by k rollouts, where k is the number of training steps per environment set, so the difficulty anchor can score each environment’s **Reasoning Agent** win rate over the full training window; the regret component is computed at generation time. Since this delay makes the **Environment Designer** objective off-policy, we correct the gradient using truncated importance sampling (Yao et al., 2025). We use an asymmetric clipping range ($\varepsilon_{\text{low}}=0.2$, $\varepsilon_{\text{high}}=0.28$) to preserve exploration and floor the raw regret at zero. The deployed **Environment Designer** reward blends this floored regret, normalized to $[0, 1]$ by a fixed scale, (weight 0.4) with a flat-top difficulty anchor that pays environments whose **Reasoning Agent** win rate falls in a target band and decays linearly outside it (weight 0.6, band $[0.4, 0.6]$): the anchor regulates difficulty, and floored regret selects the most teachable environments within the band. Full hyperparameters appear in Appendix C.2.

4.2 Hint-Based Regret Reward

The **Environment Designer**’s reward for producing environment e is:

$$r_D(e) = \bar{r}_A(e \mid h) - \bar{r}_A(e), \quad (3)$$

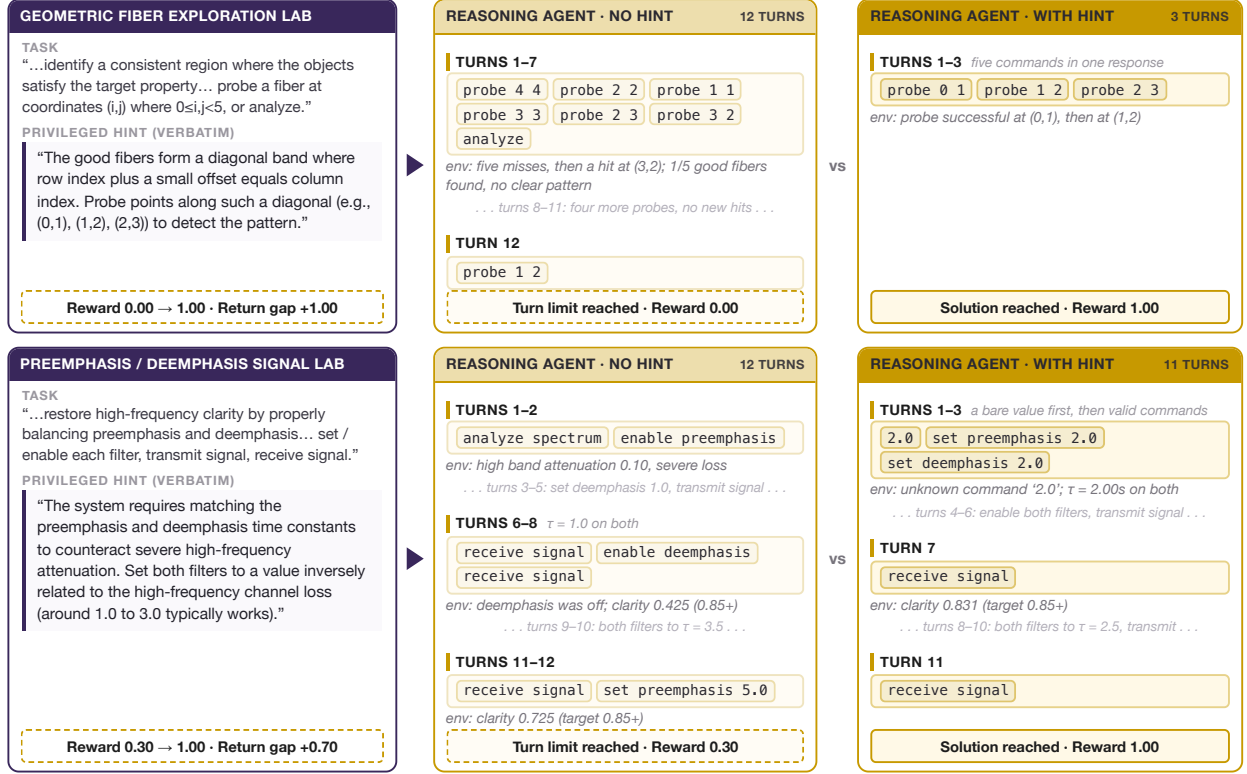


Figure 4 How a privileged hint changes Reasoning Agent play. Two positive-regret examples from the canonical 30B games run. Left: each environment’s task prompt and privileged hint, quoted verbatim (ellipses mark elided text; the standardized answer-format sentence is omitted from the hint). Right: one logged Reasoning Agent rollout per arm, condensed while preserving action order, feedback, and values; elided turns are marked and named. The two arms are independent plays with independently seeded resets, so board layouts and probe outcomes differ across arms. The dashed box on each environment card reports the two displayed rollout returns and their single-pair gap; the Environment Designer reward in Equation 3 is instead the difference of the arm means over all logged rollouts (0.00 → 1.00 for the fiber task, 0.30 → 0.65 for the audio task). An expanded task–hint set appears in Appendix G.1.1.

where $\bar{r}_A(e | h) = \frac{1}{G} \sum_{i=1}^G r_A(y'_i | e, h)$ is the Reasoning Agent’s average return over G fresh rollouts y'_i sampled with the privileged hint h in context, and $\bar{r}_A(e)$ is the corresponding average over G rollouts without it. Three regimes emerge: high regret indicates an environment at the learning frontier (solvable with hints but not without); low regret where the Reasoning Agent succeeds with and without the hint indicates mastery; and low regret with low returns even with the hint indicates an intractable environment. This implements the minimax regret objective of PAIRED (Dennis et al., 2020) without a separate antagonist: the hint-equipped Reasoning Agent serves as the upper-bound policy. For a policy that best-responds to the hint, extra information cannot reduce expected return, so regret is non-negative at the optimum; for the current policy, expected regret can dip below zero when hints mislead it, which we observe for the smaller backbones (Figure 12). We formalize this intuition and show that, under idealized assumptions, every pure Nash equilibrium yields hint-free optimality on every valid environment (Appendix B). We compare hint-based regret against an EMA-based learning-potential signal (Kanitscheider et al., 2021; Zhang et al., 2023) in the Environment Designer-reward ablation (Section 7.2).

Hint generation. For each environment e , the Environment Designer (same LLM, different prompt) emits one task-specific hint of a few sentences. The prompt asks for the key insight or strategy and the expected answer format, but forbids revealing the exact answer; its output is unlabeled hint text. Figure 4 shows two high-regret examples. Each hint is shown beside the task it conditions and one logged Reasoning Agent

Table 1 Training the Reasoning Agent on diverse synthetic games improves held-out reasoning and code benchmarks at every backbone scale. The eight held-out benchmarks probe four capability families: competition math (AIME 2025/2026, Avg@32), science reasoning (GPQA-Diamond, accuracy), code generation (LiveCodeBench-v6, Pass@1), and procedural reasoning across four cognitive skills (Reasoning-Gym, win rate at HARD). Both fixed-environment baselines are retrained per backbone from the same base model for 400 training iterations; Fixed-env RLVE follows the official RLVE sampling and curriculum settings. Avg is the unweighted mean over the eight benchmarks; green subscripts denote absolute pp gain over the same-model base. Best per column within each backbone block in **bold**, second best underlined.

Model	AIME'25	AIME'26	GPQA-D	LCB-v6	Reasoning-Gym (Stojanovski et al., 2026)				Avg	Δ vs Base
	(MAA)	(MAA)	(Rein et al., 2023)	(Jain et al., 2024)	RG-Math	RG-Algo.	RG-Cog.	RG-Logic		
Qwen3 backbones: fixed-environment baselines and SPADE (game environment design)										
Qwen3-4B-Instruct-2507	47.4	58.3	55.9	35.1	31.6	11.8	16.4	54.7	38.9	–
Fixed-env GRPO	47.1	58.6	56.2	35.4	34.0	13.5	18.1	56.3	39.9	+1.0
Fixed-env RLVE	49.6	62.1	57.3	35.9	38.6	16.3	21.0	59.1	42.5	+3.6
+ SPADE (Games)	48.9+1.5	60.2+1.9	58.1+2.2	37.2+2.1	44.6+13.0	19.8+8.0	23.1+6.7	60.8+6.1	44.1	+5.2
Qwen3-8B	67.1	71.2	59.4	46.3	47.2	19.6	24.8	63.1	49.8	–
Fixed-env GRPO	67.4	71.0	59.9	46.8	49.8	21.9	26.5	64.6	51.0	+1.2
Fixed-env RLVE	69.6	75.0	61.6	47.4	52.7	25.0	31.0	67.9	53.8	+3.9
+ SPADE (Games)	68.8+1.7	73.1+1.9	62.9+3.5	49.4+3.1	57.3+10.1	29.2+9.6	33.8+9.0	69.7+6.6	55.5	+5.7
Qwen3-30B-A3B-Instruct-2507	61.5	73.5	70.4	43.2	45.0	18.0	23.0	67.0	50.2	–
Fixed-env GRPO	61.2	73.8	70.9	43.7	48.1	20.3	24.6	68.4	51.4	+1.2
Fixed-env RLVE	56.9	69.8	69.8	42.5	55.8	24.7	30.9	73.7	53.0	+2.8
+ SPADE (Games)	62.8+1.3	74.4+0.9	75.8+5.4	47.3+4.1	63.3+18.3	32.1+14.1	37.7+14.7	72.8+5.8	58.3	+8.1

rollout from each prompt condition. The paired trajectories expose the behavioral source of the return gap: the hint hands the fiber-task agent the probe pattern almost outright, and narrows the audio-task agent’s search to a parameter range it would otherwise reach only after repeated failed attempts.

4.3 Environment Design

Corpus grounding. A generator conditioned only on its own output has no source of novelty outside its own weights, so it narrows toward the patterns it already favors and mode-collapses, the “invisible leash” (Chae et al., 2025; Zhang et al., 2026c; Jiang et al., 2025a). We treat an external corpus as the mechanism that lengthens that leash: *every* round the **Environment Designer** conditions on freshly sampled human corpus. In the games setting these are 10k mathematics and 5k science documents drawn from DCLM (Li et al., 2024) and MegaScience (Fan et al., 2025), spanning web pages, physics forums, and university-level scientific textbooks; in the tool-use setting, 15k documents from the Nemotron pretraining code corpus (NVIDIA, 2026), which supplies algorithm implementations and API documentation. SPICE (Liu et al., 2025b) established corpus grounding for *task* generation, mining documents for reasoning questions; SPADE carries the principle to *environment* generation, where a sampled passage seeds an executable MDP rather than a question-answer pair. Section 6.2 (Figure 7) shows how corpus grounding sustains environment diversity.

Environment memory. The corpus supplies breadth from outside the loop; the cross-episode memory keeps the **Environment Designer** from re-posing what the **Reasoning Agent** has already mastered (Schmidhuber, 2013). A buffer of previously generated environments, annotated with regret scores and skill tags, gives the **Environment Designer** high-regret seeds to vary and too-easy or too-hard examples to avoid, so each round starts from what the **Reasoning Agent** currently finds hard rather than from scratch. Retaining and reusing past experience this way is the mechanism agentic-memory systems rely on to keep improving without further gradient updates (Zhang et al., 2026d; Xiong et al., 2026); here it acts on the design side, holding generated difficulty at the **Reasoning Agent**’s frontier as that frontier moves. The two inputs work on different axes, the corpus on what environments are about and the memory on how hard they are, and Table 3 separates their contributions.

Environment pool management, lifecycle, and quality control details are in Appendix C.2.

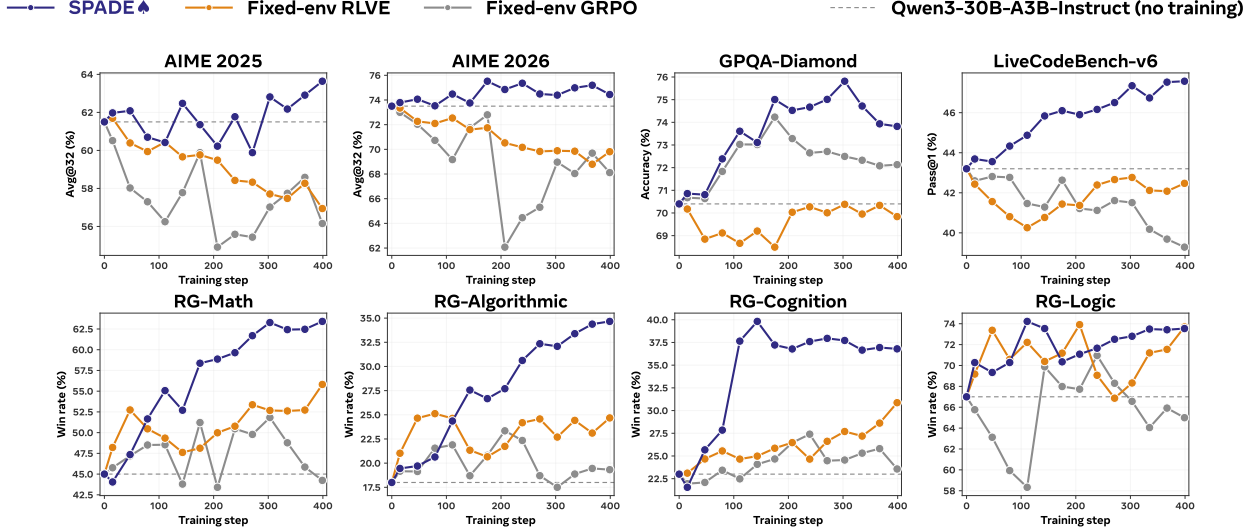


Figure 5 Training on diverse synthetic games improves science reasoning, code generation, and procedural reasoning while competition math is preserved (games setting, Qwen3-30B-A3B-Instruct-2507). Top row: competition math (AIME 2025/2026 Avg@32), science reasoning (GPQA-Diamond accuracy), and code generation (LiveCodeBench-v6 Pass@1). Bottom row: procedural reasoning across four cognitive skills (Reasoning-Gym win rate at HARD). Markers denote logged evaluation checkpoints; the dashed line marks the untrained base model.

5 Experimental Setup

5.1 SPADE Training Recipe

We train three Qwen3 backbones: Qwen3-4B-Instruct-2507, Qwen3-8B, and Qwen3-30B-A3B-Instruct-2507, the last of which is our *primary* model. The 4B and 30B runs use instruct models; the 8B run enables thinking for both the **Environment Designer** and the **Reasoning Agent**. We train with GRPO (Shao et al., 2024) for 400 rollouts of 24 environments each. The **Environment Designer** regenerates the environment set every k rollouts and its update is delayed by the same k (Section 4.1), so k sets how long the **Reasoning Agent** trains on a fixed set before the curriculum moves. The **Reasoning Agent** plays every environment $16 \times k$ times without the hint and 16 times with it; the regret subtracts hint and no-hint averages measured at the same regeneration step (16 plays each). The **Environment Designer** writes the privileged hint itself, with no external model, and each round it also draws on a memory of past environments, both high-regret seeds and too-easy/too-hard negatives. Rewards are normalized per environment, and the stabilizers described earlier (per-role advantages, the delayed **Environment Designer** update with truncated importance sampling, and the regret floor) are unchanged across settings. Our RL backbone is **slime** (Zhu et al., 2025).

The settings below differ in what the **Environment Designer** generates, how it is grounded, how the **Reasoning Agent** acts, and the regeneration interval k . Every benchmark is held out from training, and component-level controls such as freezing the **Environment Designer** or removing grounding are covered by the ablations of Section 7. Full hyperparameters, per-run adjustments, and evaluation protocol details are in Appendices C.2 and C.3.

5.2 Domain: Games

Environments. The **Environment Designer** writes each environment as a self-contained Python game with a verifiable reward, and the **Reasoning Agent** solves it. Each rollout uses 24 games, eight for each of three active skills; the six cognitive-skill categories (Mathematical Reasoning, Logical Deduction, Spatial Reasoning, Pattern Recognition, Optimization, and Causal Inference) rotate three at a time, with $k=4$. The generation is grounded in the 15k-document math and science corpus of Section 4.3. Besides floored hint-based regret, the **Environment Designer** reward includes the flat-top difficulty anchor of Section 4.1 that pays games whose

Table 2 Synthetic tool-use environments match dedicated data-synthesis systems and surpass them where multi-step interaction matters most. Per-domain results on BFCL v4 multi-turn (Patil et al., 2025), τ^2 -bench (Barres et al., 2025), and ACEBench-Agent (Chen et al., 2025a). Reference rows are transcribed from the cited papers. For our rows, Avg is the unweighted mean of the shown subcolumns and the final Avg averages the three benchmarks, computed before rounding. For reference rows, Avg is likewise the unweighted mean of the shown subcolumns, or the cited paper’s own aggregate where the subcolumns are not reported; Agent-World and AWM print τ^2 aggregates of 61.8/65.4 and (task-weighted) 33.5/39.0, and we print the means of the shown domains for cross-row consistency. The final Avg is omitted because each reference system skips at least one benchmark. ‘-’ = not reported by the cited paper. SPADE rows in **bold**.

Model	BFCL v4 (multi-turn)					τ^2 -bench				ACEBench-Agent				Δ
	Base	Miss Func	Miss Param	Long Ctx	Avg	Retail	Airline	Telecom	Avg	Multi Step	Multi Turn	Avg	Avg	
Synthetic-environment agents (per-split numbers as reported by the cited papers)														
AgentScaler-30B-A3B (Fang et al., 2025a)	–	–	–	–	–	70.2	60.0	55.3	61.8	–	–	60.0		
Agent-World-8B (Dong et al., 2026)	–	–	–	–	44.5	72.8	40.0	50.9	54.6	–	–	–		
Agent-World-14B (Dong et al., 2026)	–	–	–	–	53.9	74.5	52.0	56.1	60.9	–	–	–		
AWM-8B (Wang et al., 2026b)	–	–	–	–	45.0	41.2	38.5	23.5	34.4	–	–	–		
AWM-14B (Wang et al., 2026b)	–	–	–	–	51.9	63.6	31.5	17.8	37.6	–	–	–		
EnvScaler-4B (Song et al., 2026)	51.0	34.0	28.0	39.0	38.0	–	–	–	–	80.0	61.1	70.6		
EnvScaler-8B (Song et al., 2026)	55.5	36.0	35.0	41.0	41.9	–	–	–	–	85.0	60.0	72.5		
Ours: SPADE post-training on Qwen3 backbones (tool-use environment design)														
Qwen3-4B-Instruct-2507	34.0	16.0	12.5	25.5	22.0	43.0	32.0	18.0	31.0	55.0	41.7	48.4	33.8	
+ SPADE 🧠 (Tool Use)	46.0	26.5	22.0	34.7	32.3 _{+10.3}	47.2	35.6	21.5	34.8 _{+3.8}	65.0	49.5	57.3 _{+8.9}	41.4	+7.7
Qwen3-8B	52.0	30.0	24.0	35.6	35.4	34.0	26.5	18.0	26.2	63.3	56.7	60.0	40.5	
+ SPADE 🧠 (Tool Use)	58.0	36.0	30.0	43.2	41.8 _{+6.4}	37.8	29.5	21.2	29.5 _{+3.3}	73.0	65.0	69.0 _{+9.0}	46.8	+6.2
Qwen3-30B-A3B-Instruct-2507	66.0	44.0	38.0	48.0	49.0	62.0	50.0	35.0	49.0	70.0	54.0	62.0	53.3	
+ SPADE 🧠 (Tool Use)	72.0	50.0	44.0	52.9	54.7 _{+5.7}	65.5	53.5	38.8	52.6 _{+3.6}	82.0	69.8	75.9 _{+13.9}	61.1	+7.7

Reasoning Agent win rate falls in the target band, and every game must pass syntax and execution checks before it enters the pool.

Baselines and evaluation. We retrain two fixed-environment baselines per backbone from the same base model for 400 training iterations: *Fixed-env RLVE*, GRPO on the official RLVE set (Zeng et al., 2025), and *Fixed-env GRPO*, GRPO on static environments generated by GPT-5.5. Fixed-env RLVE follows the official RLVE sampling and curriculum settings. RLVE is the stronger of the two throughout Section 6. Evaluation spans two distances from the training distribution: *procedural reasoning*, the four Reasoning-Gym (Stojanovski et al., 2026) categories at HARD difficulty, and *out-of-distribution (OOD) reasoning and code*, AIME 2025/2026 (Avg@32) (MAA), GPQA-Diamond (accuracy) (Rein et al., 2023), and LiveCodeBench-v6 (Pass@1) (Jain et al., 2024). Table 1 reports all eight for every backbone.

5.3 Domain: Tool Use

Environments. The Environment Designer instead writes an executable tool-use environment: a set of simulated tools in OpenAI function-calling format, a backend state the tools modify, and three to five natural-language user instructions that arrive one at a time, each with a check on the resulting state. Generation is grounded in the 15k-document code corpus, and tool environments cost more to generate, so we used $k=8$. The Reasoning Agent solves an environment by calling tools over multiple turns and is rewarded only when it completes every user instruction. The privileged hint is a step-by-step plan (look up a record, update it, then reconcile the result), so the Reasoning Agent still has to find the exact arguments by calling the tools. Validation adds two checks beyond the game ones: a deterministic reset gate that exercises every success criterion under several seeds, and an LLM check that every success criterion can be met by some tool (details in Appendix C.2). The generation prompt targets the multi-turn function-calling task family of the evaluation suites (schema-defined simulated tools, a backend state, and per-instruction checks); no benchmark tasks or data are shown to the Environment Designer.

Baselines and evaluation. We compare SPADE with four synthetic-environment systems: AgentScaler (Fang et al., 2025a), Agent-World (Dong et al., 2026), Agent World Model (AWM) (Wang et al., 2026b), and EnvScaler (Song et al., 2026). Their results are transcribed from the cited papers and differ from ours in training data and budget, in some cases base model, and in evaluation protocol (Appendix C.3.1). Evaluation uses BFCL v4 multi-turn (Patil et al., 2025), τ^2 -bench (Barres et al., 2025), and ACEBench-Agent (the Agent category of ACEBench-en) (Chen et al., 2025a), reported per domain in Table 2.

6 Experimental Results

We evaluate SPADE on two settings, game and tool-use environment design, across the three Qwen3 backbones of Section 5.1. Section 6.1 reports held-out benchmark performance against the fixed-environment baselines, and Section 6.2 examines what the **Environment Designer** and **Reasoning Agent** produce over training.

6.1 Quantitative Analysis

Synthetic game environments improve science, code, and procedural reasoning. At 30B-A3B, SPADE reaches a suite average of 58.3: +8.1 over base and +5.3 over the strongest fixed-environment baseline (Fixed-env RLVE), a margin that grows with model size (Table 1). The **Environment Designer** trains on synthetic games, never on any held-out task, yet the gains transfer to science, code, and procedural reasoning and hold late in the 400-step run (Figure 5), while competition math is preserved. They concentrate on procedural reasoning, where every cognitive-skill category improves: the generated games exercise each skill through many problem structures rather than a fixed task set.

Synthetic tool-use environments improve multi-step interaction. The same recipe applied to tool-use environment design improves every backbone (Table 2), and at 30B-A3B the size of the gain tracks how closely a benchmark’s task structure matches the generated environments: ACEBench-Agent gains most (+13.9); its stateful, multi-step tasks mirroring the generated pattern of a database, a tool schema, and a multi-call goal, followed by BFCL v4 multi-turn (+5.7, and +10.3 at 4B) and τ^2 -bench (+3.6). SPADE at 30B-A3B leads the dedicated data-synthesis systems on both BFCL v4 multi-turn and ACEBench-Agent, so structural training signal transfers where domain-specific data collection does not reach.

6.2 Qualitative Analysis

The Environment Designer keeps supplying environments the Reasoning Agent can learn from. We examine what each role produces over the main 30B games run. Figure 6 tracks the learnable share of each rollout, the fraction of generated environments on which the **Reasoning Agent** wins between 20% and 80% of the time. Full SPADE raises this share over training, reaching roughly a third late in the 400-step run; the corpus, the memory, and **Environment Designer** training together sustain that supply. Figure 7 traces the diversity of that supply to the corpus: corpus grounding keeps the environments diverse (Vendi/ n 0.68, versus 0.04 without it), and a control that freezes the **Environment Designer** and removes memory while keeping the corpus retains full diversity (0.70, Appendix F.1.2). Corpus grounding provides the breadth, and **Environment Designer** training increases the difficulty. Iverson (2026) reaches the same conclusion from the failure side: an ungrounded proposer collapses onto a handful of near-identical programs, explicit diversity rewards get hacked once they are optimized against, and conditioning on an external corpus is the intervention that holds longest. Figure 7 shows that corpus grounding sustains diversity, whereas the ungrounded run collapses.

Environment Designer training makes environments harder in measurable, code-level ways. The generation instructions are fixed across all 400 steps (the same template asking for hidden state, multi-turn structure, and partial reward), while the grounding document is resampled from the corpus each step with no trend over training, so any systematic change in the generated environments traces to **Environment Designer** training rather than to the prompt. Two measurements move together. First, within Physics, the share of environments whose opening observation prints the governing formula falls from 25% to 5% over the 473 Physics environments of the canonical run (Figure 8). Second, rewards become more finely graded, from 3.7 to 5.8 distinct levels per environment, of which 2.2 $\rightarrow$ 4.0 are strictly partial (Appendix F.1.1). The no-corpus run (w/o corpus grounding in Table 3) shows none of this: over steps 290–312 it emits the same rotating-maze task 41 times in a row.

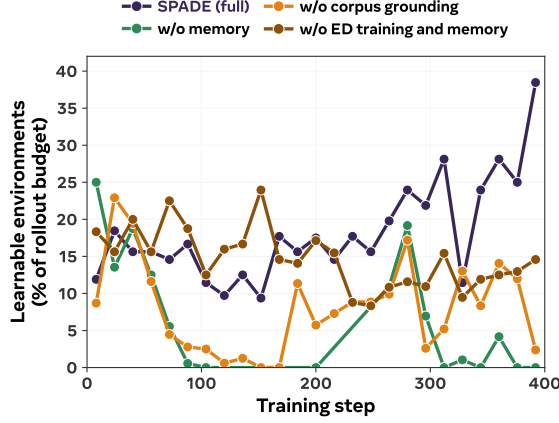


Figure 6 Full SPADE raises the learnable share of its environment budget to roughly a third by the end of training; component ablations decline or collapse. Share of each rollout’s 24 environments that is *learnable*, defined as Reasoning Agent win rate in $[0.2, 0.8]$ and weighted by the number of valid environments generated in each 16-step window. Matched 30B-A3B settings over a common 400-step budget; unfilled rollout capacity contributes zero by construction.

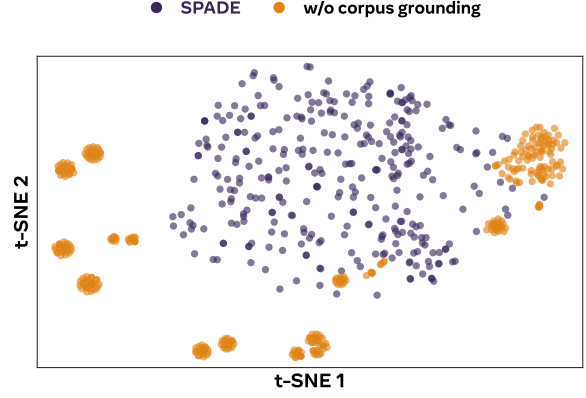


Figure 7 Corpus grounding sustains environment diversity: Vendi/ n is 0.68 with the corpus and 0.04 without. t-SNE of SBERT-embedded environments (330 sampled per run) from SPADE and the no-corpus ablation. Vendi/ n counts effective distinct environments per 100. The projection is illustrative; the quantitative comparison rests on the calibrated Vendi score (Appendix F.1.2).

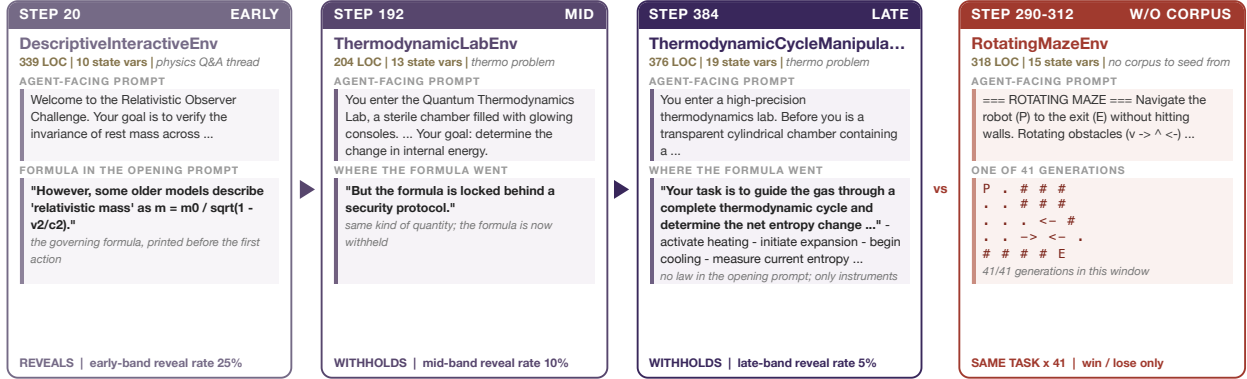


Figure 8 Trained Environment Designer environments stop revealing the solution method in the prompt. Physics environments from one 30B-A3B run at steps 20, 192, and 384; the step-20 environment retains the scaffold’s default class name. The formula-reveal rate (percentage on each panel) falls from 25% to 5% over 473 environments. Rightmost (red): over steps 290–312 the no-corpus ablation emits the same `RotatingMazeEnv` task 41 consecutive times. The complete source of the step-384 environment, together with one further exemplar, appears in Appendix G.3.2.

The Reasoning Agent learns to act on evidence rather than derive in advance. Figure 9 shows the Reasoning Agent’s side of the same run. At step 0 it reasons entirely up front and cannot recover once the interface rejects its answer; by step 200 it tests short hypotheses and revises them as results return; by step 300 it gathers evidence first and derives once, when the evidence is enough. The shift appears only where the task rewards inference: on procedural environments the same policy acts in short commands (late-step median 8 tokens). The model still derives at length when the task calls for it, and the late-checkpoint benchmark gains (Table 1) confirm it keeps the ability.

Table 3 The full adaptive configuration outperforms every partial and frozen-designer control. Games setting, Qwen3-30B-A3B-Instruct-2507. Best checkpoint per variant on suite average; Avg is the unweighted mean over the same eight benchmarks as Table 1; full trajectories in Figure 10. Best in **bold**, second best underlined.

Setting	Components				Benchmarks								Avg
	ED design	ED trained	Corpus grounding	Env. memory	AIME'25	AIME'26	GPQA-D	LCB-v6	RG-Math	RG-Algo.	RG-Cog.	RG-Logic	
Qwen3-30B-A3B-Instruct-2507	–	–	–	–	<u>61.5</u>	73.5	70.4	43.2	45.0	18.0	23.0	67.0	50.2
SPADE ♠	Self	✓	✓	✓	62.8	<u>74.4</u>	75.8	47.3	63.3	32.1	37.7	72.8	58.3
w/o memory	Self	✓	✓	✗	59.3	75.0	72.3	45.7	49.1	22.9	30.7	<u>70.9</u>	53.2
w/o corpus grounding	Self	✓	✗	✓	61.1	74.1	71.8	<u>46.3</u>	<u>51.6</u>	22.3	<u>32.4</u>	68.7	<u>53.5</u>
w/o ED training and memory	Self	✗	✓	✗	59.4	73.5	65.8	39.1	22.5	10.0	7.6	46.0	40.5
Fixed Environment Designer (GPT-5.5)	GPT-5.5	✗	✓	✓	59.9	72.8	<u>74.2</u>	42.6	51.2	<u>24.3</u>	30.7	68.0	53.0

7 Ablations

We ablate two design choices for the **Environment Designer**: whether it adapts during training (Section 7.1), and how it is rewarded (Section 7.2). Table 3 covers the ablation for training settings and environment sources. The first two settings remove corpus grounding and the environment memory, one at a time. The remaining two variants freeze the **Environment Designer**: one also drops the environment memory, and the other replaces the self-play **Environment Designer** with a fixed frontier model (GPT-5.5) that keeps both inputs. Unlike Fixed-env GRPO’s static pre-generated pool, this control uses a frontier model as the environment designer. All variants use games on Qwen3-30B-A3B-Instruct-2507 and are evaluated on the same eight-benchmark suite as the main experiments. Each variant reports its best checkpoint on the suite average; full eval curves are in Figure 10, and per-ablation breakdowns in Appendix D.

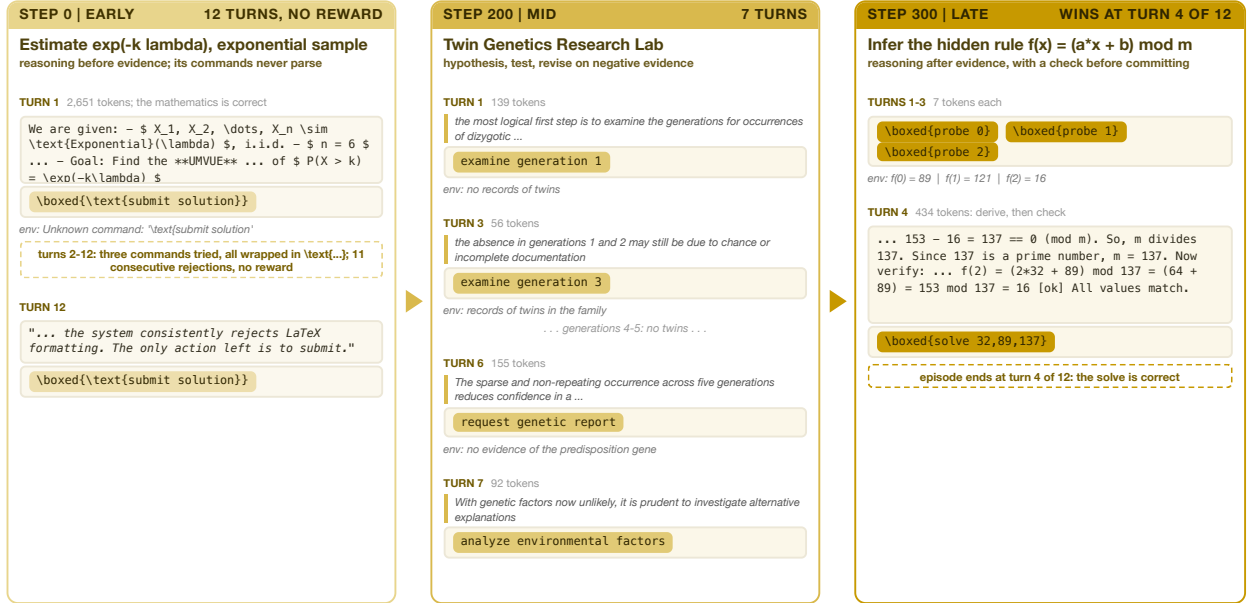


Figure 9 From front-loaded derivation to evidence-first interaction. Reasoning Agent episodes from one 30B-A3B run at steps 0, 200, and 300. At step 0 the agent derives in advance and cannot recover from format errors; by step 200 it tests short hypotheses and revises on evidence; by step 300 it probes first and derives once. Benchmark gains of late checkpoints (Table 1) confirm the model keeps its long-form derivation ability. Transcripts verbatim (math glyphs transliterated to ASCII; environment feedback abridged); token counts use the backbone’s tokenizer.

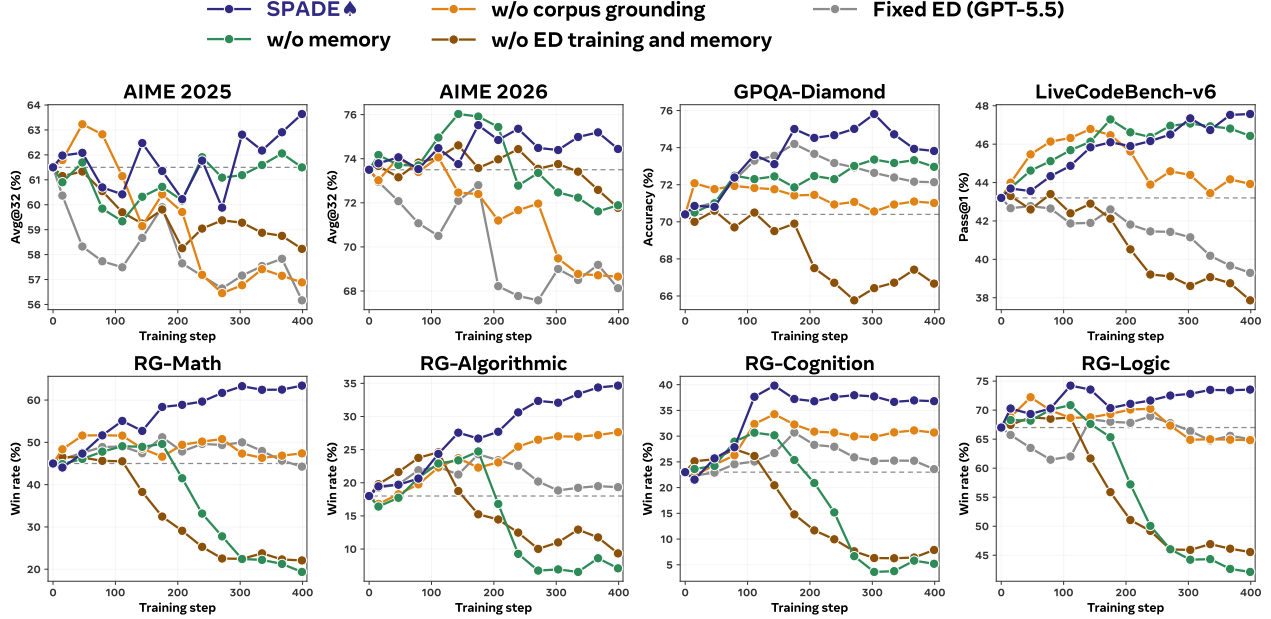


Figure 10 Removing Environment Designer training and memory together drops self-play below base; removing either one alone has an above-base selected checkpoint but peaks early and can fall below base late. One curve per variant of Table 3 (Qwen3-30B-A3B-Instruct-2507, games setting). Top row: AIME 2025/2026 Avg@32, GPQA-Diamond accuracy, and LiveCodeBench-v6 Pass@1; bottom row: the four Reasoning-Gym categories; the dashed line marks the untrained base model.

7.1 Ablation: Environment Designer Adaptation

Co-adaptation between the two roles is what drives the gains: the curriculum and the downstream improvements come from the **Environment Designer** and **Reasoning Agent** training together, and freezing the designer loses them. Freezing the self-play **Environment Designer** and dropping its memory makes the model worse than no training at all, with the eight-benchmark average falling to 40.5, 9.7 points *below* the untrained base. A fixed GPT-5.5 designer that keeps corpus grounding and memory does better, beating the base (53.0 versus 50.2), but recovers only about 35% of SPADE’s +8.1 gain and does not improve code (LiveCodeBench-v6 42.6 versus 43.2). The partial and frozen variants peak early and then fade (the no-corpus run near step 111, the GPT-5.5 designer near step 175), while full SPADE stays strongest late in training (Figure 10). Each control changes more than one thing at once, freezing the designer and also removing its memory or swapping in a different model, so together they show that the full co-adaptive setup wins without attributing the gain to gradient updates by themselves.

7.2 Ablation: Environment Designer Reward

SPADE’s **Environment Designer** reward is built on hint-based regret (Eq. 3), blended with the difficulty anchor of Section 4.1. The alternative below is cheaper: it reuses the **Reasoning Agent** rollouts already collected and needs no hinted replay.

EMA-based learning potential. This reward design (Kanitscheider et al., 2021; Zhang et al., 2023) scores an environment by how far the **Reasoning Agent**’s success on it departs from its skill’s recent average. Environments are grouped by skill s . Let $\bar{r}_A(e)$ be the **Reasoning Agent**’s average return on environment e (Eq. 3), and $\bar{r}_{A,t}(s)$ its mean over the skill’s environments in scoring round t . Two moving averages of $\bar{r}_{A,t}(s)$ are tracked at rates $\gamma_1 > \gamma_2$:

$$\mu_t^\gamma(s) = (1 - \gamma) \mu_{t-1}^\gamma(s) + \gamma \bar{r}_{A,t}(s), \quad \mu_{\text{fast}} \equiv \mu^{\gamma_1}, \quad \mu_{\text{slow}} \equiv \mu^{\gamma_2}. \quad (4)$$

Here γ weights the new observation, so μ_{fast} adapts faster than μ_{slow} , and μ^γ has half-life $\log(0.5)/\log(1 - \gamma)$

rounds. The reward uses μ_{slow} as the skill baseline:

$$\rho(e) = |\bar{r}_A(e) - \mu_{\text{slow},t}(s)|, \quad (5)$$

$$r_D^{\text{LP}}(e) = \rho(e) - \frac{1}{|\mathcal{E}_s|} \sum_{e' \in \mathcal{E}_s} \rho(e'), \quad (6)$$

where $\mathcal{E}_s$ is the round’s environment set for skill s .

Two consequences follow. The deviation is unsigned, so an environment the **Reasoning Agent** always solves can score as highly as one it never solves when the slow mean sits mid-range, though neither gives gradient; only a variance-style bonus such as $\bar{r}_A(e)(1 - \bar{r}_A(e))$ would favour mixed outcomes. And μ_{fast} is tracked but unused: the classical gap $|\mu_{\text{fast}} - \mu_{\text{slow}}|$ is logged only as a diagnostic. The signal also needs a per-skill history before it is meaningful, and cannot separate deviation caused by environment design from **Reasoning Agent** drift or sampling noise.

Results. Figure 11 shows the trajectories for the two trained-**Environment Designer** rewards and the frozen control on the six-benchmark trajectory average, while Table 6 gives their selected checkpoints on the full eight-benchmark suite. Hint-based regret lifts the eight-benchmark average from 50.2 to 58.3 (+8.1); EMA-based learning potential reaches around 70% of that gain (+5.7, to 55.9) and climbs more slowly, the two signals separating after the first ~ 50 steps. Both stay far clear of the jointly frozen-**Environment Designer**/no-memory control, whose selected checkpoint is 9.7 points *below* the untrained model. The comparison between the two trained-**Environment Designer** variants isolates the reward choice, the regret-based blend versus the standalone learning-potential signal; the frozen comparison does not isolate **Environment Designer** training from memory. The form of Eq. 6 helps explain the remaining reward gap: an unsigned deviation from a per-skill running mean needs history before it means anything, and even then scores mastered and hopeless environments alike, so it finds the frontier later and less sharply than a hint gap measured on the current policy. Per-benchmark numbers are in Table 6.

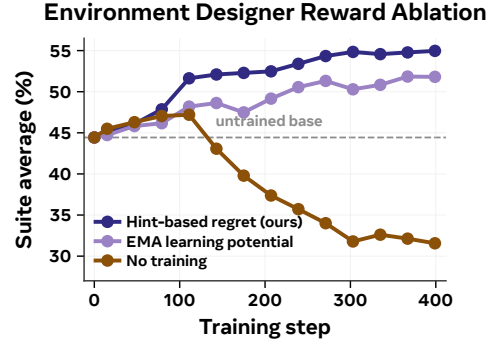


Figure 11 Rewarding the Environment Designer matters more than which reward is used, but regret still leads. Averaged trajectories over GPQA-Diamond, LiveCodeBench-v6, and the four Reasoning-Gym categories (Qwen3-30B-A3B-Instruct-2507, games setting); the dashed line marks the corresponding untrained base. Eight-benchmark checkpoint results are reported in Table 6.

8 Scaling Results

Scaling model size. Under a shared training recipe (Section 5.1), the average gain over each backbone’s base grows from +5.2 at 4B and +5.7 at 8B to +8.1 at 30B-A3B, while Fixed-env GRPO stays near +1.2 at every size (Figure 12; Table 1). Static environments are a fixed training signal that larger models fit quickly and stop learning from; the **Environment Designer**’s adaptive curriculum keeps generating environments at the frontier of the **Reasoning Agent**’s ability, and larger models benefit more from it.

Scaling curriculum diversity. The cognitive-skill curriculum is the set of skill categories the **Environment Designer** targets when it generates environments: the six listed in Section 5.2, three active per regeneration in round-robin. We compare the full six-skill curriculum against a two-skill version, with everything else held fixed (Figure 13; per-benchmark panels in Figure 14, Appendix D). The two-skill run also improves, but it captures only about half of the Reasoning-Gym gains and much smaller GPQA-Diamond and LiveCodeBench-v6 gains

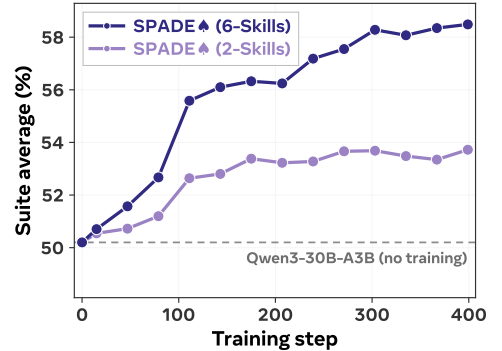


Figure 13 Curriculum breadth accounts for most of the gain. Suite average (eight benchmarks of Table 1), six-skill vs. two-skill curriculum. Per-benchmark panels: Figure 14.

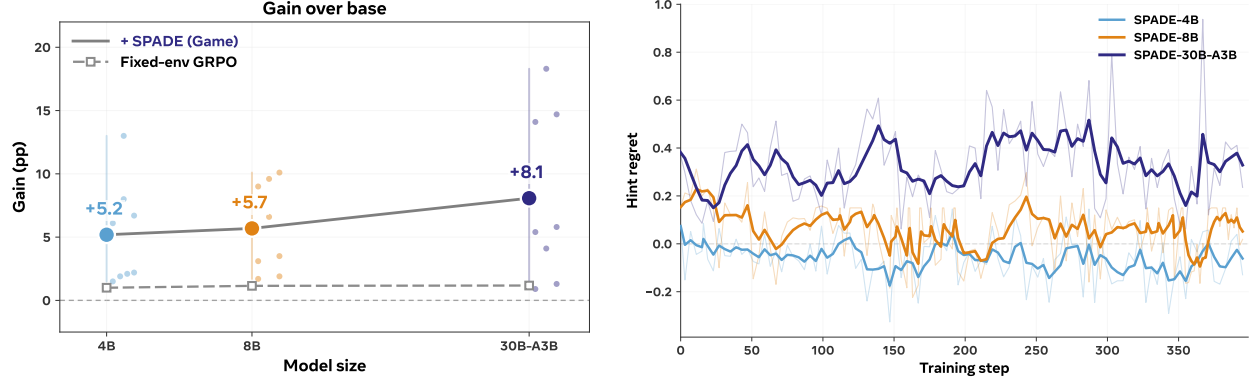


Figure 12 SPADE’s average gain over base grows with model size, from +5.2 at 4B to +8.1 at 30B-A3B, while matched-budget Fixed-env GRPO stays near +1.2. **Left:** average gain over each backbone’s own base across the eight benchmarks of Table 1 (large markers), the eight per-benchmark gains beside each mean, Fixed-env GRPO in gray. **Right:** Environment Designer hint-based regret over training (dark: EMA-smoothed; light: per-step). Only the 30B-A3B estimate stays positive; at 4B and 8B it dips below zero for long stretches, where the finite-sample estimate turns negative even though regret is non-negative at the optimum (Section 4). Both smaller backbones still gain over base (+5.2, +5.7), so the environments help even where the signal is noisy.

(best checkpoint on the eight-benchmark suite of Table 1: 53.7 vs. 58.3). The gains grow with the diversity of the curriculum, not with any single game family.

9 Discussion

SPADE’s results support two conclusions. First, training the **Environment Designer** to produce environments adapted to the **Reasoning Agent**’s current ability outperforms both training on a fixed set of environments (Table 1, Section 6.1) and training on synthetic environments from a frozen designer, even when that designer is a stronger frontier model (GPT-5.5; Table 3, Section 7). The reason is adaptivity: environments that keep adapting to the **Reasoning Agent** sustain gains after fixed pools stop improving. Second, one code-as-environment interface spans two very different settings. In the games setting, reasoning skills the **Reasoning Agent** develops (planning, constraint satisfaction, strategic thinking) generalize to held-out mathematics, science, and code, well beyond the game format they were learned in. In the tool-use setting, the same recipe lifts multi-step agentic benchmarks most where interaction matters, by +13.9 on ACEBench-Agent and +5.7 on BFCL v4 multi-turn at 30B-A3B. Making environment design a learnable role drives gains in both.

Limitations. (a) *Complexity bounded by scale and the invisible leash* (Chae et al., 2025). The **Environment Designer** cannot produce environments more complex than its base model can express in context, so reachable environment complexity grows with model scale and generation budget. (b) *A human-designed optimizer.* Both roles are updated by a fixed, human-authored RL algorithm (GRPO); SPADE does not modify its own learning rule. (c) *No formal optimality, and fixed-task evaluation.* Hint-based regret is motivated by PAIRED (Dennis et al., 2020) but not proven to yield an optimal curriculum, and current benchmarks measure fixed-task performance rather than open-ended reasoning growth.

Future directions. We improve the **Environment Designer** with gradient updates, but co-adaptation between the **Environment Designer** and **Reasoning Agent** might also come from an **Environment Designer** that improves without weight updates, accumulating and refining design strategies from past attempts in context; whether learned weights or in-context evolution makes the better designer, and at what scale, is an open question. SPADE also automates one stage of post-training; combining adaptive environment design with systems that automate the remaining stages, from data curation through the learning rule itself, could extend co-adaptive self-play across the whole training pipeline.

10 Conclusion

We presented SPADE, a framework that makes environment design a learnable component of LLM post-training through self-play. Three contributions enable this: (1) a hint-based regret reward that trains the **Environment Designer** to produce environments at the **Reasoning Agent**’s learning frontier, grounded in minimax regret theory; (2) environment design anchored in a pretraining corpus and an environment memory, with a code-as-environment interface that unifies single-turn and multi-turn settings; and (3) a practical recipe at 30B+ scale on Qwen3 models. SPADE retains its gains after fixed-environment baselines begin to saturate, raising held-out reasoning and code benchmarks by +8.1 average points over base at 30B-A3B (+5.2 and +5.7 at 4B and 8B), with the margin over the strongest fixed-environment baseline growing with scale; the same recipe lifts multi-step tool-use benchmarks by +13.9 on ACEBench-Agent and +5.7 on BFCL v4 multi-turn.

SPADE shows that a single model can design its own training environments and improve from them, a step from fixed benchmarks toward open-ended, continual self-improvement.

Acknowledgments

This research was supported by the UW-Amazon Science Gift Hub, UW-Tsukuba Amazon NVIDIA Cross Pacific AI Initiative (XPAI), Sony Research Award, Modal Research Grants, Tinker Research Grants, Character.AI, DoorDash, Open Philanthropy, Coefficient Giving, Toyota Research Institute, and the Schmidt AI2050 Fellows program. This material is based upon work supported by the Defense Advanced Research Projects Agency and the Air Force Research Laboratory, contract number(s): FA8650-23-C-7316. Any opinions, findings and conclusions, or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of AFRL or DARPA.

References

- Marwa Abdulhai, Isadora White, Yanming Wan, Ibrahim Qureshi, Joel Leibo, Max Kleiman-Weiner, and Natasha Jaques. How LLMs distort our written language, 2026. URL <https://arxiv.org/abs/2603.18161>.
- Emre Can Acikgoz, Cheng Qian, Jonas Hübötter, Heng Ji, Dilek Hakkani-Tür, and Gokhan Tur. Tool-r0: Self-evolving llm agents for tool-learning from zero data. *arXiv preprint arXiv:2602.21320*, 2026.
- Luke Bailey, Kaiyue Wen, Kefan Dong, Tatsunori Hashimoto, and Tengyu Ma. Scaling self-play with self-guidance. *arXiv preprint arXiv:2604.20209*, 2026.
- Bowen Baker, Ingmar Kanitscheider, Todor Markov, Yi Wu, Glenn Powell, Bob McGrew, and Igor Mordatch. Emergent tool use from multi-agent autocurricula. In *International conference on learning representations*, 2019.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.
- Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*, pages 41–48, 2009.
- Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Dębiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, et al. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680*, 2019.
- Adrian Bolton, Alexander Lerchner, Alexandra Cordell, Alexandre Moufarek, Andrew Bolt, Andrew Lampinen, Anna Mitenkova, Arne Olav Hallingstad, Bojan Vujatovic, Bonnie Li, et al. Sima 2: A generalist embodied agent for virtual worlds. *arXiv preprint arXiv:2512.04797*, 2025.
- Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.
- Roger Creus Castanyer, Geoffrey Bradway, Lorenz Wolf, Maxwell Lin, Augustine N Mavor-Parker, and Matthew James Sargent. Populora: Co-evolving llm populations for reasoning self-play. *arXiv preprint arXiv:2605.16727*, 2026.
- Zhepeng Cen, Haolin Chen, Shiyu Wang, Zuxin Liu, Zhiwei Liu, Ding Zhao, Silvio Savarese, Caiming Xiong, Huan Wang, and Weiran Yao. Webscale-rl: Automated data pipeline for scaling rl data to pretraining levels. *arXiv preprint arXiv:2510.06499*, 2025.
- Justin Yang Chae, Md Tanvirul Alam, and Nidhi Rastogi. Towards understanding self-play for llm reasoning. *arXiv preprint arXiv:2510.27072*, 2025.
- Chao Chen, Chengzu Li, Zhiwei Li, Yinhong Liu, and Zhijiang Guo. From trainee to trainer: Llm-designed training environment for rl with multi-agent reasoning. *arXiv preprint arXiv:2606.17682*, 2026.
- Chen Chen, Xinlong Hao, Weiwen Liu, Xu Huang, Xingshan Zeng, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Yuefeng Huang, et al. Acebench: Who wins the match point in tool usage? *arXiv preprint arXiv:2501.12851*, 2025a.
- Jiaqi Chen, Bang Zhang, Ruotian Ma, Peisong Wang, Xiaodan Liang, Zhaopeng Tu, Xiaolong Li, and Kwan-Yee K Wong. Spc: Evolving self-play critic via adversarial games for llm reasoning. *arXiv preprint arXiv:2504.19162*, 2025b.
- Lili Chen, Mihir Prabhudesai, Katerina Fragkiadaki, Hao Liu, and Deepak Pathak. Self-questioning language models. *arXiv preprint arXiv:2508.03682*, 2025c.
- Xiaoyin Chen, Jiarui Lu, Minsu Kim, Dinghuai Zhang, Jian Tang, Alexandre Piché, Nicolas Gontier, Yoshua Bengio, and Ehsan Kamalloo. Self-evolving curriculum for llm reasoning. *arXiv preprint arXiv:2505.14970*, 2025d.
- Zhaorun Chen, Zhuokai Zhao, Kai Zhang, Bo Liu, Qi Qi, Yifan Wu, Tarun Kalluri, Sara Cao, Yuanhao Xiong, Haibo Tong, et al. Scaling agent learning via experience synthesis. *arXiv preprint arXiv:2511.03773*, 2025e.
- Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models. *arXiv preprint arXiv:2401.01335*, 2024.
- Daixuan Cheng, Shaohan Huang, Yuxian Gu, Huatong Song, Guoxin Chen, Li Dong, Wayne Xin Zhao, Ji-Rong Wen, and Furu Wei. Llm-in-sandbox elicits general agentic intelligence. *arXiv preprint arXiv:2601.16206*, 2026.

- Pengyu Cheng, Yong Dai, Tianhao Hu, Han Xu, Zhisong Zhang, Lei Han, Nan Du, and Xiaolong Li. Self-playing adversarial language game enhances llm reasoning. *Advances in Neural Information Processing Systems*, 37: 126515–126543, 2024.
- Caroline Choi, Zeyneb Kaya, Shirley Wu, Tengyu Ma, Tatsunori Hashimoto, and Ludwig Schmidt. Anchored self-play for code repair. *arXiv preprint arXiv:2607.03523*, 2026.
- Jeff Clune. Ai-gas: Ai-generating algorithms, an alternate paradigm for producing general artificial intelligence. *arXiv preprint arXiv:1905.10985*, 2019.
- DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Michael Dennis, Natasha Jaques, Eugene Vinitzky, Alexandre Bayen, Stuart Russell, Andrew Critch, and Sergey Levine. Emergent complexity and zero-shot transfer via unsupervised environment design. *Advances in neural information processing systems*, 33:13049–13061, 2020.
- Leander Diaz-Bone, Marco Bagatella, Jonas Hübötter, and Andreas Krause. Discover: Automated curricula for sparse-reward reinforcement learning. *Advances in Neural Information Processing Systems*, 38:21863–21896, 2026.
- Guanting Dong, Keming Lu, Chengpeng Li, Tingyu Xia, Bowen Yu, Chang Zhou, and Jingren Zhou. Self-play with execution feedback: Improving instruction-following capabilities of large language models. *arXiv preprint arXiv:2406.13542*, 2024.
- Guanting Dong, Junting Lu, Junjie Huang, Wanjun Zhong, Longxiang Liu, Shijue Huang, Zhenyu Li, Yang Zhao, Xiaoshuai Song, Xiaoxi Li, et al. Agent-world: Scaling real-world environment synthesis for evolving general agent intelligence. *arXiv preprint arXiv:2604.18292*, 2026.
- Meta Fundamental AI Research Diplomacy Team (FAIR)[†], Anton Bakhtin, Noam Brown, Emily Dinan, Gabriele Farina, Colin Flaherty, Daniel Fried, Andrew Goff, Jonathan Gray, Hengyuan Hu, et al. Human-level play in the game of diplomacy by combining language models with strategic reasoning. *Science*, 378(6624):1067–1074, 2022.
- Maxence Faldor, Jenny Zhang, Antoine Cully, and Jeff Clune. Omni-epic: Open-endedness via models of human notions of interestingness with environments programmed in code. *arXiv preprint arXiv:2405.15568*, 2024.
- Run-Ze Fan, Zengzhi Wang, and Pengfei Liu. Megascience: Pushing the frontiers of post-training datasets for science reasoning. *arXiv preprint arXiv:2507.16812*, 2025.
- Zhiyuan Fan, Tinghao Yu, Yuanjun Cai, Jiangtao Guan, Yun Yang, Dingxin Hu, Jiang Zhou, Xing Wu, Zhuo Han, Feng Zhang, et al. Toward scalable terminal task synthesis via skill graphs. *arXiv preprint arXiv:2604.25727*, 2026.
- Runnan Fang, Shihao Cai, Baixuan Li, Jialong Wu, Guangyu Li, Wenbiao Yin, Xinyu Wang, Xiaobin Wang, Liangcai Su, Zhen Zhang, et al. Towards general agentic intelligence via environment scaling. *arXiv preprint arXiv:2509.13311*, 2025a.
- Wenkai Fang, Shunyu Liu, Yang Zhou, Kongcheng Zhang, Tongya Zheng, Kaixuan Chen, Mingli Song, and Dacheng Tao. Serl: Self-play reinforcement learning for large language models with limited data. *arXiv preprint arXiv:2505.20347*, 2025b.
- Dan Friedman and Adji Bousso Dieng. The Vendi score: A diversity evaluation metric for machine learning, 2023. URL <https://arxiv.org/abs/2210.02410>.
- Kanishk Gandhi, Shivam Garg, Noah D Goodman, and Dimitris Papailiopoulos. Endless terminals: Scaling rl environments for terminal agents. *arXiv preprint arXiv:2601.16443*, 2026.
- Judah Goldfeder, Philippe Wyder, Yann LeCun, and Ravid Shwartz Ziv. Ai must embrace specialization via superhuman adaptable intelligence. *arXiv preprint arXiv:2602.23643*, 2026.
- Leon Guertler, Bobby Cheng, Simon Yu, Bo Liu, Leshem Choshen, and Cheston Tan. Textarena. *arXiv preprint arXiv:2504.11442*, 2025.
- Jiacheng Guo, Ling Yang, Peter Chen, Qixin Xiao, Yinjie Wang, Xinzhe Juan, Jiahao Qiu, Ke Shen, and Mengdi Wang. Genenv: Difficulty-aligned co-evolution between llm agents and environment simulators. *arXiv preprint arXiv:2512.19682*, 2025.
- Ahmet H Güzel, Matthew Thomas Jackson, Jarek Luca Liesen, Tim Rocktäschel, Jakob Nicolaus Foerster, Ilija Bogunovic, and Jack Parker-Holder. Imagined autotutorials. *arXiv preprint arXiv:2509.13341*, 2025.

- Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. R-zero: Self-evolving reasoning llm from zero data. *arXiv preprint arXiv:2508.05004*, 2025.
- Chengsong Huang, Haolin Liu, Tong Zheng, Runpeng Dai, Langlin Huang, Jinyuan Li, Zongxia Li, Zhepei Wei, Yu Meng, and Jiaxin Huang. G-zero: Self-play for open-ended generation from zero data. *arXiv preprint arXiv:2605.09959*, 2026.
- Edward Hughes, Michael Dennis, Jack Parker-Holder, Feryal Behbahani, Aditi Mavalankar, Yuge Shi, Tom Schaul, and Tim Rocktaschel. Open-endedness is essential for artificial superhuman intelligence. *arXiv preprint arXiv:2406.04268*, 2024.
- Geoffrey Irving, Paul Christiano, and Dario Amodei. Ai safety via debate. *arXiv preprint arXiv:1805.00899*, 2018.
- Hamish Ivison. Diversity as the bottleneck in self-play. Blog post, May 2026. URL <https://ivison.id.au/2026/05/06/self-play.html>.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL <https://arxiv.org/abs/2403.07974>.
- Swadesh Jana, Cansu Sancaktar, Tomáš Daniš, Georg Martius, Antonio Orvieto, and Pavel Kolev. Gasp: Guided asymmetric self-play for coding llms. *arXiv preprint arXiv:2603.15957*, 2026.
- Liwei Jiang, Yuanjun Chai, Margaret Li, Mickel Liu, Raymond Fok, Nouha Dziri, Yulia Tsvetkov, Maarten Sap, Alon Albalak, and Yejin Choi. Artificial hivemind: The open-ended homogeneity of language models (and beyond), 2025a. URL <https://arxiv.org/abs/2510.22954>.
- Minqi Jiang, Michael Dennis, Jack Parker-Holder, Jakob Foerster, Edward Grefenstette, and Tim Rocktäschel. Replay-guided adversarial environment design. *Advances in Neural Information Processing Systems*, 34:1884–1897, 2021a.
- Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In *International Conference on Machine Learning*, pages 4940–4950. PMLR, 2021b.
- Minqi Jiang, Andrei Lupu, and Yoram Bachrach. Bootstrapping task spaces for self-improvement. *arXiv preprint arXiv:2509.04575*, 2025b.
- Ingmar Kanitscheider, Joost Huizinga, David Farhi, William Hebgen Guss, Brandon Houghton, Raul Sampedro, Peter Zhokhov, Bowen Baker, Adrien Ecoffet, Jie Tang, Oleg Klimov, and Jeff Clune. Multi-task curriculum learning in a complex, visual, hard-exploration domain: Minecraft, 2021. URL <https://arxiv.org/abs/2106.14876>.
- Jan Hendrik Kirchner, Yining Chen, Harri Edwards, Jan Leike, Nat McAleese, and Yuri Burda. Prover-verifier games improve legibility of llm outputs. *arXiv preprint arXiv:2407.13692*, 2024.
- Jakub Grudzien Kuba, Mengting Gu, Qi Ma, Yuandong Tian, Vijai Mohan, and Jason Chen. Language self-play for data-free training. *arXiv preprint arXiv:2509.07414*, 2025.
- Ilia Kulikov, Chenxi Whitehouse, Tianhao Wu, Yixin Nie, Swarnadeep Saha, Eryk Helenowski, Weizhe Yuan, Olga Golovneva, Jack Lanchantin, Yoram Bachrach, et al. Autodata: An agentic data scientist to create high quality synthetic data. *arXiv preprint arXiv:2606.25996*, 2026.
- Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, Sedrick Keh, Kushal Arora, Saurabh Garg, Rui Xin, Niklas Muennighoff, Reinhard Heckel, Jean Mercat, Mayee Chen, Suchin Gururangan, Mitchell Wortsman, Alon Albalak, Yonatan Bitton, Marianna Nezhurina, Amro Abbas, Cheng-Yu Hsieh, Dhruva Ghosh, Josh Gardner, Maciej Kilian, Hanlin Zhang, Rulin Shao, Sarah Pratt, Sunny Sanyal, Gabriel Ilharco, Giannis Daras, Kalyani Marathe, Aaron Gokaslan, Jieyu Zhang, Khyathi Chandu, Thao Nguyen, Igor Vasiljevic, Sham Kakade, Shuran Song, Sujay Sanghavi, Fartash Faghri, Sewoong Oh, Luke Zettlemoyer, Kyle Lo, Alaaeldin El-Nouby, Hadi Pouransari, Alexander Toshev, Stephanie Wang, Dirk Groeneveld, Luca Soldaini, Pang Wei Koh, Jenia Jitsev, Thomas Kollar, Alexandros G. Dimakis, Yair Carmon, Achal Dave, Ludwig Schmidt, and Vaishaal Shankar. Datacomp-lm: In search of the next generation of training sets for language models. *arXiv preprint arXiv:2406.11794*, 2024.
- Yuetai Li, Huseyin A Inan, Xiang Yue, Wei-Ning Chen, Lukas Wutschitz, Janardhan Kulkarni, Radha Poovendran, Robert Sim, and Saravan Rajmohan. Simulating environments with reasoning models for agent training. *arXiv preprint arXiv:2511.01824*, 2025.

- William Liang, Sam Wang, Hung-Ju Wang, Osbert Bastani, Dinesh Jayaraman, and Yecheng Jason Ma. Eurekaverse: Environment curriculum generation via large language models. *arXiv preprint arXiv:2411.01775*, 2024.
- Michael L Littman. Markov games as a framework for multi-agent reinforcement learning. In *Machine learning proceedings 1994*, pages 157–163. Elsevier, 1994.
- Bo Liu, Leon Guertler, Simon Yu, Zichen Liu, Penghui Qi, Daniel Balcells, Mickel Liu, Cheston Tan, Weiyan Shi, Min Lin, et al. Spiral: Self-play on zero-sum games incentivizes reasoning via multi-agent multi-turn reinforcement learning. *arXiv preprint arXiv:2506.24119*, 2025a.
- Bo Liu, Chuanyang Jin, Seungone Kim, Weizhe Yuan, Wenting Zhao, Ilia Kulikov, Xian Li, Sainbayar Sukhbaatar, Jack Lanchantin, and Jason Weston. Spice: Self-play in corpus environments improves reasoning. *arXiv preprint arXiv:2510.24684*, 2025b.
- Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Peter Belcak, Mingjie Liu, Min-Hung Chen, Hongxu Yin, Yu-Chiang Frank Wang, Kwang-Ting Cheng, Yejin Choi, Jan Kautz, and Pavlo Molchanov. Gdpo: Group reward-decoupled normalization policy optimization for multi-reward rl optimization, 2026a. URL <https://arxiv.org/abs/2601.05242>.
- Wei Liu, Siya Qi, Yali Du, and Yulan He. Self-play only evolves when self-synthetic pipeline ensures learnable information gain. *arXiv preprint arXiv:2603.02218*, 2026b.
- Zichen Liu, Anya Sims, Keyu Duan, Changyu Chen, Simon Yu, Xiangxin Zhou, Haotian Xu, Shaopan Xiong, Bo Liu, Chenmian Tan, et al. Gem: A gym for agentic llms. *arXiv preprint arXiv:2510.01051*, 2025c.
- Siyuan Lu, Zechuan Wang, Hongxuan Zhang, Qintong Wu, Leilei Gan, Chenyi Zhuang, Jinjie Gu, and Tao Lin. Don’t just fine-tune the agent, tune the environment. *arXiv preprint arXiv:2510.10197*, 2025.
- Ximing Lu, David Acuna, Jaehun Jung, Jian Hu, Di Zhang, Shizhe Diao, Yunheng Zou, Shaokun Zhang, Brandon Cui, Mingjie Liu, et al. Golden goose: A simple trick to synthesize unlimited rlvr tasks from unverifiable internet text. *arXiv preprint arXiv:2601.22975*, 2026.
- MAA. American invitational mathematics examination (AIME). Mathematics Competition Series, n.d. URL <https://maa.org/math-competitions/aime>.
- Ishita Mediratta, Minqi Jiang, Jack Parker-Holder, Michael Dennis, Eugene Vinitzky, and Tim Rocktäschel. Stabilizing unsupervised environment design with a learned adversary. In *Conference on Lifelong Learning Agents*, pages 270–291. PMLR, 2023.
- Nathan Monette, Alistair Letcher, Michael Beukman, Matthew T Jackson, Alexander Rutherford, Alexander D Goldie, and Jakob N Foerster. An optimisation framework for unsupervised environment design. *arXiv preprint arXiv:2505.20659*, 2025.
- Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- NVIDIA. Nemotron 3 ultra: Open, efficient mixture-of-experts hybrid mamba-transformer model for agentic reasoning, 2026. URL <https://research.nvidia.com/labs/nemotron/files/NVIDIA-Nemotron-3-Ultra-Technical-Report.pdf>. White Paper.
- OpenAI. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- Jack Parker-Holder, Minqi Jiang, Michael Dennis, Mikayel Samvelyan, Jakob Foerster, Edward Grefenstette, and Tim Rocktäschel. Evolving curricula with regret-based environment design. In *International Conference on Machine Learning*, pages 17473–17498. PMLR, 2022.
- Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.
- Renjie Pi, Grace Lam, Mohammad Shoeybi, Pooya Jannaty, Bryan Catanzaro, and Wei Ping. On data engineering for scaling llm terminal capabilities. *arXiv preprint arXiv:2602.21193*, 2026.
- PYMNTS. Prime Intellect Raises \$130 Million to Help Companies Train AI Agents. <https://www.pymnts.com/news/investment-tracker/2026/prime-intellect-raises-130-million-to-help-companies-train-ai-agents/>, 2026. PYMNTS.

- Ram Ramrakhya, Andrew Szot, Omar Attia, Yuhao Yang, Anh Nguyen, Bogdan Mazouze, Zhe Gan, Harsh Agrawal, and Alexander Toshev. Scaling synthetic task generation for agents via exploration. *arXiv preprint arXiv:2509.25047*, 2025.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks, 2019. URL <https://arxiv.org/abs/1908.10084>.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*, 2023.
- Alex Rutherford, Michael Beukman, Timon Willi, Bruno Lacerda, Nick Hawes, and Jakob Foerster. No regrets: Investigating and improving regret approximations for curriculum discovery. *Advances in Neural Information Processing Systems*, 37:16071–16101, 2024.
- Bidipta Sarkar, Warren Xia, C Karen Liu, and Dorsa Sadigh. Training language models for social deduction with multi-agent reinforcement learning. *arXiv preprint arXiv:2502.06060*, 2025.
- Jürgen Schmidhuber. Powerplay: Training an increasingly general problem solver by continually searching for the simplest still unsolvable problem. *Frontiers in psychology*, 4:313, 2013.
- Sheikh Shafayat, Fahim Tajwar, Ruslan Salakhutdinov, Jeff Schneider, and Andrea Zanette. Can large reasoning models self-train? *arXiv preprint arXiv:2505.21444*, 2025.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Yucheng Shi, Zhenwen Liang, Kishan Panaganti, Dian Yu, Wenhao Yu, and Haitao Mi. Learning to build the environment: Self-evolving reasoning rl via verifiable environment synthesis. *arXiv preprint arXiv:2605.14392*, 2026.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2303.11366>.
- David Silver and Richard S Sutton. Welcome to the era of experience. *Google AI*, 1:11, 2025.
- David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.
- David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *nature*, 550(7676):354–359, 2017.
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharmashan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.
- Avi Singh, John D Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J Liu, James Harrison, Jaehoon Lee, Kelvin Xu, et al. Beyond human data: Scaling self-training for problem-solving with language models. *arXiv preprint arXiv:2312.06585*, 2023.
- Xiaoshuai Song, Hao-fei Chang, Guanting Dong, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. Envscaler: Scaling tool-interactive environments for llm agent via programmatic synthesis. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 8326–8357, 2026.
- Yuda Song, Hanlin Zhang, Carson Eisenach, Sham Kakade, Dean Foster, and Udaya Ghai. Mind the gap: Examining the self-improvement capabilities of large language models. *arXiv preprint arXiv:2412.02674*, 2024.
- Zafir Stojanovski, Oliver Stanley, Joe Sharratt, Richard Jones, Abdulhakeem Adefioye, Jean Kaddour, and Andreas Köpf. Reasoning gym: Reasoning environments for reinforcement learning with verifiable rewards. *Advances in Neural Information Processing Systems*, 38, 2026.
- Sainbayar Sukhbaatar, Zeming Lin, Ilya Kostrikov, Gabriel Synnaeve, Arthur Szlam, and Rob Fergus. Intrinsic motivation and automatic curricula via asymmetric self-play. *arXiv preprint arXiv:1703.05407*, 2017.

- Shobhita Sundaram, John Quan, Ariel Kwiatkowski, Kartik Ahuja, Yann Ollivier, and Julia Kempe. Teaching models to teach themselves: Reasoning at the edge of learnability. *arXiv preprint arXiv:2601.18778*, 2026.
- Fahim Tajwar, Yiding Jiang, Abitha Thankaraj, Sumaita Sadia Rahman, J Zico Kolter, Jeff Schneider, and Ruslan Salakhutdinov. Training a generally curious agent. *arXiv preprint arXiv:2502.17543*, 2025.
- Jayden Teoh, Wenjun Li, and Pradeep Varakantham. Improving environment novelty quantification for effective unsupervised environment design. *Advances in Neural Information Processing Systems*, 37:135299–135333, 2024.
- Gerald Tesauro et al. Temporal difference learning and td-gammon. *Communications of the ACM*, 38(3):58–68, 1995.
- Mark Towers, Ariel Kwiatkowski, John Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Kallinteris Andreas, Markus Krimmel, Arjun Kg, Rodrigo Perez-Vicente, et al. Gymnasium: A standard interface for reinforcement learning environments. *Advances in Neural Information Processing Systems*, 38, 2026.
- Dunwei Tu, Hongyan Hao, Hansi Yang, Yihao Chen, Yi-Kai Zhang, Zhikang Xia, Yu Yang, Yueqing Sun, Xingchen Liu, Furao Shen, et al. Scaleenv: Scaling environment synthesis from scratch for generalist interactive tool-use agent training. *arXiv preprint arXiv:2602.06820*, 2026.
- Pablo Villalobos, Anson Ho, Jaime Sevilla, Tamay Besiroglu, Lennart Heim, and Marius Hobbhahn. Will we run out of data? Limits of LLM scaling based on human-generated data. *arXiv preprint arXiv:2211.04325*, 2024. URL <https://arxiv.org/abs/2211.04325>.
- Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *nature*, 575(7782):350–354, 2019.
- Ziyu Wan, Yunxiang Li, Xiaoyu Wen, Yan Song, Hanjing Wang, Linyi Yang, Mark Schmidt, Jun Wang, Weinan Zhang, Shuyue Hu, et al. Rema: Learning to meta-think for llms with multi-agent reinforcement learning. *arXiv preprint arXiv:2503.09501*, 2025.
- Qinsi Wang, Bo Liu, Tianyi Zhou, Jing Shi, Yueqian Lin, Yiran Chen, Hai Helen Li, Kun Wan, and Wentian Zhao. Vision-zero: Scalable vlm self-improvement via strategic gamified self-play. *arXiv preprint arXiv:2509.25541*, 2025a.
- Rui Wang, Joel Lehman, Jeff Clune, and Kenneth O Stanley. Paired open-ended trailblazer (poet): Endlessly generating increasingly complex and diverse learning environments and their solutions. *arXiv preprint arXiv:1901.01753*, 2019.
- Yinjie Wang, Tianbao Xie, Ke Shen, Mengdi Wang, and Ling Yang. Rlanything: Forge environment, policy, and reward model in completely dynamic rl system. *arXiv preprint arXiv:2602.02488*, 2026a.
- Yuxing Wang, Zhiyu Chen, Tiantian Zhang, Qiyue Yin, Yongzhe Chang, Zhiheng Li, Liang Wang, and Xueqian Wang. Embodied co-design for rapidly evolving agents: Taxonomy, frontiers, and challenges. *arXiv preprint arXiv:2512.04770*, 2025b.
- Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent world model: Infinity synthetic environments for agentic reinforcement learning. *arXiv preprint arXiv:2602.10090*, 2026b.
- Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, et al. Ragen: Understanding self-evolution in llm agents via multi-turn reinforcement learning. *arXiv preprint arXiv:2504.20073*, 2025c.
- Yuxiang Wei, Zhiqing Sun, Emily McMilin, Jonas Gehring, David Zhang, Gabriel Synnaeve, Daniel Fried, Lingming Zhang, and Sida Wang. Toward training superintelligent software agents through self-play swe-rl. *arXiv preprint arXiv:2512.18552*, 2025.
- Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida Wang. Swe-rl: Advancing llm reasoning via reinforcement learning on open software evolution. *Advances in Neural Information Processing Systems*, 38:78500–78525, 2026.
- Yiming Xiong, Shengran Hu, and Jeff Clune. Learning to continually learn via meta-learning agentic memory designs. *arXiv preprint arXiv:2602.07755*, 2026.
- Caijun Xu, Changyi Xiao, Zhongyuan Peng, Xinrun Wang, and Yixin Cao. Scaler: Synthetic scalable adaptive learning environment for reasoning. *arXiv preprint arXiv:2601.04809*, 2026a.
- Fengli Xu, Qian Yue Hao, Chenyang Shao, Zefang Zong, Yu Li, Jingwei Wang, Yunke Zhang, Jingyi Wang, Xiaochong Lan, Jiahui Gong, et al. Toward large reasoning models: A survey of reinforced reasoning with large language models. *Patterns*, 6(10), 2025.

- Minrui Xu, Zilin Wang, Mengyi Deng, Zhiwei Li, Zhicheng Yang, Xiao Zhu, Yinhong Liu, Boyu Zhu, Baiyu Huang, Chao Chen, et al. Envfactory: Scaling tool-use agents via executable environments synthesis and robust rl. *arXiv preprint arXiv:2605.18703*, 2026b.
- Taofeng Xue, Chong Peng, Mianqiu Huang, Linsen Guo, Tiancheng Han, Haozhe Wang, Jianing Wang, Xiaocheng Zhang, Xin Yang, Dengchang Zhao, et al. Evocua: Evolving computer use agents via learning from scalable synthetic experience. *arXiv preprint arXiv:2601.15876*, 2026a.
- Tianci Xue, Zeyi Liao, Tianneng Shi, Zilu Wang, Kai Zhang, Dawn Song, Yu Su, and Huan Sun. Autonomous continual learning of computer-use agents for environment adaptation. *arXiv preprint arXiv:2602.10356*, 2026b.
- Chengyi Yang, Zhishang Xiang, Yunbo Tang, Zongpei Teng, Chengsong Huang, Fei Long, Yuhan Liu, and Jinsong Su. Ttcs: Test-time curriculum synthesis for self-evolving. *arXiv preprint arXiv:2601.22628*, 2026.
- Ziyi Yang, Weizhou Shen, Chenliang Li, Ruijun Chen, Fanqi Wan, Ming Yan, Xiaojun Quan, and Fei Huang. Spell: Self-play reinforcement learning for evolving long-context language models. *arXiv preprint arXiv:2509.23863*, 2025.
- Feng Yao, Liyuan Liu, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, and Jianfeng Gao. Your efficient rl framework secretly brings you off-policy rl training, August 2025. URL <https://fengyao.notion.site/off-policy-rl>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason E Weston. Self-rewarding language models. In *Forty-first International Conference on Machine Learning*, 2024.
- Maxwell Zeff. Silicon Valley bets big on ‘environments’ to train AI agents. <https://techcrunch.com/2025/09/21/silicon-valley-bets-big-on-environments-to-train-ai-agents/>, 2025. TechCrunch.
- Zhiyuan Zeng, Hamish Ivison, Yiping Wang, Lifan Yuan, Shuyue Stella Li, Zhuorui Ye, Siting Li, Jacqueline He, Runlong Zhou, Tong Chen, et al. Rlve: Scaling up reinforcement learning for language models with adaptive verifiable environments. *arXiv preprint arXiv:2511.07317*, 2025.
- Hanchen Zhang, Xiao Liu, Bowen Lv, Xueqiao Sun, Bohao Jing, Iat Long Iong, Zhenyu Hou, Zehan Qi, Hanyu Lai, Yifan Xu, et al. Agentrl: Scaling agentic reinforcement learning with a multi-turn, multi-task framework. *arXiv preprint arXiv:2510.04206*, 2025a.
- Jenny Zhang, Joel Lehman, Kenneth Stanley, and Jeff Clune. Omni: Open-endedness via models of human notions of interestingness. *arXiv preprint arXiv:2306.01711*, 2023.
- Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. Hyperagents. *arXiv preprint arXiv:2603.19461*, 2026a.
- Jiayi Zhang, Yiran Peng, Fanqi Kong, Cheng Yang, Yifan Wu, Zhaoyang Yu, Jinyu Xiang, Jianhao Ruan, Jinlin Wang, Maojia Song, et al. Autoenv: Automated environments for measuring cross-environment agent learning. *arXiv preprint arXiv:2511.19304*, 2025b.
- Jiayi Zhang, Fanqi Kong, Guibin Zhang, Maojia Song, Zhaoyang Yu, Jianhao Ruan, Jinyu Xiang, Bang Liu, Chenglin Wu, and Yuyu Luo. Scalable environments drive generalizable agents. *arXiv preprint arXiv:2605.18181*, 2026b.
- Jiayi Zhang, Simon Yu, Derek Chong, Anthony Sicilia, Michael R. Tomz, Christopher D. Manning, and Weiyan Shi. Verbalized sampling: How to mitigate mode collapse and unlock llm diversity, 2026c. URL <https://arxiv.org/abs/2510.01171>.
- Kai Zhang, Xiangchao Chen, Bo Liu, Tianci Xue, Zeyi Liao, Zhihan Liu, Xiyao Wang, Yuting Ning, Zhaorun Chen, Xiaohan Fu, et al. Agent learning via early experience. *arXiv preprint arXiv:2510.08558*, 2025c.
- Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, et al. Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory. *arXiv preprint arXiv:2601.03192*, 2026d.
- Zhengxin Zhang, Chengyu Huang, Aochong Oliver Li, and Claire Cardie. Better llm reasoning via dual-play. *arXiv preprint arXiv:2511.11881*, 2025d.

- Ziyun Zhang, Zezhou Wang, Xiaoyi Zhang, Zongyu Guo, Jiahao Li, Bin Li, and Yan Lu. Infiniteweb: Scalable web environment synthesis for gui agent training. *arXiv preprint arXiv:2601.04126*, 2026e.
- Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025.
- Yifei Zhou, Sergey Levine, Jason Weston, Xian Li, and Sainbayar Sukhbaatar. Self-challenging language model agents. *arXiv preprint arXiv:2506.01716*, 2025.
- Yuhang Zhou, Lizhu Zhang, Yifan Wu, Jiayi Liu, Xiangjun Fan, Zhuokai Zhao, and Hong Yan. Synthetic sandbox for training machine learning engineering agents. *arXiv preprint arXiv:2604.04872*, 2026.
- Kaijie Zhu, Yuzhou Nie, Yijiang Li, Yiming Huang, Jialian Wu, Jiang Liu, Ximeng Sun, Zhenfei Yin, Lun Wang, Zicheng Liu, et al. Termigen: High-fidelity environment and robust trajectory synthesis for terminal agents. *arXiv preprint arXiv:2602.07274*, 2026.
- Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. slime: An llm post-training framework for rl scaling. <https://github.com/THUDM/slime>, 2025. GitHub repository. Corresponding author: Xin Lv.

Appendix

Contents of the Appendix

A	Notation	28
B	Theoretical Analysis	29
B.1	Setup and Assumptions	29
B.2	Main Results	30
C	Method and Experiment Details	32
C.1	System Prompts and Templates	32
C.2	Implementation Details	43
C.3	Reproducibility	44
D	Extended Ablations	45
E	Extended Related Work	47
E.1	Self-Play for LLMs	47
E.2	Unsupervised Environment Design and Open-Endedness	48
E.3	Synthetic Environment Generation	49
E.4	Environment Scaling	51
E.5	Agentic Memory Design and Self-Improving Code Systems	52
F	Extended Quantitative Analysis	53
F.1	Games	53
G	Extended Qualitative Analysis	57
G.1	Games	57
G.2	Tool Use	60
G.3	Generated Environment Gallery	61

A Notation

[\[back to contents\]](#)

Table 4 consolidates symbols used throughout the paper.

Symbol	Description
<i>MDP and environment</i>	
$\mathcal{S}$	State space
$\mathcal{A}$	Action space
$T(s' s, a)$	Transition function
$R(s, a)$	Reward function
ρ_0	Initial state distribution
s, s', a	State, next state, action
$\mathcal{E}$	Space of valid (executable Python) environments
$e \in \mathcal{E}$	A single environment instance
e_b	The b -th environment in a generation batch
<i>Policy and roles</i>	
π_θ	Shared LLM policy with parameters θ
π_D	Policy in Environment Designer role, $\pi_\theta(\cdot \text{role}=D)$
π_A	Policy in Reasoning Agent role, $\pi_\theta(\cdot \text{role}=A)$
role= D	System-prompt switch selecting Environment Designer role
role= A	System-prompt switch selecting Reasoning Agent role
<i>Hints and rewards</i>	
h	Privileged hint (strategy / partial solution / key observation)
h_b	Hint for the b -th environment
y, y_i	Reasoning Agent response (rollout)
y'_i	Reasoning Agent rollout sampled with the privileged hint in context
$r_A(y e)$	Per-rollout correctness reward (without hint)
$r_A(y e, h)$	Per-rollout correctness reward conditioned on hint
$\bar{r}_A(e)$	Average Reasoning Agent return on e without hints
$\bar{r}_A(e h)$	Average Reasoning Agent return on e with hint h
$r_D(e)$	Environment Designer reward: hint-based regret $\bar{r}_A(e h) - \bar{r}_A(e)$
<i>GRPO and training</i>	
x	Prompt / input sequence (generic GRPO notation)
$\mathcal{L}(\theta)$	GRPO clipped-surrogate training objective
$\pi_{\text{old}}, \pi_{\text{ref}}$	Behavior policy (importance ratio) and KL reference policy
$\hat{A}^i$	Group-normalized advantage (generic); role-specific forms below
$\hat{A}_D^b$	Group-normalized advantage for the b -th designed environment
$\hat{A}_A^i$	Group-normalized advantage for the i -th agent rollout
$\varepsilon_{\text{low}}, \varepsilon_{\text{high}}$	Asymmetric PPO clipping range (DAPO clip-higher (Yu et al., 2025))
β_{KL}	KL-regularization coefficient
B	Generation batch size (environments per iteration)
G	Group size (Reasoning Agent rollouts per environment)
k	Regeneration interval: rollouts per environment set, and the Environment Designer update delay
N	Total number of training iterations
p_d	Domain prompt sampled to seed environment generation
r^i, r^j	Scalar reward of the i -th / j -th response in a GRPO group
M	Environment memory: buffer of past environments with regret scores and skill tags
C	Pretraining corpus used to ground environment design

Table 4 Symbols used throughout the paper.

B Theoretical Analysis

[\[back to contents\]](#)

This section formally analyzes the incentive structure of the hint-based regret reward (hereafter hint-regret) introduced in Section 4.2. We model the interaction between the **Environment Designer** and **Reasoning Agent** as a two-player game: the **Environment Designer** selects an environment distribution, while the **Reasoning Agent** selects a policy. Since LLM inference is stochastic, all payoffs are defined through expected verifier returns.

The main result is an equilibrium characterization of the idealized game. If positive hint-regret remains anywhere, then the current unhinted **Reasoning Agent** is suboptimal there, and the **Environment Designer** can profitably concentrate on it. Consequently, at any pure Nash equilibrium, the **Environment Designer**’s expected regret is zero, the **Reasoning Agent** is hint-free optimal on *every* environment in $\mathcal{M}$, and privileged hints become vacuous.

B.1 Setup and Assumptions

Environment class. Let $\mathcal{U}$ be a finite universe of candidate environments. The set of *mathematically valid executable* environments is denoted by $\mathcal{M} \subseteq \mathcal{U}$. These are environments whose specifications are internally consistent and whose attempted solutions can be evaluated by an external verifier. In the implementation, $\mathcal{M}$ is approximated by syntax checks, executability checks, and, in the tool-use setting, solvability filtering.

Distribution notation. For any finite set $\mathcal{X}$, let $\Delta(\mathcal{X})$ denote the set of probability distributions over $\mathcal{X}$. If $D \in \Delta(\mathcal{M})$ (the **Environment Designer**’s environment distribution; we reuse the letter D for this distribution, while the subscript in u_D below labels the **Environment Designer** role) and $\mathcal{B} \subseteq \mathcal{M}$, write

$$D(\mathcal{B}) := \sum_{e \in \mathcal{B}} D(e).$$

A property holds for D -almost every environment if the set on which it fails has D -mass zero.

Inference protocol. Fix an inference budget B_{inf} . This budget includes all evaluation-time choices that affect the set of possible **Reasoning Agent** trajectories: model architecture, context length, prompt format, maximum generation length, sampling rule, temperature, number of rollouts, tool access, memory access, and verifier. Let $\mathcal{Y}_{B_{\text{inf}}}$ denote the finite trajectory space induced by this budget.

Fine-tuning budget. Fix a fine-tuning budget F . This budget includes all training-time choices that determine which policies the procedure can produce: optimizer, objective, number of updates, and regularization. Let Π denote the set of policies attainable under this budget; we take Π to be finite, so maxima over Π are attained.

Returns with and without hints. For each environment $e \in \mathcal{M}$, let $h(e)$ denote the privileged hint emitted by the **Environment Designer**; hints are modeled as a fixed map $e \mapsto h(e)$, so strategic hint choice by the **Environment Designer** is outside this idealization. A **Reasoning Agent** policy $\pi \in \Pi$ induces a distribution over trajectories in $\mathcal{Y}_{B_{\text{inf}}}$ given either the unhinted input e or the hinted input $(e, h(e))$. Let

$$r(e, y) \in [0, 1]$$

denote the verifier return of trajectory $y \in \mathcal{Y}_{B_{\text{inf}}}$ on environment e . Returns are normalized to $[0, 1]$ for the analysis; training uses a shaped reward in $[-1, 1]$ (Algorithm 1). Here $r(e, y)$ is the per-trajectory return written $r_A(y \mid e)$ in Section 4, and $R_\pi(e)$, $R_\pi^h(e)$ below are the population analogues of the empirical means $\bar{r}_A(e)$ and $\bar{r}_A(e \mid h)$.

The unhinted expected return of policy π on environment e is

$$R_\pi(e) := \mathbb{E}_{y \sim \pi(\cdot \mid e)}[r(e, y)],$$

and the hinted expected return is

$$R_\pi^h(e) := \mathbb{E}_{y \sim \pi(\cdot \mid e, h(e))}[r(e, y)].$$

Game payoffs. The **Environment Designer** chooses a distribution $D \in \Delta(\mathcal{M})$. Its payoff is the expected hint-based regret

$$u_D(D, \pi) := \mathbb{E}_{e \sim D} [R_\pi^h(e) - R_\pi(e)].$$

The **Reasoning Agent**'s payoff is its unhinted expected return

$$u_A(D, \pi) := \mathbb{E}_{e \sim D} [R_\pi(e)].$$

Together these define the expected-regret self-play game on $\Delta(\mathcal{M}) \times \Pi$.

Pure Nash equilibrium. A pair $(D^\circ, \pi^\circ) \in \Delta(\mathcal{M}) \times \Pi$ is a pure Nash equilibrium of this game if neither player can improve by unilateral deviation:

$$u_D(D^\circ, \pi^\circ) \geq u_D(D, \pi^\circ) \quad \forall D \in \Delta(\mathcal{M}),$$

and

$$u_A(D^\circ, \pi^\circ) \geq u_A(D^\circ, \pi) \quad \forall \pi \in \Pi.$$

Optimal values. For each environment, define the optimal unhinted value

$$R^*(e) := \max_{\pi \in \Pi} R_\pi(e).$$

Assumption B.1 (Sound generation). The **Environment Designer** only samples mathematically valid executable environments:

$$D \in \Delta(\mathcal{M}).$$

Operationally, this corresponds to syntax checks, executability checks, and, in the tool-use setting, solvability filtering. (Stated for completeness: the game is defined over $\Delta(\mathcal{M})$ throughout.)

Assumption B.2 (Articulated hints). Conditioning on the hint attains the optimal unhinted value: for every policy $\pi \in \Pi$ and every environment $e \in \mathcal{M}$,

$$R_\pi^h(e) = R^*(e).$$

In particular $R_\pi^h(e) \geq R_\pi(e)$.

Assumption B.3 (Internalizability of hinted behavior). Hinted behavior is attainable without the hint: for every policy $\pi \in \Pi$ there exists a policy $\pi' \in \Pi$ with

$$R_{\pi'}(e) = R_\pi^h(e) \quad \text{for all } e \in \mathcal{M}.$$

B.2 Main Results

Theorem B.5 shows that every pure Nash equilibrium in this setup drives hint-regret to zero, so that the **Reasoning Agent** is optimal on every environment in $\mathcal{M}$. The ideal hint-regret enables this result, because it vanishes on environments the **Reasoning Agent** already solves and on unsolvable environments, where the hint cannot help, and is positive only where the **Reasoning Agent** fails unaided but a hint would close the gap. The **Environment Designer** therefore profits exactly by targeting this learnable frontier, and at equilibrium no gap can remain anywhere, since the **Environment Designer** could otherwise deviate to it profitably. This is what distinguishes the regret reward from the plain adversarial-difficulty reward $-\mathbb{E}_{e \sim D} [R_\pi(e)]$, which provides no mechanism restricting the **Environment Designer**'s support to useful environments.

Lemma B.4 (Hint-regret equals hint-free regret). *Under Assumption B.2, for any Reasoning Agent policy $\pi \in \Pi$ and any environment $e \in \mathcal{M}$,*

$$R_\pi^h(e) - R_\pi(e) = R^*(e) - R_\pi(e).$$

In particular the hint-regret is nonnegative, and strictly positive if and only if $R_\pi(e) < R^(e)$.*

Proof. By Assumption B.2, $R_\pi^h(e) = R^*(e)$; subtracting $R_\pi(e)$ gives the identity. The remaining claims follow since $R^*(e) \geq R_\pi(e)$. $\square$

Theorem B.5 (Nash equilibria imply hint-free optimality on every environment). *Under Assumptions B.1–B.3, every pure Nash equilibrium (D°, π°) of the expected-regret self-play game satisfies*

$$u_D(D^\circ, \pi^\circ) = 0.$$

Moreover, the pointwise regret vanishes everywhere: for every $e \in \mathcal{M}$,

$$R_{\pi^\circ}(e) = R^*(e), \quad R_{\pi^\circ}^h(e) = R_{\pi^\circ}(e).$$

Such equilibria exist: Assumptions B.2 and B.3 guarantee a uniformly optimal policy π^* with $R_{\pi^*}(e) = R^*(e)$ for all e , and any pair (D, π^*) is a pure Nash equilibrium, so the statement is not vacuous.

Proof. Let (D°, π°) be a pure Nash equilibrium. By Lemma B.4,

$$\rho^{\text{reg}}(e) := R_{\pi^\circ}^h(e) - R_{\pi^\circ}(e) = R^*(e) - R_{\pi^\circ}(e) \geq 0 \quad \text{for every } e \in \mathcal{M}.$$

Reasoning Agent side. By Assumption B.3, there is $\pi' \in \Pi$ with $R_{\pi'}(e) = R_{\pi^\circ}^h(e)$ for all e ; by Assumption B.2, $R_{\pi^\circ}^h(e) = R^*(e)$, so $R_{\pi'}(e) = R^*(e) \geq R_{\pi^\circ}(e)$ for every $e \in \mathcal{M}$. Hence

$$u_A(D^\circ, \pi') = \mathbb{E}_{e \sim D^\circ}[R^*(e)] \geq \mathbb{E}_{e \sim D^\circ}[R_{\pi^\circ}(e)] = u_A(D^\circ, \pi^\circ).$$

Since π° is a best response to D° , equality holds, so $\mathbb{E}_{e \sim D^\circ}[R^*(e) - R_{\pi^\circ}(e)] = 0$. The integrand is nonnegative, hence $\rho^{\text{reg}}(e) = 0$ for D° -almost every e , and therefore

$$u_D(D^\circ, \pi^\circ) = \mathbb{E}_{e \sim D^\circ}[\rho^{\text{reg}}(e)] = 0.$$

Environment Designer side. Fix any $e \in \mathcal{M}$. The **Environment Designer** may deviate to the point mass $\delta_e \in \Delta(\mathcal{M})$, which yields $u_D(\delta_e, \pi^\circ) = \rho^{\text{reg}}(e)$. Since (D°, π°) is a Nash equilibrium,

$$\rho^{\text{reg}}(e) = u_D(\delta_e, \pi^\circ) \leq u_D(D^\circ, \pi^\circ) = 0.$$

Combined with $\rho^{\text{reg}}(e) \geq 0$ from Lemma B.4, this gives $\rho^{\text{reg}}(e) = 0$ for every $e \in \mathcal{M}$. Therefore

$$R_{\pi^\circ}(e) = R^*(e), \quad R_{\pi^\circ}^h(e) = R_{\pi^\circ}(e)$$

for every $e \in \mathcal{M}$. $\square$

C Method and Experiment Details

[\[back to contents\]](#)

C.1 System Prompts and Templates

Environment-generation prompt. The **Environment Designer** receives the prompt below to generate a new single-turn game as executable Python; the skill name, its description, and a short list of example concepts fill the slots shown in angle brackets. In the canonical corpus-grounded runs the **Environment Designer** instead conditions on a freshly sampled corpus document, and the example-concept slot is unused. The single-turn prompt serves the single-turn setting of Section 3.1 (one derivation task, graded per attempt), while the multi-turn prompts demand stateful interaction. The `\boxed{}` extraction in the displayed contracts stops at the first closing brace, so generated environments use plain, brace-free answer strings.

Single-turn environment-generation prompt (Environment Designer)

```
<|im_start|>system
You are an expert Python programmer and game designer. You create educational language games for training Large
Language Models.<|im_end|>
<|im_start|>user
Create a challenging single-player text-based game as a Python class that tests: <SKILL_NAME>
(<SKILL_DESCRIPTION>).

GAME CONCEPT IDEAS (pick one or invent your own):
<EXAMPLE_GAMES>

RULES:
- Be creative with the game concept - it can be a puzzle, riddle, logic problem, code tracing, word game, optimization
  task, or any reasoning challenge.
- One task per episode. Player answers with \boxed{answer}.
- Give your class a descriptive, unique name.
- The task must be DETERMINISTICALLY SOLVABLE from the observation alone - all information needed to derive
  the answer must be explicitly stated. No hidden state the player cannot see.
- The observation should be clearly written so a careful reader can follow the logic to the answer.

INFORMATION HIDING (critical for RL training):
- The observation must NOT contain the answer. The player must DERIVE it.
- For sequence/pattern games: show only partial data, never the target value.
- Never state the pattern rule directly - the player must discover it.
- Wrong-answer feedback must give hints (e.g., higher/lower, which part is wrong), never reveal the full solution.
- The player must use \boxed{answer} format - remind them in the observation.

DIFFICULTY:
- The task MUST require at least 3 distinct reasoning steps - not solvable in one or two arithmetic operations.
- Random guessing should succeed < 1% of the time - use large answer spaces.
- Use randomized parameters large enough that the answer is not obvious.
- Do NOT generate simple formula-substitution or single-operation problems.
- Compute the solution from the generated puzzle; never hardcode it.
  WRONG: return 'recursive' # same answer regardless of puzzle
  WRONG: return str(random.randint(1,10)) # not derived from the actual task
  RIGHT: generate puzzle first, then compute self._solution from it

INTERFACE CONTRACT:
```python
import random
import re
from typing import Any, Optional, Tuple, Dict, List
from math_verify import parse, verify

def verify_answer(player_answer, solution):
 """Check if answer is correct: string match first, then math equivalence."""
 if str(player_answer).strip() == str(solution).strip():
 return True
 try:
 return verify(parse(player_answer), parse(solution))
 except:
 return False
```

```

class DescriptiveNameEnv:
 def __init__(self, max_turns=10, **kwargs):
 ...
 self.reset()

 def reset(self, seed=None) -> Tuple[str, dict]:
 # Generate a new task. Returns (observation_with_instructions, {})
 ...

 def solution(self) -> str:
 # REQUIRED: Return the exact answer string that solves the current task.
 ...

 def step(self, action: str) -> Tuple[str, float, bool, bool, dict]:
 self.turn_count += 1
 truncated = self.turn_count >= self.max_turns
 match = re.search(r'\\boxed\\{([~}+)}\\}', action)
 if match and verify_answer(match.group(1), self.solution()):
 return ("Correct! ...", 1.0, True, False, {})
 elif truncated:
 return ("Time's up. The answer was {self.solution()}. [task]", 0.0, False, True, {})
 else:
 return ("Wrong. [specific hint about why]. Try again. [task description]", 0.0, False, False, {})
 # EVERY code path must return (str, float, bool, bool, dict) - no exceptions.

 def close(self): pass
...

EPISODE STRUCTURE (critical for RL training):
1. reset(): Generate ONE task/puzzle for this episode
2. step(): Player tries to solve that SAME task multiple times
3. Correct answer -> terminated=True (episode ends with success)
4. Max turns reached -> truncated=True, reveal the solution so the model learns from failure
5. NEVER generate a new task inside step() - the task is fixed for the whole episode

OBSERVATION REQUIREMENTS:
- Turn 1 (reset): Show instructions + task + remind player to use \\boxed{answer} format
- Turn 2+ (step): Show specific feedback on the attempt + the SAME task (no instructions)
- The task/puzzle MUST be visible every turn or the model won't know what to solve
- Feedback must be specific to the attempted answer, not generic ('Wrong. Try again.' is not enough)
- No \\boxed{} in action: remind the player of the format + show task (do not terminate)

ROBUSTNESS:
- Never divide by a value that could be zero - regenerate if needed
- Initialize ALL instance variables in __init__ before calling reset()
- Return empty dict {} for info (never strings)
- Every code path in step() must return a 5-tuple (str, float, bool, bool, dict).
- Naming: never store data in self.solution - that shadows the required solution() method. Use self._solution or self._answer instead.
- Brace escaping: in f-strings, {word} evaluates word as a Python expression. To write a literal \\boxed{answer} escape it as \\boxed{{answer}}. Same rule with .format(): any literal brace must be doubled.

Generate the complete Python code in a ```python block.<|im_end|>
<|im_start|>assistant

```

*Multi-turn environment-generation prompt.* Because gameplay spans multiple turns, the **Environment Designer** also receives a dedicated prompt for generating interactive, multi-turn games (state that evolves across turns, branching actions), shown below with the same placeholder convention.

#### Multi-turn environment-generation prompt (Environment Designer)

```
<|im_start|>system
```

You are an expert Python programmer and game designer specializing in interactive, multi-turn text-based games. You create environments where the player must make sequential decisions across multiple turns, with each action changing the game state.<|im\_end|>

<|im\_start|>user

Create an interactive multi-turn text-based game as a Python class that tests: <SKILL\_NAME> (<SKILL\_DESCRIPTION>).

GAME CONCEPT IDEAS for <SKILL\_NAME> (pick one or invent similar):

- <EXAMPLE\_GAMES>

WHAT MAKES A GOOD MULTI-TURN GAME:

Think of classic text adventures, board games, or strategy games. The player navigates a world that changes with every action. Good examples:

- Grid exploration: move through rooms, find keys to unlock doors, reach the exit
- Trading: buy low / sell high across rounds with fluctuating prices
- Survival: manage health, food, tools while exploring - wrong choices kill you
- Investigation: question suspects, search locations, piece together clues
- Tower defense: place defenses, then waves arrive - adapt strategy each wave
- Crafting: gather materials, combine them in the right order to build something

WHAT TO AVOID (common failure modes):

BAD: A math puzzle where the player guesses the answer and retries on failure.

That is a single-turn puzzle with retry, NOT a multi-turn game.

BAD: Increment/decrement a variable until it matches a target.

That is a linear search, not a game - there are no decisions.

BAD: The player has only one reasonable action each turn (e.g., always 'allocate').

If the optimal path is obvious, there is no strategic depth.

BAD: The game can be solved in 1-3 turns. Too short for multi-turn training.

BAD: Embedding single-turn math/logic puzzles inside a multi-turn frame.

E.g., 'go to forest, solve 2+3, collect wood' - the actual decisions are trivial.

BAD: Using input() to get player answers. NEVER use input(). The action parameter to step() IS the player's full response. Parse everything from it.

DESIGN REQUIREMENTS:

- The game world has STATE that changes on every action (positions, inventories, health, money, unlocked areas, NPC attitudes, etc.).
- Each turn, the player chooses from 2+ meaningfully different actions.
- Some actions are BETTER than others - wrong choices waste turns or cause harm.
- The game will be played for at most 20 turns. Design difficulty, resource budgets, and pacing accordingly. Optimal play should win in roughly 10-15 turns.
- Random play should lose most of the time.
- The game must have a clear WIN condition and ideally a LOSE condition too.

ACTION FORMAT:

- The player wraps every action in \boxed{action}. The step() method extracts the content inside \boxed{} and interprets it.
- Show available actions clearly in every observation, e.g.:  
Actions: \boxed{go north}, \boxed{go south}, \boxed{pick up key}, \boxed{rest}
- If the player's input has no \boxed{}, return a format reminder. Do NOT terminate.

OBSERVATION FORMAT:

- Each observation must show: (1) current state, (2) result of last action, (3) available actions, (4) progress (e.g., 'Turn 3/12 | HP: 7/10 | Items: [key]').
- The initial observation (from reset) includes rules, goal, and starting state.

REWARD:

- Win: reward = 1.0, terminated = True
- Lose: reward = 0.0, terminated = True
- Intermediate turns: reward = 0.0
- Max turns without winning: reward = 0.0, truncated = True

INTERFACE:

```
```python
import random
import re
from typing import Tuple
```



```

from math_verify import parse, verify

def verify_answer(player_answer, solution):
    """Check if answer is correct: string match first, then math equivalence."""
    if str(player_answer).strip() == str(solution).strip():
        return True
    try:
        return verify(parse(player_answer), parse(solution))
    except:
        return False

class DescriptiveNameEnv:
    def __init__(self, max_turns=20, **kwargs):
        self.max_turns = max_turns
        self.turn_count = 0
        # ALL state variables initialized here
        self.reset()

    def reset(self, seed=None) -> Tuple[str, dict]:
        if seed is not None:
            random.seed(seed)
        self.turn_count = 0
        # Randomize the game world
        return observation, {}

    def step(self, action: str) -> Tuple[str, float, bool, bool, dict]:
        self.turn_count += 1
        match = re.search(r'\\boxed\\{([~})*\\}', action)
        if not match:
            return ('Use \\boxed{action} format.', 0.0, False, False, {})
        cmd = match.group(1).strip().lower()
        # Update state, check win/lose
        # Use verify_answer(cmd, solution) for numeric comparisons
        truncated = self.turn_count >= self.max_turns
        return (observation, reward, terminated, truncated, {})

    def close(self): pass
    ...

REFERENCE EXAMPLE - Trading Game (shows the pattern; yours must be DIFFERENT):
```python
class TradingGameEnv:
 """Buy low, sell high across rounds with fluctuating prices. Reach target gold."""
 def __init__(self, max_turns=12, **kwargs):
 self.max_turns = max_turns
 self.turn_count = 0
 self.gold = 0
 self.stock = 0
 self.price = 0
 self.price_history = []
 self.target_gold = 0
 self.trend = 0 # hidden: +1 rising, -1 falling
 self.reset()

 def reset(self, seed=None):
 if seed is not None:
 random.seed(seed)
 self.turn_count = 0
 self.gold = 100
 self.stock = 0
 self.price = random.randint(8, 15)
 self.price_history = [self.price]
 self.target_gold = 200
 self.trend = random.choice([-1, 1])
 return (f'Welcome to the Trading Game! Reach {self.target_gold} gold to win.\n'
 f'Price: {self.price} | Gold: {self.gold} | Stock: {self.stock}\n'
 f'History: {self.price_history}\n')

```

```

 f'Turn 0/{self.max_turns}\n'
 f'Actions: \\boxed{{{buy N}}}, \\boxed{{{sell N}}}, \\boxed{{{wait}}}', {})

def step(self, action):
 self.turn_count += 1
 match = re.search(r'\\boxed{{{([~}]*)\\}}', action)
 if not match:
 return ('Use \\boxed{action} format.', 0.0, False, False, {})
 cmd = match.group(1).strip().lower()
 truncated = self.turn_count >= self.max_turns
 msg = ''
 # Parse action
 if cmd.startswith('buy '):
 try:
 n = int(cmd[4:])
 cost = n * self.price
 if cost <= self.gold and n > 0:
 self.gold -= cost
 self.stock += n
 msg = f'Bought {n} at {self.price}.'
 else:
 msg = f'Cannot buy {n} (need {cost} gold, have {self.gold}).'
 except ValueError:
 msg = 'Invalid number.'
 elif cmd.startswith('sell '):
 try:
 n = int(cmd[5:])
 if n <= self.stock and n > 0:
 self.gold += n * self.price
 self.stock -= n
 msg = f'Sold {n} at {self.price}.'
 else:
 msg = f'Cannot sell {n} (have {self.stock}).'
 except ValueError:
 msg = 'Invalid number.'
 elif cmd == 'wait':
 msg = 'You wait.'
 else:
 msg = 'Unknown action.'
 # Update price with trend + noise
 if random.random() < 0.3: # trend reversal
 self.trend *= -1
 self.price = max(1, self.price + self.trend * random.randint(1, 4))
 self.price_history.append(self.price)
 # Check win
 total = self.gold + self.stock * self.price
 if self.gold >= self.target_gold:
 return (f'{msg} Gold: {self.gold}. You win!', 1.0, True, False, {})
 if truncated:
 return (f'{msg} Time up. Gold: {self.gold}, Stock: {self.stock}. Needed {self.target_gold}.', 0.0, False, True, {})
 obs = (f'{msg}\nPrice: {self.price} | Gold: {self.gold} | Stock: {self.stock}\n'
 f'History: {self.price_history[-5:]}\n'
 f'Turn {self.turn_count}/{self.max_turns}\n'
 f'Actions: \\boxed{{{buy N}}}, \\boxed{{{sell N}}}, \\boxed{{{wait}}}')
 return (obs, 0.0, False, False, {})

def close(self): pass
...

```

#### ROBUSTNESS:

- NEVER use input(). The step(action) parameter IS the player's response.
- Never divide by a value that could be zero.
- Initialize ALL instance variables in `__init__` before calling `reset()`.
- Return empty dict {} for info (never strings).
- Every code path in `step()` must return a 5-tuple (str, float, bool, bool, dict).
- Handle unexpected player input gracefully (show valid actions, don't crash).
- Use only the Python standard library (random, re, collections, etc.).

- Brace escaping: in f-strings, literal braces must be doubled `{{ }}`.
- NEVER put backslashes inside f-string expressions. Use a variable instead:  
BAD: `f'{"\n".join(items)}'`  
GOOD: `sep = '\n'; f'{sep.join(items)}'` OR `'\n'.join(items)`

Generate the complete Python code in a ```python block.

Remember: the game must have BRANCHING DECISIONS where different choices

lead to different outcomes. A game with only one reasonable action per turn is not acceptable.<|im\_end|>

<|im\_start|>assistant

*Corpus-grounded interactive generation prompt.* The canonical games runs use the corpus-grounded variant below: the sampled document replaces the example-concept list, and the specification demands hidden state, multi-turn structure, and partial reward. The skill, difficulty, and document slots are shown in angle brackets.

#### Corpus-grounded multi-turn environment-generation prompt (Environment Designer)

You are given a reference document. Create an INTERACTIVE, MULTI-TURN Python game environment grounded in a concept/technique from this document.

<REFERENCE\_DOCUMENT>

<DOCUMENT\_TEXT>

</REFERENCE\_DOCUMENT>

This MUST be a genuine multi-turn INTERACTIVE environment, NOT a one-shot question-and-answer quiz.

HOW THE AGENT ACTS (this matches the training pipeline - follow it exactly):

- Every turn the agent submits ONE action wrapped as `\boxed{<action>}`, e.g. `\boxed{measure node 3}`, `\boxed{move north}`, `\boxed{open valve A}`. `step()` receives that string; extract the action with `re.search(r'\boxed{\{(.+?)\}}', action)` and treat the extracted text as an interactive COMMAND.
- The extracted text is a COMMAND / move / query that CHANGES the environment - it is NOT a final answer to grade once. `step()` must UPDATE the environment's state from it and return a NEW observation reflecting the changed state.

WHAT MAKES IT MULTI-TURN (required):

- The goal CANNOT be reached in one action. It needs a SEQUENCE of actions - explore to uncover hidden information, then act on it; or manipulate state step-by-step toward a target.
- HIDDEN STATE: the agent does NOT see everything at `reset()`; it must act to reveal/probe state across turns (the observation grows/changes as it acts).
- PARTIAL reward for progress toward the goal; `terminated=True` only when the goal is reached.
- Each turn's observation must (a) reflect the updated state and (b) remind the agent to reply with its next action as `\boxed{<action>}`.

DO NOT (these collapse it back to single-shot QA):

- Do NOT extract `\boxed{answer}`, grade it once, and terminate. `\boxed` carries a per-turn COMMAND, not a final answer.
- Do NOT state the full problem at `reset()` and just check one submitted value.
- Do NOT repeat the SAME static task every turn with only 'wrong, try again' feedback.

GROUNDING: the interaction must require understanding a concept/technique from the document (e.g. if it describes an algorithm, the agent EXECUTES it step-by-step interactively; if it describes a system, the agent OPERATES it over turns). Never reference the document in the game text (no 'according to the passage'); write it as a standalone environment.

INTERFACE CONTRACT (interactive; `\boxed` wraps each turn's COMMAND):

```
```python
import random, re
from typing import Tuple, Dict

class DescriptiveInteractiveEnv:
    def __init__(self, max_turns=12, **kwargs):
        self.reset()
    def reset(self, seed=None) -> Tuple[str, dict]:
        self.turn_count = 0
        # set up HIDDEN state + a goal needing several actions; randomize by seed.
        # return (observation: situation + AVAILABLE ACTIONS + 'reply with \boxed{<action>}',
```

```

#         but NOT the full solution), {}
...
def step(self, action: str) -> Tuple[str, float, bool, bool, dict]:
    self.turn_count += 1
    truncated = self.turn_count >= self.max_turns
    m = re.search(r'\boxed\{(.+)\}', action)
    cmd = (m.group(1) if m else action).strip() # the per-turn COMMAND
    # 1) PARSE cmd into an operation; 2) UPDATE self state; 3) build a NEW observation
    #    reflecting the updated state; 4) reward = partial progress in [0,1],
    #    terminated=True only when the goal is reached.
    # every code path returns (str, float, bool, bool, dict)
    ...
def solution(self) -> str:
    # a reference action-sequence (or goal description) that solves it
    ...
def close(self): pass
...

```

REWARD SCALE: keep total reward in [0, 1]; partial progress < 1.0, full success = 1.0.

ROBUSTNESS (the most common failures - follow ALL of these or the env won't load):

- BRACE ESCAPING: in f-strings and .format(), {word} evaluates word; to write a LITERAL brace you must DOUBLE it. To print \boxed{<action>} inside an f-string, write \boxed{{<action>}}.
- Initialize ALL instance variables in __init__ BEFORE calling reset() (no AttributeError mid-episode).
- Imports: use ONLY the Python standard library plus `math` and `random`. Import everything you use; never call an unimported name (no bare `integrate`, no `np.`, no `x.cos()` - use `math.cos(x)`).
- Every step() code path returns the 5-tuple (str, float, bool, bool, dict); info is always a dict {}.
- Never crash on an unrecognized/garbage command - return a helpful observation + reward 0.0 instead.
- Never divide by a value that could be zero; never store data in self.solution (use self._solution).

SELF-VERIFICATION before finalizing:

- Trace TWO different action sequences from reset(seed=0): does the observation CHANGE based on the actions? (If it never changes, it is NOT interactive - fix it.)
- Does \boxed carry a per-turn COMMAND that evolves state, rather than a final answer that ends it?
- Is partial reward given for progress, kept in [0,1]?

Target skill: <SKILL_NAME> (<SKILL_DESCRIPTION>). DIFFICULTY: <DIFFICULTY> - the goal should need <N>+ interaction steps.

Generate the complete Python code in a ```python block.

Tool-use generation prompt. The tool-use runs generate environments with the multi-turn variant below (selected by SPARE_MULTITURN_ENV_GEN=1 in the released launchers). The skill, difficulty, and example slots fill as in the games prompts; each generation call additionally samples one of ten application domains and one of four task variants (weighted 3:2:1:1), whose filler blocks follow the main template.

Multi-turn tool-use environment-generation prompt (Environment Designer)

Create a MULTI-TURN tool-use environment by subclassing ToolUseBaseEnv.
The environment tests: __SKILL_NAME__ (__SKILL_DESCRIPTION__).

THIS TASK MUST MIRROR THE STRUCTURE OF A REAL MULTI-TURN INTERACTION.

A real user does NOT describe the entire workflow up front. Instead, they issue ONE atomic instruction, wait for it to be done, then issue the next.
Your env must reproduce this turn-by-turn structure.

WHAT YOU IMPLEMENT (the base class handles parsing/dispatch):

- reset(seed) -- generate state, define self._tools, define self._user_messages (list of 3-5 atomic instruction strings), self._message_criteria (list of callables: state -> bool), self._current_msg = 0, self._expected_answer = "done". Return (self._user_messages[0], {})

```

- tool_xxx(**kwargs) -> str -- one method per tool. After mutating
  state, EACH tool method must call self._advance_if_done() and
  APPEND its output to the tool result, like:
  base_result = "Moved file.pdf to /temp"
  progress = self._advance_if_done() # appends next instruction or completion marker
  return base_result + progress

- solution() -> str -- describe the full multi-turn solution

- _advance_if_done(self) -> str -- helper you write yourself:
  ...
  def _advance_if_done(self):
    if self._current_msg >= len(self._user_messages):
      return ""
    criterion = self._message_criteria[self._current_msg]
    if criterion(self._state):
      self._current_msg += 1
      if self._current_msg >= len(self._user_messages):
        return "\n\n[ALL STEPS COMPLETE] Submit <answer>done</answer>."
      next_msg = self._user_messages[self._current_msg]
      return f"\n\n[STEP {self._current_msg} COMPLETE - NEW INSTRUCTION] {next_msg}"
    return ""
  ...

- Override _check_answer(answer) so it returns True only if
  answer == "done" AND self._current_msg == len(self._user_messages).

```

USER MESSAGE STRUCTURE (THE CORE OF MULTI-TURN):

self._user_messages must be 3-5 ATOMIC instructions, e.g.:

```

[
  "Navigate to the document folder.",
  "Move final_report.pdf to a new 'temp' subdirectory.",
  "Now create an 'archive' folder and move all .txt files there.",
  "Finally, list the contents of the archive folder.",
]

```

self._message_criteria are callables that read self._state and return True when the criterion is met:

```

[
  lambda s: s["cwd"] == "/document",
  lambda s: "final_report.pdf" in s["tree"].get("/document/temp", []),
  lambda s: all(f in s["tree"].get("/document/archive", []) for f in s["txt_files"]),
  lambda s: s["last_ls_path"] == "/document/archive",
]

```

DOMAIN RULES:

- Each user message is a SHORT, SPECIFIC, ATOMIC instruction (1-2 sentences).
- Do NOT describe the whole workflow in advance. The user reveals instructions ONE AT A TIME as previous ones are completed.
- Do NOT mention tool names, file structures, or implementation details in user messages. Use natural-language goals only.
- Each instruction's success criterion must be checkable from self._state.
- The actor must complete the CURRENT message before progressing. Trying to skip ahead won't reveal future messages.
- Allow 4-8 tool calls per message.

CRITICAL - SOLVABILITY & CRITERION CORRECTNESS (the single biggest source of BROKEN games):

A game is BROKEN if a success criterion cannot be satisfied by following its instruction, or is already satisfied before the agent acts. Broken games waste training and produce deadlocks. Obey ALL of these:

1. FALSE AT RESET / NOT FREE. Every criterion MUST require the agent to actually perform its instruction's action. For a WRITE instruction, the criterion must check the CHANGE it makes and be False on the state reset() returns - a criterion already True at reset (e.g. `s['inventory']['monitor'] >= 5` when monitor starts at 8, or `any('phone' in i for i in s['electronics'])` when a phone is already stocked) lets the agent skip the step for free. For a READ / VERIFY instruction ("check the line is active", "confirm the balance"), do NOT use a criterion that is already True at reset (e.g. `s['line\_status']=='active'` when it starts 'active') - that advances on ANY tool call without the agent doing the read. Instead have the read tool record that it ran (e.g. set `s['last\_checked']='line\_status'`) and make the criterion check THAT flag, so the step requires actually calling the read tool.
2. TYPE-CORRECT. The criterion must read self._state with the SAME shape the tools write it. If order_history is a list of DICTS, check `any(o['item']=='laptop' for o in s['order\_history'])`, NOT `s['laptop' in s['order\_history']]` - string-in-list-of-dicts is ALWAYS False, so the step can NEVER complete and the game deadlocks. Type mismatches are the most common unsolvable-game bug.
3. DERIVABLE BY THE AGENT. Every value a criterion requires must be obtainable by the agent from EITHER (a) the words of its instruction, OR (b) a tool result it can read. NEVER hide an exact date / id / amount / threshold the instruction does not state and no tool reveals. BAD: instruction "schedule a visit next week" but criterion `s['last\_updated'] >= '2024-01-22'` - the agent cannot know the threshold, must guess, and usually fails. FIX: state it in the instruction ("schedule for 2024-01-22 or later"), OR accept ANY valid action (`s['last\_updated'] != '2024-01-15'`, i.e. any new date), OR have a tool reveal the value. Same for ids: if a criterion needs order '1024', the instruction must name it or a tool must return it.
4. EVERY REQUIRED TOOL CAN SUCCEED. For each instruction, the tool that satisfies its criterion must have a reachable success path. Trace the field the tool looks up - it must be a key some tool (or reset) actually SETS. BAD: tool_process_refund finds a refund by `r['refund\_id']` but tool_create_refund never stores a 'refund_id' key -> process_refund can NEVER succeed. If a tool reads an id/field, an earlier tool or reset MUST write that exact key.
5. INSTRUCTION MATCHES CRITERION. Each instruction must describe exactly what its criterion checks - no more, no less. Do NOT write "after the refund is processed, list inventory" if the criterion only checks that inventory was listed (and process_refund is broken/unneeded): the agent chases the irrelevant step and stalls.
6. SELF-TRACE BEFORE YOU FINISH (REQUIRED). Mentally run reset(seed=0), then execute your own solution() sequence step by step. After EACH solution step the matching criterion must flip False->True, IN ORDER, and no later criterion may be True yet; at the end self._current_msg must equal len(self._user_messages) and _check_answer("done") must be True. If any criterion is True at reset, never becomes True, or throws (KeyError/IndexError), the game is BROKEN - fix it before returning.

 DIFFICULTY: __DIFFICULTY__

__DOMAIN_BLOCK__

__TASK_TYPE_BLOCK__

SKILL FOCUS: __SKILL_CATEGORY__

EXAMPLE TASKS (pick one or invent your own):

__EXAMPLES__

__CORPUS_SECTION__

 RULES:

- Class MUST inherit from ToolUseBaseEnv.

- Do NOT import ToolUseBaseEnv. Just write: `class MyEnv(ToolUseBaseEnv):`
- Only standard library imports (random, json, re, math). No custom packages.
- Tools simulated as methods. No real APIs, network, or subprocess.
- Tools must be DETERMINISTIC. All randomness in `reset()` only.
- 3-8 tools as `tool_xxx()` methods returning strings.
- Give at least one tool a typed/constrained parameter using 'enum' and 'required' in its schema (e.g. `status: {'type':'string','enum':['pending','shipped','cancelled']}`).
- Include 1-2 DISTRACTOR tools: plausible but wrong for the task, so the agent must select the correct one from alternatives.
- Each tool ENDS with ``return base_result + self._advance_if_done()``.
- Mutation tools must error gracefully on bad preconditions (e.g., mv without target dir -> "Error: target dir not found").
- CRITICAL: tool method parameter names MUST EXACTLY MATCH the names declared in `self._tools[name]['parameters']['properties']`. If schema says `{'account': {...}}`, the method MUST be ``def tool_check(self, account=...)``. NOT ``account_type``, NOT ``acct``, NOT a renamed alias. Mismatched names cause every tool call to fail with "got an unexpected keyword argument".

EXAMPLE STRUCTURE (multi-turn filesystem task):

```
```python
import random
import json
from typing import Tuple

class FileWorkflowMTEnv(ToolUseBaseEnv):
 def reset(self, seed=None) -> Tuple[str, dict]:
 self.turn_count = 0
 self._call_history = []
 if seed is not None:
 random.seed(seed)
 self._state = {
 'cwd': '/home',
 'tree': {
 '/home': ['document', 'temp.txt'],
 '/home/document': ['report.txt', 'notes.txt', 'log.txt'],
 },
 'last_ls_path': None,
 }
 self._tools = {
 'pwd': {'description': 'Show current dir', 'parameters': {'type':'object','properties':{}}},
 'ls': {'description': 'List a directory', 'parameters': {'type':'object','properties':{'path':{'type':'string'}}},
 'required':['path']}},
 'cd': {'description': 'Change dir', 'parameters': {'type':'object','properties':{'path':{'type':'string'}}},
 'required':['path']}},
 'mkdir': {'description': 'Create a subdirectory in current dir', 'parameters':
 {'type':'object','properties':{'name':{'type':'string'}}, 'required':['name']}},
 'mv': {'description': 'Move file from current dir to a destination dir', 'parameters':
 {'type':'object','properties':{'src':{'type':'string'},'dst':{'type':'string'}}, 'required':['src','dst']}},
 }
 self._user_messages = [
 "Navigate to the document folder.",
 "Create a 'temp' subdirectory and move report.txt into it.",
 "Now list what's in the temp folder.",
]
 self._message_criteria = [
 lambda s: s['cwd'] == '/home/document',
 lambda s: 'report.txt' in s['tree'].get('/home/document/temp', []),
 lambda s: s['last_ls_path'] == '/home/document/temp',
]
 self._current_msg = 0
 self._expected_answer = "done"
 return (self._user_messages[0], {})

 def _advance_if_done(self) -> str:
 if self._current_msg >= len(self._user_messages):
```

```

 return ""
 if self._message_criteria[self._current_msg](self._state):
 self._current_msg += 1
 if self._current_msg >= len(self._user_messages):
 return "\n\n[ALL STEPS COMPLETE] Submit <answer>done</answer>."
 next_msg = self._user_messages[self._current_msg]
 return f"\n\n[STEP {self._current_msg} COMPLETE - NEW INSTRUCTION] {next_msg}"
 return ""

def _check_answer(self, answer: str) -> bool:
 return answer.strip().lower() == 'done' and self._current_msg >= len(self._user_messages)

def tool_pwd(self) -> str:
 return self._state['cwd'] + self._advance_if_done()

def tool_ls(self, path='') -> str:
 if path not in self._state['tree']:
 return f"Error: {path} not found" + self._advance_if_done()
 self._state['last_ls_path'] = path
 out = json.dumps(self._state['tree'][path])
 return out + self._advance_if_done()

def tool_cd(self, path='') -> str:
 if path not in self._state['tree']:
 return f"Error: {path} not found" + self._advance_if_done()
 self._state['cwd'] = path
 return f"Changed to {path}" + self._advance_if_done()

def tool_mkdir(self, name='') -> str:
 new_path = f"{self._state['cwd']}/{name}"
 if new_path in self._state['tree']:
 return f"Error: {new_path} exists" + self._advance_if_done()
 self._state['tree'][new_path] = []
 self._state['tree'][self._state['cwd']].append(name)
 return f"Created {new_path}" + self._advance_if_done()

def tool_mv(self, src='', dst='') -> str:
 cwd_files = self._state['tree'].get(self._state['cwd'], [])
 if src not in cwd_files:
 return f"Error: {src} not in current dir" + self._advance_if_done()
 dst_path = f"{self._state['cwd']}/{dst}"
 if dst_path not in self._state['tree']:
 return f"Error: target {dst} not found" + self._advance_if_done()
 cwd_files.remove(src)
 self._state['tree'][dst_path].append(src)
 return f"Moved {src} to {dst}" + self._advance_if_done()

def solution(self) -> str:
 return ("1. cd(path='/home/document') 2. mkdir(name='temp') "
 "3. mv(src='report.txt', dst='temp') 4. ls(path='/home/document/temp') "
 "5. <answer>done</answer>")
...

```

Notice:

- self.\_user\_messages defines the multi-turn STRUCTURE
- Each tool\_xxx() ends with self.\_advance\_if\_done() so when criterion is met, the actor sees "[STEP N COMPLETE - NEW INSTRUCTION] ..." inline in the result
- \_check\_answer requires "done" AND all messages completed
- Reset returns ONLY the first instruction; the rest are revealed progressively

Generate the complete Python code in a ```python block.

### Domain and task-variant filler blocks

\_\_DOMAIN\_BLOCK\_\_ (one of ten application domains per call):

REAL-WORLD DOMAIN (build the tools, state, and user goal for THIS domain; do NOT use the reference snippet's academic subject as the domain):  
 \_\_DOMAIN\_\_  
 The task must be a realistic user accomplishing a goal here (search, book, order, update, cancel, transfer, schedule) - NOT an exam or puzzle about any document.

\_\_TASK\_TYPE\_BLOCK\_\_ (one per call; execute and read\_then\_write are the common cases):

TASK VARIANT: execute (happy-path). The user issues atomic instructions; the agent calls the right tools in order. Each criterion checks the resulting state.  
 TASK VARIANT: read\_then\_write. At least ONE instruction must force the agent to FIRST query state with a read tool, branch on a predicate over the result (e.g. 'for every order over \$100', 'all lines past their data cap'), THEN make the dependent write. Its criterion must check the final state reflects the predicate-conditioned writes - not merely that one tool was called.  
 TASK VARIANT: missing\_value. At least ONE instruction references a value the user does NOT state (e.g. 'cancel my most recent order' with no order\_id; 'pay the overdue invoice' with no invoice id). The agent must DISCOVER it via a read tool before the write. The write tool MUST return an error if given a wrong/guessed value, so a fabricated argument cannot satisfy the criterion.  
 TASK VARIANT: with\_irrelevant. Include EXACTLY ONE instruction that NONE of the domain tools can satisfy (e.g. in a retail env, 'what will the weather be tomorrow?'). Add a tool named 'decline\_request' (one required string param 'reason') whose method sets a state flag and then advances; that instruction's criterion is met ONLY by calling decline\_request. Provide NO other tool that could plausibly fulfill the irrelevant request.

*Reasoning Agent gameplay prompt.* The Reasoning Agent receives each turn's observation wrapped in the template below and answers in `\boxed{}` form.

#### Reasoning Agent gameplay prompt

```
<|im_start|>user
You are playing a language game. Make valid actions to win.
Observation: <OBSERVATION>
Please reason step by step, and put your final answer within \boxed{<|im_end|>
<|im_start|>assistant
```

## C.2 Implementation Details

Two auxiliary Environment Designer reward terms are disabled in all full-SPADE runs: a per-environment EMA-based learning-potential bonus (Kanitscheider et al., 2021; Zhang et al., 2023), enabled only as the standalone reward replacement evaluated in Section 7.2, and a frontier bonus (extra reward for environments with a large fast-versus-slow EMA gap  $|\mu_{\text{fast}} - \mu_{\text{slow}}|$ ). No separate variance bonus (a  $\bar{r}_A(1 - \bar{r}_A)$ -style reward for mixed outcomes) is used. Failed environment candidates are regenerated up to a fixed number of attempts. Table 5 lists the full training configuration.

**Pool lifecycle.** At each regeneration the previous environment set is deleted and replaced; the environment memory evicts oldest-first at its 200-record cap; rejected candidates are persisted for inspection; generation retries up to five attempts per environment.

**Hint generation detail.** The privileged hint is produced by a separate designer-side call conditioned on the generated environment's source code: the hint writer sees the code, while the Reasoning Agent never does.

**Validation detail.** Every candidate environment must pass a programmatic smoke test: the class is instantiated, `reset()` is called, and a few probe actions are stepped through, discarding candidates that fail to parse or crash. The tool-use setting adds the two semantic checks of Section 5.3: a deterministic reset gate that rejects environments in which a success criterion errors on the freshly reset state under every tested seed, and an LLM screen that rejects impossible environments (unreachable, pre-satisfied, or underivable success criteria) while keeping hard-but-feasible ones.

**Table 5 Training hyperparameters.** Shared across the three games-setting SPADE backbone runs; per-model exceptions appear in parentheses.

Hyperparameter	Value
Models	Qwen3-4B-Instruct-2507, Qwen3-8B, Qwen3-30B-A3B-Instruct-2507
Learning rate	$1 \times 10^{-6}$ (constant)
Optimizer	Adam, $\beta = (0.9, 0.98)$ , weight decay 0.1
KL penalty $\beta_{\text{KL}}$	0 (0.005 for 8B)
Clipping $\varepsilon_{\text{low}}/\varepsilon_{\text{high}}$	0.20/0.28
Truncated importance sampling	yes
Reward normalization	outcome-only, per-game $z$ -score
Rollout batch	24
Global batch	192 (dynamic)
Group size $G$	16
Total rollouts	400
Environments per rollout	24 ( $8 \times 3$ active skills of 6, round-robin)
Regeneration interval $k$	4 rollouts
Environment Designer temperature	0.6
Environment Designer max tokens	16,384 (20,000 for 8B)
Reasoning Agent temperature	0.6
Reasoning Agent max tokens	8,192
Max turns per episode	25
Max context length	32,768 (49,152 later in the 4B run)
Environment Designer reward blend	plateau 0.6, band $[0.4, 0.6]$ ( $[0.2, 0.4]$ later for 4B) + floored regret 0.4
Regret scale (normalizer)	0.15
Plateau ramp width	0.25
Delayed Environment Designer update	4 rollouts
Corpus grounding	15k docs (10k math, 5k science)
Environment memory	on

### C.3 Reproducibility

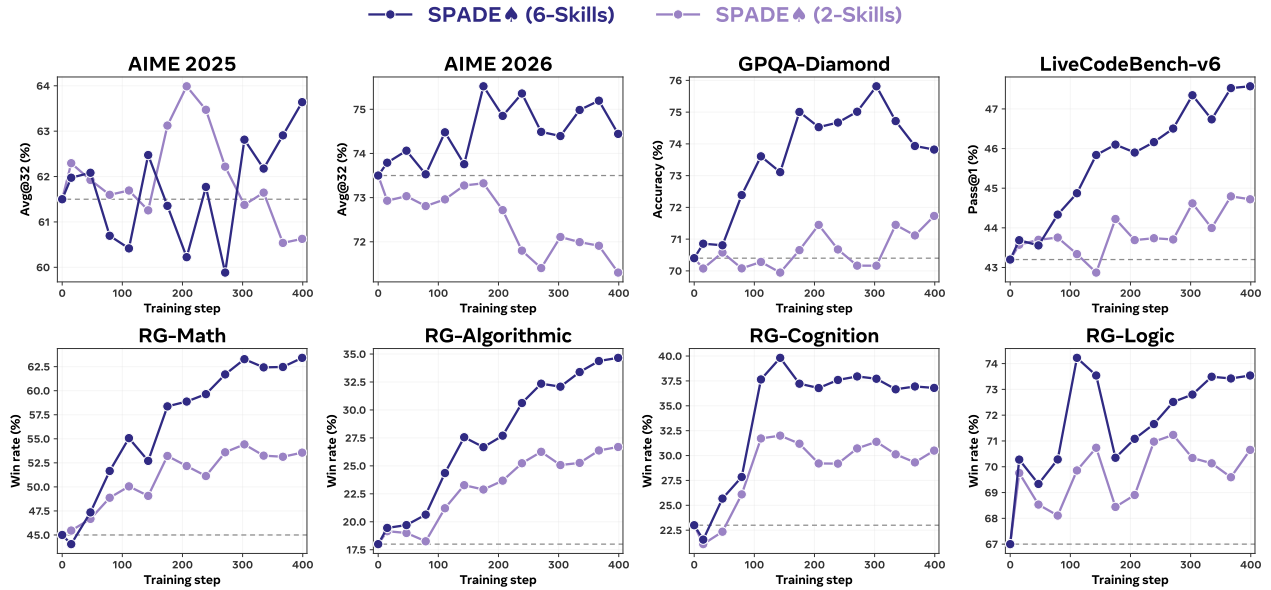
The training and evaluation code is released. The configuration files for every run reported here are released with the paper. The evaluation output JSONs and the scripts that produce every figure are included alongside the code.

### C.3.1 Comparability Notes for Table 2

The reference systems in Table 2 were trained and evaluated by their own authors on their own harnesses; we reprint their published scores for context and record the protocol differences here. **BFCL versions.** Our rows and Agent-World (Dong et al., 2026) report the BFCL v4 multi-turn suite. AWM (Wang et al., 2026b) and EnvScaler (Song et al., 2026) label the suite v3, which contains the same four multi-turn subcategories; EnvScaler evaluates a frozen 2024-09-22 data snapshot whose repository notes minor differences from later releases.  **$\tau^2$ -bench variants.** AWM evaluates  $\tau^2$ -bench-verified, a fork with task corrections, runs the Telecom domain in solo mode (agent only, no user simulator), and uses a GPT-5.1 user simulator in the other domains; AgentScaler (Fang et al., 2025a) does not state its user simulator. We evaluate the standard  $\tau^2$ -bench release. **Aggregation.** Reference-row Avg cells follow the rule stated in the table caption; Agent-World and AWM print their own aggregates (61.8/65.4 and task-weighted 33.5/39.0), which do not equal the unweighted means of the domain scores they report. **Base models.** Our rows post-train Qwen3-4B-Instruct-2507, Qwen3-8B, and Qwen3-30B-A3B-Instruct-2507; the reference systems post-train their own base checkpoints, with training data and budgets that differ from ours.

## D Extended Ablations

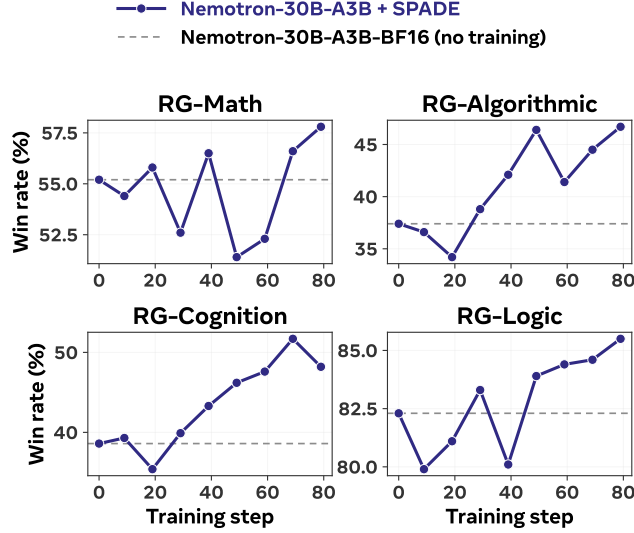
[\[back to contents\]](#)



**Figure 14 The full 6-skill curriculum lifts held-out benchmarks more than the restricted 2-skill variant; curriculum breadth drives the gains.** Qwen3-30B-A3B-Instruct-2507, games setting. Top row: AIME 2025/2026 Avg@32, GPQA-Diamond accuracy, and LiveCodeBench-v6 Pass@1. Bottom row: the four Reasoning-Gym categories; the dashed line marks the untrained base model. Discussed in Section 8.

**Cross-family transfer.** As a check on a non-Qwen backbone, we ran the same recipe on Nemotron-30B-A3B-BF16, evaluating only the Reasoning-Gym suite; AIME 2025/2026, GPQA-Diamond, and LiveCodeBench-v6 were not evaluated on this backbone, so this checks that the training dynamics are not Qwen-specific rather than serving as a matched cross-family comparison. All four Reasoning-Gym categories rise above the untrained base, with the largest Nemotron gains in RG-Cognition (+9.6) and RG-Algorithmic (+9.3; Figure 15).

Table 6 expands the component-controlled ablations of Table 3 to every games-setting variant reported in this paper, including the skill-diversity run, with each variant’s selected checkpoint and its GEM overall win rate. Checkpoints are selected per variant by the best suite average (the mean of the eight benchmark columns); the corresponding evaluation trajectories appear in Figures 10, 14, and 11.



**Figure 15** SPADE improves a second backbone family: all four Reasoning-Gym categories end above the untrained Nemotron-30B-A3B-BF16 base (RG-Cognition +9.6, RG-Algorithmic +9.3, RG-Math +2.6, RG-Logic +3.2). Reasoning-Gym win rate across checkpoints; the dashed line marks the untrained base model. Gains arrive after an initial dip early in training, consistent with the **Environment Designer** initially generating environments too hard for the **Reasoning Agent**, as observed on the Qwen backbones.

**Table 6** Full ablation breakdown (games setting, Qwen3-30B-A3B-Instruct-2507). Best checkpoint per variant on the suite average. AIME reports Avg@32; GPQA-D accuracy; LCB-v6 Pass@1; Reasoning-Gym (RG) win rate at HARD; GEM the overall win rate across the GEM game suite (Liu et al., 2025c). Best in **bold**.

Setting	Ckpt	AIME'25	AIME'26	GPQA-D	LCB-v6	RG-Math	RG-Algo.	RG-Cog.	RG-Logic	GEM	Avg
Qwen3-30B-A3B-Instruct-2507	–	61.5	73.5	70.4	43.2	45.0	18.0	23.0	67.0	41.0	50.2
<b>SPADE</b> ♠	303	<b>62.8</b>	<b>74.4</b>	<b>75.8</b>	<b>47.3</b>	<b>63.3</b>	<b>32.1</b>	<b>37.7</b>	<b>72.8</b>	<b>50.2</b>	<b>58.3</b>
Environment Designer w/ learning potential	–	62.4	74.1	74.2	46.1	57.8	27.9	33.3	71.1	47.4	55.9
2-skill curriculum	399	60.6	71.3	71.7	44.7	53.6	26.7	30.5	70.7	46.0	53.7
w/o corpus grounding	111	61.1	74.1	71.8	46.3	51.6	22.3	32.4	68.7	45.1	53.5
w/o memory	111	59.3	<b>75.0</b>	72.3	45.7	49.1	22.9	30.7	70.9	42.4	53.2
w/o Environment Designer training and memory	271	59.4	73.5	65.8	39.1	22.5	10.0	7.6	46.0	26.4	40.5
Fixed Environment Designer (GPT-5.5)	175	59.9	72.8	74.2	42.6	51.2	24.3	30.7	68.0	45.6	53.0



## E.1 Self-Play for LLMs

Self-play has been a cornerstone of AI since [Tesauro et al. \(1995\)](#) trained TD-Gammon through self-play in backgammon. AlphaGo ([Silver et al., 2016](#)) combined Monte Carlo tree search with deep neural networks and self-play to defeat a human Go champion, and AlphaZero ([Silver et al., 2017, 2018](#)) extended the approach to chess, shogi, and Go from tabula rasa with no human data. OpenAI Five ([Berner et al., 2019](#)) and AlphaStar ([Vinyals et al., 2019](#)) scaled multi-agent self-play to Dota 2 and StarCraft II, while Cicero ([FAIR](#)) combined language models with strategic reasoning in Diplomacy. [Littman \(1994\)](#) formalized the Markov-game framework underlying these results, and [Irving et al. \(2018\)](#) proposed debate as an AI-safety mechanism grounded in self-play. The asymmetric self-play paradigm of [Sukhbaatar et al. \(2017\)](#), in which one agent proposes challenges while another solves them, provides the conceptual template for teacher-student environment design.

Translating self-play to LLMs introduces new challenges because rewards are sparse, outputs are discrete sequences, and the “environment” is open-ended language. SPIN ([Chen et al., 2024](#)) applies self-play fine-tuning by training the LLM to distinguish its own outputs from human demonstrations, converting weak models to strong ones. Self-Rewarding Language Models ([Yuan et al., 2024](#)) let the LLM judge its own outputs to generate preference data, iteratively improving both generation and evaluation. SPAG ([Cheng et al., 2024](#)) uses adversarial language games (Adversarial Taboo) to incentivize strategic reasoning through a zero-sum objective. ReST<sup>EM</sup> ([Singh et al., 2023](#)) alternates between sampling model solutions filtered by a binary correctness reward and supervised fine-tuning on the filtered correct rollouts. Prover-Verifier Games ([Kirchner et al., 2024](#)) train a prover to produce legible solutions that a weaker verifier can check, improving output interpretability. ReMA ([Wan et al., 2025](#)) decomposes reasoning via multi-agent RL into a high-level meta-thinking agent (strategy and decomposition) and a low-level reasoning agent (step-by-step execution), trained jointly.

The most relevant line of recent work uses LLMs as their own source of training data in self-play, with varying degrees of reliance on external data. At one extreme, fully data-free methods bootstrap from minimal seeds: Absolute Zero Reasoner (AZR) ([Zhao et al., 2025](#)) has a single model (shared parameters) propose and solve code-reasoning triplets (deduction, abduction, induction) with proposer reward  $1 - \bar{r}_{\text{solve}}$  computed from solver Monte-Carlo success rates verified by a code executor, bootstrapping from a single identity function. R-Zero ([Huang et al., 2025](#)) instead instantiates a Challenger and a Solver as two independently optimized copies of the same base LLM, training the Challenger via GRPO with a self-consistency uncertainty reward; the paper reports that this two-model design outperforms a single-model variant in ablations but plateaus after a few iterations. PopuLoRA ([Castanyer et al., 2026](#)) extends this asymmetric paradigm to co-evolving populations of LoRA-adaptor teachers and students with cross-evaluation between sub-populations. In a related but mechanistically distinct line, G-Zero ([Huang et al., 2026](#)) performs DPO-based self-distillation over hint-induced preference pairs: a Proposer trained via GRPO emits a hint  $h$  for query  $q$ , the Generator  $\pi_G$  produces both an unassisted response  $a_{\text{hard}} \sim \pi_G(\cdot | q)$  and a hint-conditional response  $a_{\text{assisted}} \sim \pi_G(\cdot | q, h)$ , and DPO is applied with  $a_{\text{assisted}}$  as chosen and  $a_{\text{hard}}$  as rejected; the Proposer’s reward is Hint- $\delta$ , the per-token mean log-probability shift induced on  $a_{\text{hard}}$  when the hint is prepended to the Generator’s context. All supervision is derived from a single model’s own outputs under hint vs no-hint contexts, placing the method closer to the self-rewarding and self-distillation line ([Chen et al., 2024](#); [Yuan et al., 2024](#); [Singh et al., 2023](#)) than to verifier-grounded agent-environment co-evolution. Tool-R0 ([Acikgoz et al., 2026](#)) adapts the same Challenger-Solver paradigm to tool use, training the generator with a solve-rate-band reward plus a semantic alignment score and the solver with soft tool-call matching, similarly finding that separate parameters outperform a shared-weight setup. Self-Questioning Language Models ([Chen et al., 2025c](#)) have a single model propose and solve problems from only a topic prompt, with the proposer rewarded for majority-vote calibration (problems neither always solved nor always missed). Language Self-Play ([Kuba et al., 2025](#)) proposes a data-free framework where question generation and answer improvement reinforce each other. PasoDoble ([Zhang et al., 2025d](#)) pairs an adversarial proposer and solver grounded in a pretraining knowledge base, sustaining improvement beyond R-Zero’s plateau. At the other extreme, methods that retain limited seed data or curated evaluation sets include SeRL ([Fang et al., 2025b](#)), which uses a few-shot-prompted question generator with a difficulty-band filter and trains only the solver via Reinforce++ on majority-vote

rewards from 500 seed instructions, and Sundaram et al. (2026), who ground the teacher’s reward in measured student improvement on curated target problems instead of proxy statistics. Self-Challenging Language Model Agents (Zhou et al., 2025) let agents propose harder task variants for themselves and then attempt to solve them. Autodata (Kulikov et al., 2026) generalizes the self-instruct line by casting an LLM agent as a meta-optimizing data scientist that iteratively constructs training data; its Agentic Self-Instruct instantiation selects examples by the separation between a strong and a weak reference solver, targeting learnable examples over merely hard ones, a frontier-targeting principle analogous to SPADE’s hint-based regret but operating over offline data creation with fixed reference solvers, without the online co-evolution of a gradient-trained generator. Vision-Zero (Wang et al., 2025a) brings gamified multi-agent self-play (Who Is the Spy?) to vision-language models with iterative self-play policy optimization. SPELL (Yang et al., 2025) co-trains a three-role self-play loop (questioner, responder, verifier) on long documents, using a history-memory curriculum to generate progressively harder questions for evolving long-context capabilities. SPC (Chen et al., 2025b) trains a self-play critic via adversarial games to improve LLM reasoning evaluation. TextArena (Guertler et al., 2025) provides an open-source collection of 57+ competitive text-based game environments with a Gym-compatible API for LLM evaluation and self-play training. Sarkar et al. (2025) train LLMs for Among Us-style social deduction with multi-agent RL. Self-Play with Execution Feedback (Dong et al., 2024) uses code execution to verify instruction-following in a self-play loop. SPIRAL (Liu et al., 2025a) introduces self-play on zero-sum games as a multi-agent, multi-turn RL framework that incentivizes reasoning through game-theoretic competition. SPICE (Liu et al., 2025b) extends self-play to corpus environments via information asymmetry: a Challenger mines documents from a large corpus to generate diverse reasoning tasks with document-grounded answers, while a Reasoner solves them without document access; the corpus prevents hallucination amplification and information-symmetry collapse seen in ungrounded self-play. SWE-RL (Wei et al., 2026) applies RL to open software evolution tasks, and SSR (Wei et al., 2025) trains software engineering agents through self-play. Anchored Self-Play (Choi et al., 2026) has a single model play a bug-generator and a fixer for code repair with a difficulty-band reward, adding reference-bug mixing and embedding-similarity shaping to keep self-generated bugs on the realistic distribution, preventing drift off it, and GASP (Jana et al., 2026) guides asymmetric self-play for coding LLMs with grounding in real data. Chae et al. (2025) provide a systematic analysis of when LLM self-play succeeds and when it fails, analyzing the “invisible leash” where generation quality is bounded by base-model capability. Shafayat et al. (2025) investigate whether large reasoning models can self-train and find that naive self-play degrades performance without careful curriculum control. Liu et al. (2026b) analyze when self-synthetic self-play actually improves, showing that sustained evolution requires the synthetic pipeline to guarantee learnable information gain.

Two themes emerge across this body of work. First, all data-free self-play methods generate *tasks* (a problem statement paired with a sparse terminal reward) rather than full environments with state transitions. Second, the generator in every data-free case is either frozen or trained with heuristic proxy rewards (learnability, difficulty, variance) that risk reward hacking and distributional drift. By contrast, SPADE generates *full MDP environments* (state space, transition function, reward function, and verification code) as executable Python, and trains the **Environment Designer** via RL with hint-based regret that is grounded in measured **Reasoning Agent** return rather than proxy statistics.

## E.2 Unsupervised Environment Design and Open-Endedness

Curriculum learning (Bengio et al., 2009) established the principle that ordering training examples from easy to hard accelerates learning. Quality-Diversity (QD) algorithms, exemplified by MAP-Elites (Mouret and Clune, 2015), illuminate the search space by maintaining an archive of high-performing solutions across a structured behavior space, providing the algorithmic substrate for many open-ended environment-generation systems. Paired Open-Ended Trailblazer (POET) (Wang et al., 2019) pioneered the paradigm of co-evolving environments alongside agents, using evolutionary search to grow a population of parameterized BipedalWalker terrains together with the policies that solve them, with explicit transfer attempts moving high-performing agents across environments. Unsupervised Environment Design (UED) extends this idea by automatically generating training environments instead of selecting from a fixed pool. PAIRED (Dennis et al., 2020) formalized UED via a minimax regret objective: an adversary generates environment parameters to maximize the gap between an antagonist’s and a protagonist’s returns, producing curricula of increasing complexity while avoiding unsolvable environments. Dennis et al. (2020) proved that at Nash equilibrium, the protagonist plays a

minimax regret policy, connecting UED to decision theory. Prioritized Level Replay (PLR) (Jiang et al., 2021b) replaced the learned adversary with a replay buffer that prioritizes high-regret levels, achieving comparable curriculum quality with lower computational cost. Replay-Guided Adversarial Environment Design (Jiang et al., 2021a) combined the adversarial generator with prioritized replay, using the replay distribution to guide the adversary toward high-regret regions of the environment space. ACCEL (Parker-Holder et al., 2022) introduced evolutionary mutations of high-regret levels, compounding complexity over training without domain-specific heuristics and recovering minimax regret guarantees at a fraction of the compute required by population-based methods like POET. Mediratta et al. (2023) stabilized PAIRED-style learned adversaries via entropy regularization and behavioral cloning between protagonist and antagonist, addressing the entropy collapse that destabilizes UED. Rutherford et al. (2024) investigated regret approximations used in PLR and ACCEL, showing that common proxies (positive value loss, maximum Monte Carlo) correlate with success rate rather than true regret, and proposing Sampling For Learnability (SFL) as an improved estimator. Monette et al. (2025) recast UED as a nonconvex-concave optimization over a categorical level distribution, proving convergence to first-order Nash equilibria and generalizing learnability scores to continuous-return settings. CENIE (Teoh et al., 2024) augmented regret-based UED with a novelty objective that measures how much a candidate environment pushes the agent into unexplored regions of the state-action space, finding that novelty and regret are synergistic rather than competing. DISCOVER (Diaz-Bone et al., 2026) provides a complementary formal-analysis view from goal-conditioned RL: it scores candidate goals via current-policy value estimates balancing achievability, relevance, and novelty, and proves a UCB-style bound on time-to-target-achievability that depends only on the agent’s initial distance to the target, independent of task-space volume. The three-axis selection rule parallels SPADE’s hint-based regret as a frontier-targeting curriculum signal, but DISCOVER operates over a fixed goal pool whereas SPADE generates the environment distribution itself.

A broader thread of open-endedness research motivates the need for unbounded environment spaces. Hughes et al. (2024) argue that open-endedness, defined as simultaneous novelty and learnability, is essential for superhuman AI, and that systems plateau when the environment parameterization is exhausted. Baker et al. (2019) demonstrated emergent tool use from multi-agent autotutorials in hide-and-seek, showing that competitive self-play in rich physics environments produces increasingly sophisticated strategies. OMNI (Zhang et al., 2023) uses foundation models as a “Model of Interestingness” to filter open-ended task proposals, focusing the curriculum on tasks that are both learnable and interesting. OMNI-EPIC (Faldor et al., 2024) extends this line by representing each environment as executable code generated by a foundation model, with a frozen FM judge of interestingness gating which generated tasks enter the archive; SPADE shares the code-as-environment representation but trains the **Environment Designer** via RL with hint-based regret, with no frozen interestingness judge. Imagined Autotutorials (Güzel et al., 2025) applies PLR inside a learned diffusion world model trained from offline data, demonstrating that UED principles transfer to imagined environments. SIMA 2 (Bolton et al., 2025) scales open-ended self-improvement to embodied 3D worlds: a Gemini-based agent uses foundation models to generate its own tasks and rewards and acquires new skills in previously unseen and even model-generated (Genie 3) environments, though it distributes the process across separate task-setter, agent, and reward models plus a distinct generative world model instead of a single policy that authors its own environments. Goldfeder et al. (2026) argue for Superhuman Adaptable Intelligence over AGI, emphasizing speed of adaptation over static benchmark performance. PAPRIKA (Tajwar et al., 2025) trains generally curious agents on ten hand-designed task groups, showing that diverse multi-turn training produces transferable exploration strategies.

All classical UED methods operate in *parameterized* environment spaces (maze dimensions, terrain friction, grid layouts) with small, fixed design vocabularies. To our knowledge, SPADE is the first system to bring *regret-based UED with an RL-trained **Environment Designer** that produces full MDP environments* to LLM post-training: the environment space is unbounded (arbitrary Python code), the **Environment Designer** is the LLM rather than a separate adversary network, and hint-based regret provides the co-evolution signal without requiring a trained antagonist.

### E.3 Synthetic Environment Generation

A growing body of work uses LLMs to programmatically generate training environments for agentic RL, addressing the bottleneck of hand-curated environment design. Agent World Model (AWM) (Wang et al.,

2026b) decomposes environment synthesis into five stages (scenario, task, database, interface, verification) to produce 1,000 SQLite-backed tool-use environments with MCP interfaces, training agents via GRPO with hybrid step-level and task-level rewards. ScaleEnv (Tu et al., 2026) generates multi-turn tool-use environments by synthesizing API specifications, databases, and verification code from seed domains, scaling to thousands of environments for generalist interactive agent training. TermiGen (Zhu et al., 2026) synthesizes high-fidelity terminal environments inside Docker containers with automated trajectory verification, producing diverse command-line tasks for terminal agent training. Nemotron-Terminal (Pi et al., 2026) introduces Terminal-Task-Gen, a two-stage pipeline that combines *dataset adaptation* (transforming math, code, and SWE benchmarks into terminal-formatted prompts) with *synthetic task generation* (seed-based and skill-based, the latter driven by a Skill Taxonomy of nine domains and primitive skills); the resulting Terminal-Corpus is open-sourced, and Qwen3-32B post-trained via SFT reaches 27.4% on Terminal-Bench 2.0 (from a 3.4% baseline), surpassing Qwen3-Coder-480B (23.9%) at a fraction of the parameter count. SkillSynth (Fan et al., 2026) constructs a scenario-mediated skill graph with 82,073 scenarios as nodes, 57,214 filtered skills as directed transitions, and 185,529 LLM-verified bridges, samples directed paths through the graph as workflow abstractions, and instantiates them via a multi-agent harness with dual execution-based and rubric-based verification (95.7% oracle pass rate, 3,560 verified task instances per run); the explicit objective is maximizing the diversity of execution trajectories over raw task count. EurekaVerse (Liang et al., 2024) uses LLMs to generate Python terrain-program curricula (height-field functions) for robot skill learning, producing progressively harder physics environments guided by agent performance feedback, and validates on quadrupedal parkour. LLM-in-Sandbox (Cheng et al., 2026) places LLMs in sandboxed code-execution environments and shows that agentic intelligence emerges from multi-turn interaction with executable feedback. EvoCUA (Xue et al., 2026a) evolves computer-use agents by synthesizing scalable GUI interaction experiences, generating training trajectories across diverse desktop applications. Xue et al. (2026b) extend this to autonomous continual learning where computer-use agents adapt to new environments through self-generated experience. DreamGym (Chen et al., 2025e) trains an LLM-based world model on demonstration trajectories from existing agentic environments (WebShop, ALFWorld, WebArena) and uses it to generate abstract-text rollouts that replace expensive real-environment interactions during RL. Simia (Li et al., 2025) pushes this direction further by removing real environments entirely: Simia-SFT amplifies small seed trajectories into diverse SFT data via reasoning-model-simulated feedback, and Simia-RL performs PPO/GRPO directly against LLM-generated environment transitions; fine-tuned 7–32B Qwen and Llama models reach 36–59 average on  $\tau^2$ -bench (Airline+Retail subset) without ever executing real tool code, but the framework still inherits the LLM-as-transition-model hallucination risk and the reduced two-domain  $\tau^2$  setup is not directly comparable to the three-domain Avg used elsewhere. Lu et al. (2025) argue that tuning the environment (reward shaping, observation design) can be more effective than tuning the agent, providing a complementary perspective on environment optimization. Exploratory Iteration (Jiang et al., 2025b) grows a self-improvement task space by sampling informative intermediate solution iterates from previous episodes as starting points for new single-step training tasks, training  $K$ -step inference-time self-improvement while only training on single-step transitions. Golden Goose (Lu et al., 2026) synthesizes RLVR tasks from unverifiable internet text by extracting verifiable sub-claims, converting passive corpora into training signal without requiring curated datasets. AutoEnv (Zhang et al., 2025b) provides automated environments for measuring cross-environment agent transfer, enabling systematic evaluation of generalization. Synthetic Sandbox (Zhou et al., 2026) generates sandboxed ML-engineering environments for training software agents on realistic development workflows. RLAnything (Wang et al., 2026a) proposes a dynamic RL system that jointly optimizes policy and reward model while adapting the task distribution via perturbation of existing tasks. GenEnv (Guo et al., 2025) targets difficulty-aligned environment generation with a curriculum reward signal. Agent-World (Dong et al., 2026) mines real-world environments and toolsets from web content (1,978 environments, 19,822 tools) and applies a self-evolving training arena that diagnoses agent weaknesses and generates targeted tasks across rounds. AgentScaler (Fang et al., 2025a) clusters over 30,000 real APIs into more than 1,000 simulated tool-use domains via Louvain community detection on a parameter-similarity graph, materializes each tool as a Python class operating on a per-domain database schema, and trains 4B/8B/30B-A3B agents via two-phase supervised fine-tuning on filtered agent-user trajectories without RL. EnvScaler (Song et al., 2026) programmatically synthesizes tool-interactive environments for multi-turn tool-use agent training, and EnvFactory (Xu et al., 2026b) scales executable tool-use environment synthesis paired with a stable RL recipe. From Trainee to Trainer (Chen et al., 2026) narrows the adaptivity gap by letting the current RL checkpoint reconfigure its own environment generator from diagnosed failures across training rounds, though it tunes a fixed set of



generator parameters and does not emit executable environment code. AutoPlay (Ramrakhya et al., 2025) addresses task synthesis for UI agents (mobile and desktop) by first running an MLLM explorer in each app to gather state and functionality information, then having a frozen GPT-4o task-generator condition on those exploration trajectories plus task-guideline prompts to propose grounded tasks; the executor is trained via SFT and GRPO with binary outcome rewards from an MLLM verifier, gaining 20.6 points on AndroidWorld over Qwen2.5-VL-7B. InfiniteWeb (Zhang et al., 2026e) synthesizes complete static-HTML/localStorage websites alongside tasks and dense-reward JavaScript evaluators via task-centric test-driven development; UI-TARS-1.5-7B post-trained with GRPO on 600 generated tasks improves on OSWorld from 24.5 to 31.4 and on Online-Mind2Web from 23.0 to 28.7, with the largest gains concentrated in easy and medium difficulty buckets. EvoEnv (Shi et al., 2026) co-trains a single LLM under generator and solver role-conditioning via shared-parameter GRPO, producing reusable Python verifier-and-prompt artifacts whose oracle and scorer remain frozen at training time; the generator reward combines staged validation, solver-relative difficulty calibration targeting 30% solver accuracy, and an embedding-based novelty bonus, with evaluation on three model families restricted to single-shot reasoning benchmarks (Qwen3-4B-Thinking-2507 improves from 72.4 to 74.8 averaged across eight math, code, and science benchmarks). The environment interface is single-shot scoring (sampler, oracle, renderer, scorer) without state evolution or step-level rewards, so the framework does not extend to multi-turn agentic settings such as tool use or terminal workflows; SPADE differs in three respects: (i) the **Environment Designer** is rewarded via hint-based regret grounded in **Reasoning Agent** return rather than validation and novelty signals, (ii) environments are full MDPs with reset/step interfaces that unify single-turn and multi-turn settings, and (iii) evaluation spans games and tool use in addition to reasoning.

These methods produce useful training material, and some incorporate feedback-driven adaptation where the generation distribution shifts in response to agent progress: Eurekaverse evolves environments based on agent performance statistics, and EvoCUA iterates synthesis with an improving policy. However, even in these cases the generator LLM receives no RL gradients; adaptation operates through prompting heuristics or iterative data filtering, without joint optimization. By contrast, SPADE’s **Environment Designer** is a genuine RL learner, trained via hint-based regret in the same optimization loop as the **Reasoning Agent**, producing full MDP environments at the **Reasoning Agent**’s competence frontier.

## E.4 Environment Scaling

A growing consensus holds that the next frontier for agentic RL is scaling environments rather than algorithms. Silver and Sutton (2025) articulate this vision most directly, arguing that the field is entering an “era of experience” in which the primary bottleneck is generating sufficiently diverse and abundant training environments. Zhang et al. (2026b) argue in the same vein that scalable, diverse environments are what drive generalizable agents. AgentRL (Zhang et al., 2025a) provides a multi-turn, multi-task RL framework with a fully asynchronous generation-training pipeline, cross-policy sampling for improved exploration, and task advantage normalization, training a single agent across five agentic domains to outperform frontier models. SCALER (Xu et al., 2026a) converts competitive programming problems (from CodeContests) into parameterized single-turn reasoning environments at controlled difficulty levels, demonstrating that scaling the number and diversity of training problems yields consistent improvements in mathematical reasoning. RLVE (Zeng et al., 2025) creates 400 hand-engineered verifiable environments for RL training with adaptive difficulty levels and per-environment custom verifiers (mixing rule-based parsing, soft scoring, and code execution) in place of LLM-as-judge. WebScale-RL (Cen et al., 2025) builds an automated data pipeline that harvests web content to create RL training environments at pretraining scale, showing that more diverse environments produce better generalization. Endless Terminals (Gandhi et al., 2026) procedurally generates 3,255 verified terminal-use tasks via a four-stage pipeline and demonstrates that vanilla PPO with binary rewards yields substantial gains as the number of training environments scales, with transfer to TerminalBench 2.0. Self-Evolving Curriculum (Chen et al., 2025d) dynamically adjusts the difficulty distribution of training problems based on the learner’s current performance, preventing saturation on easy examples. TTCS (Yang et al., 2026) co-evolves a gradient-trained problem synthesizer with a solver via GRPO, using a capability-adaptive reward that targets the solver’s learning frontier at test time. Xu et al. (2025) survey the path toward Large Reasoning Models, organizing the field into three pillars (automated data construction, learning-to-reason via RL, and test-time scaling) and identifying environment scaling as a key open problem. Song

et al. (2024) examine self-improvement capabilities of LLMs and find that naive self-training saturates quickly, motivating the need for adaptive curricula. Embodied Co-Design (Wang et al., 2025b) provides a taxonomy of co-design approaches where agent morphology and controller evolve jointly, drawing parallels to SPADE’s co-evolution of environment and agent. Agent Learning via Early Experience (Zhang et al., 2025c) introduces a reward-free training paradigm in which agents propose alternative actions at expert-visited states and learn from the resulting future states (implicit world modeling and self-reflection), bridging imitation learning and full RL across eight agentic environments.

These works independently demonstrate that more environments yield better generalization and that curriculum design matters at least as much as algorithm choice. Several incorporate adaptive mechanisms: RLVE adjusts per-environment difficulty levels, SCALER tracks accuracy for difficulty control, and Self-Evolving Curriculum learns a sampling policy over problem categories. SPADE combines gradient-trained environment design with the code-as-environment representation, producing an adaptive curriculum of executable MDPs that co-evolves with the Reasoning Agent.

## E.5 Agentic Memory Design and Self-Improving Code Systems

MemRL (Zhang et al., 2026d) introduces self-evolving agents that learn via runtime reinforcement learning on episodic memory, storing and retrieving past interaction experiences to improve future decision-making without additional gradient updates. ALMA (Xiong et al., 2026) meta-learns agentic memory designs themselves: a foundation model searches the space of Python memory-architecture programs (an open-ended scaffold over update/retrieve operations), discovering memory designs that improve continual learning at test time. HyperAgents (Zhang et al., 2026a) extends self-referential code evolution by fusing the task-solver and the meta-improver into a single editable program, so the self-modification mechanism is itself modifiable; tasks are externally fixed and the foundation model is frozen. These directions are complementary to SPADE: they address how agents *use* accumulated experience or *rewrite their own scaffolding* at the meta-level, while SPADE addresses how to *generate* the training environments that produce that experience in the first place via RL. HyperAgents explicitly leaves co-evolving the task distribution as future work, which SPADE addresses directly. The approaches are composable: SPADE’s adaptive environment generation could populate the experience streams that ALMA-style memory systems and MemRL-style runtime RL operate over.

## F Extended Quantitative Analysis

[\[back to contents\]](#)

Throughout, environment text is embedded with SBERT (all-MiniLM-L6-v2) (Reimers and Gurevych, 2019), and every embedding-based result is replicated under a second, purely lexical embedding (TF-IDF with LSA-128), following the multi-embedding robustness protocol of Abdulhai et al. (2026); conclusions are identical under both. Step-level training dynamics use the matched 30B-A3B runs (full SPADE is the resume-stitched 0–399 step run; ablations end earlier), with curves EMA-smoothed over real logged values, expanding on Section 6.2.

### F.1 Games

The games-setting analyses below expand the environment-quality and diversity signals summarized in the main text.

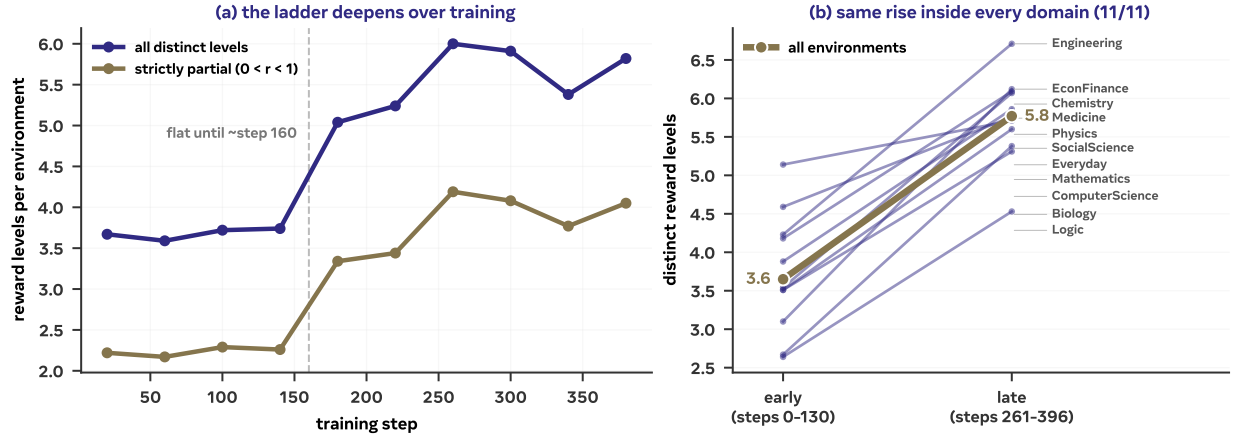
#### F.1.1 Environment Quality

We track five quality signals; four are tracked over training (Table 7) and executability is measured once over the raw generations. **(i) Learnability (ground truth):** the fraction of environments in the learnable band (Reasoning Agent win rate in  $[0.2, 0.8]$ ) rises from 0.16 to 0.31, and Reasoning Agent win-rate rises  $0.30 \rightarrow 0.62$ : the Environment Designer keeps targeting tasks at the frontier of the improving agent (Figure 6). **(ii) Well-posedness:** an LLM rubric over all environments scores 97 to 98 % well-posed, flat over training. **(iii) Verifiability:** 90 to 93 % of environments have an objectively checkable terminal answer, flat to slightly rising. **(iv) Executability:** re-executing raw generated code in a sandbox (parse, instantiate, `reset()`), 84.9 % runs as emitted and 90.3 % after stripping a stray Markdown fence; the dominant residual failure is a systematic f-string brace-escaping bug in `\boxed{}` action templates (7.4 %), which the training pipeline’s sanitizer repairs before execution (100 % post-filter validity). **(v) Richness:** environments hold steady at  $\sim 320$  lines of code,  $\sim 13$  hidden state variables, and 8 to 10 interaction turns per episode; the Environment Designer does not shortcut to trivial programs to inflate its reward. Reward granularity also rises, from 3.7 to 5.8 distinct levels per environment ( $2.2 \rightarrow 4.0$  strictly partial; Figure 16).

Quality signal	Early (0–40)	Mid (150–250)	Late (340–396)
Learnable-band fraction (win-rate $\in [0.2, 0.8]$ )	0.16	0.16	<b>0.31</b>
Reasoning Agent win-rate	0.30	0.46	<b>0.62</b>
Well-posed (LLM rubric)	0.98	0.97	0.97
Verifiable terminal answer (LLM rubric)	0.90	0.91	0.93
Interaction depth (turns / episode)	8.8	8.2	9.8
Program length (lines of code)	316	333	321
Hidden state variables	13.0	13.3	13.4

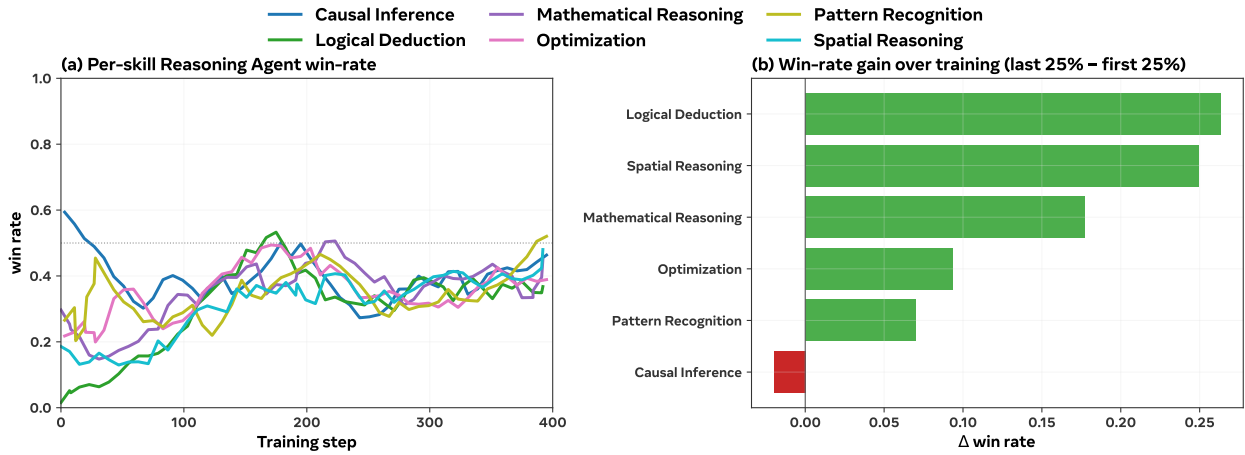
**Table 7 Environment quality over training.** Learnability nearly doubles while well-posedness, verifiability, and structural richness hold constant: the quality gains come from sharper difficulty targeting rather than simpler environments. Win-rate rows use the released per-step evaluation logs.





**Figure 16 Reward granularity increases over training.** Left: mean distinct reward levels per environment, including strictly partial levels. Right: early-to-late change in distinct levels overall and by domain (canonical 30B games run).

Figure 17 gives the per-skill decomposition of the Reasoning Agent’s gains behind Table 7.



**Figure 17 Per-skill learning** (canonical SPADE-30B run). **(a)** Per-skill Reasoning Agent win rate over training (EMA over  $\sim 26$  logged points/skill). **(b)** Win-rate gain (last 25% minus first 25%): Logical Deduction and Spatial Reasoning improve most; Causal Inference, which starts high, declines slightly.

### F.1.2 Environment Diversity

*Metric and calibration.* Within each training step’s batch we report the Vendi Score (Friedman and Dieng, 2023), the effective number of distinct items in a sample,

$$VS(K) = \exp \left( - \sum_i \lambda_i \log \lambda_i \right), \quad (7)$$

where  $\lambda_i$  are the eigenvalues of the normalized cosine-similarity matrix  $K/n$  over SBERT embeddings. The score is calibrated: batches of 24 identical environments score 1.0, single-domain batches score 14.8 to 16.5, and mixed batches drawn across the whole run score 21.3 (the practical ceiling for  $n=24$ ).

*Diversity is maintained at the ceiling.* Per-step Vendi is flat for the whole run, 20.8 (steps 0 to 40) versus 21.0 (steps 340 to 396), directly at the mixed-population ceiling; mean pairwise cosine distance is likewise flat (0.94). Novelty stays saturated (97 of 100 logged steps consist entirely of never-seen initial states), and

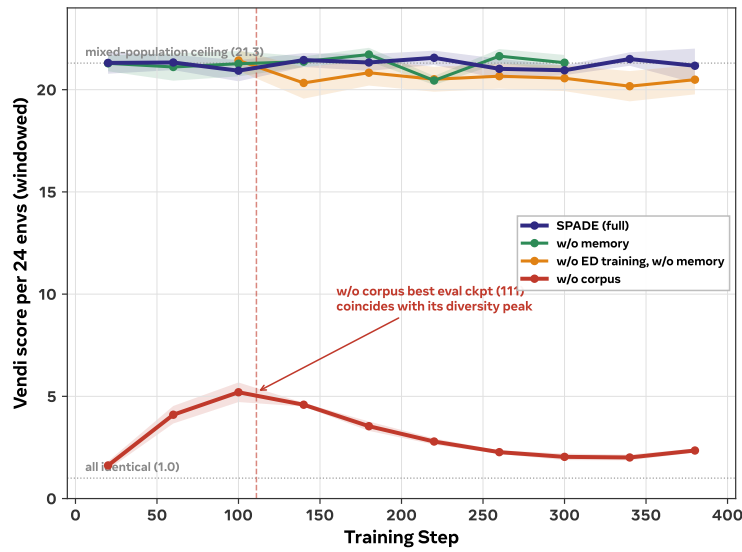
all 13 domains of an LLM-labeled taxonomy are present from step 0 with 8.4 per batch of 24 on average (Mathematics 30 %, Physics 20 %, Medicine 11 %, Chemistry 10 %, CS 7 %, Engineering 6 %, other 16 %). The distribution is also stationary: after removing seed-document reuse, a linear probe cannot separate early-half from late-half environments (5-fold AUC  $0.551 \pm 0.024$ ).

*Corpus grounding drives diversity.* The ablations quantify the matched-step generation contrast of Appendix G (Table 8, Figure 18). The attribution is two-sided: freezing the **Environment Designer** and removing memory while keeping the corpus leaves diversity at the mixed-population reference level (0.70), whereas training the **Environment Designer** without the corpus collapses it to 0.04. Corpus grounding drives the diversity; memory and **Environment Designer** training do not. The windowed dynamics (Figure 18) sharpen this further: the no-corpus run starts near-collapsed (Vendi  $\sim 1.6$ ), RL exploration briefly lifts it to a peak of  $\sim 5.2$  around step 100, and continued optimization then re-collapses it to  $\sim 2$ . That run’s best evaluation checkpoint (step 111, Table 6) coincides with its diversity peak; once the environment stream re-collapses, downstream evaluation stops improving. The corpus-grounded runs hold the ceiling for their full run lengths under the same optimization pressure (full SPADE for all 400 steps).

Run	Sample	Vendi/ $n$ $\uparrow$	Mean pairwise dist. $\uparrow$	Reading
SPADE (full)	3,310	0.68	0.94	diverse
w/o memory	3,746	0.69	0.94	diverse
w/o <b>Environment Designer</b> training, w/o memory	4,929	<b>0.70</b>	0.94	diverse
w/o corpus	866	<b>0.04</b>	0.34	collapsed

**Table 8 Corpus ablations collapse environment diversity.** Vendi Score per 100 environments (SBERT embeddings; mean over 20 balanced draws) on the verified-matched 30B-A3B runs: identical backbone, skill set, and **Environment Designer** system prompt, differing in the listed ablations. The identical analysis under TF-IDF/LSA embeddings reproduces the corpus/no-corpus separation (0.53/0.58/0.48/0.05). A fifth run with a static pre-generated environment pool (**Reasoning Agent**-only training, no live **Environment Designer**) is excluded as a different generation protocol; its fixed pool measures Vendi/ $n = 0.12$ .

*Scope and limitations.* Per-environment scalar rewards are not retained in the public run log, so learnability is measured at the step level (win rate, learnable-band fraction) rather than per environment; the LLM rubric measures absolute difficulty and coherence rather than fit to the current agent; and executability is measured on raw generations, upstream of the pipeline’s sanitizer. Diversity numbers use batch size 24 with larger boot-step batches subsampled for comparability.



**Figure 18 Environment-diversity dynamics across the verified-matched runs.** Windowed Vendi score (40-step windows, subsampled to exactly 24 environments; mean  $\pm$  s.d. over 12 draws). The corpus-grounded variants hold the mixed-population reference level for their full run lengths; the no-corpus run starts near-collapsed, peaks at Vendi  $\sim 5.2$  near step 100, where its best evaluation checkpoint (111) also falls, and re-collapses as optimization continues. Environment counts differ across runs with run length and acceptance rate; the windowed score uses fixed 24-environment subsamples, so counts do not bias the curves.

## G Extended Qualitative Analysis

[\[back to contents\]](#)

**Summary.** Analyzing all 3,310 environments generated by the canonical SPADE-30B run, each a distinct program, seeded by one of 1,513 distinct documents drawn from the 15k-document corpus, four results emerge. **(1) Quality:** the share of environments in the learnable band nearly doubles ( $0.16 \rightarrow 0.31$ ) while well-posedness, verifiability, and program richness hold constant. **(2) Diversity:** semantic diversity stays at the mixed-population ceiling for all 400 training steps, and the stream keeps producing never-seen environments to the end. **(3) Mechanism:** the diversity comes from corpus grounding; removing the corpus collapses generation to a single task family (Vendi/ $n$ : 0.68 vs. 0.04), whereas freezing **Environment Designer** training or the memory leaves it intact. **(4) Jointly:** difficulty targeting improves within an unchanged task distribution, so the curriculum sharpens in difficulty while holding its breadth.

We analyze all 3,310 environments generated by the canonical SPADE-30B run at the level of their content: the seed document each was generated from, the generated program, and the opening observation. Every generated environment is a distinct program (3,310 of 3,310 unique program hashes; 2,388 distinct initial states; 1,513 distinct seed documents), so all analyses operate on content, never on surface identifiers such as class names (585 distinct; 58% keep the scaffold default despite the prompt’s naming instruction). This appendix presents the qualitative evidence; the matching proxy metrics and training dynamics appear in Appendix F.

### G.1 Games

For the games setting we give additional privileged-hint examples and trace how a single environment evolves from early to late training.

#### G.1.1 Additional Privileged-Hint Examples

Figure 19 expands the main-text selection to four positive-regret task–hint pairs from the same canonical 30B games run, spanning the regret range from large frontier gaps to a small mastery-regime gap. Each pair is drawn from one logged game record: the task description is a concise summary of its reset observation, while the hint is a verbatim excerpt of the substantive guidance appended to the with-hint arm.

<b>Geometric fiber pattern</b> Mean return 0.00 → 1.00	<b>Seed-saving compliance</b> Mean return 0.85 → 0.95
<b>Task</b>   Identify a consistent target-property region in a 5×5 grid of geometric fibers. Probe chosen coordinates, then analyze the discovered pattern.   <b>Hint (verbatim)</b>   “The good fibers form a diagonal band where row index plus a small offset equals column index. Probe points along such a diagonal (e.g., (0,1), (1,2), (2,3)) to detect the pattern.”	<b>Task</b>   Decide whether a farmer may legally save a new seed variety. Probe the available evidence, then declare compliance.   <b>Hint (verbatim)</b>   “To determine if seed saving is legal, you must gather all four pieces of evidence: seed type and protection status, patent status, yield loss percentage, and legal agreement implications. Only after collecting all information can you correctly declare compliance.”
<b>Audio-filter balancing</b> Mean return 0.30 → 0.65	<b>Double-slit coherence</b> Mean return 0.00 → 0.81
<b>Task</b>   Restore high-frequency clarity in a noisy analog channel by setting and enabling preemphasis and deemphasis filters, then transmitting and evaluating the received signal.   <b>Hint (verbatim)</b>   “The system requires matching the preemphasis and deemphasis time constants to counteract severe high-frequency attenuation. Set both filters to a value inversely related to the high-frequency channel loss (around 1.0 to 3.0 typically works).”	<b>Task</b>   Make the interference fringes disappear as source width increases. First measure the wavelength, source distance, and slit separation; then compute and set the critical source width.   <b>Hint (verbatim)</b>   “The critical source width that causes fringe disappearance is calculated as (wavelength × source_distance) / slit_separation. After measuring all three parameters, set the source width to this computed value to satisfy the coherence condition.”

**Figure 19 Task context makes hint utility legible.** Four same-record, positive-regret task–hint pairs from the canonical 30B games run. The task summaries retain the goal, hidden information, and usable interaction while removing generic runtime scaffolding. Hint excerpts omit only the standardized answer-format sentence and runtime wrapper. In each header, the two values report the mean return without hint / with hint.

## G.1.2 Environment Evolution over Training

The cards below show one early and one late environment in the exact format produced by the **Environment Designer** (cf. the minimal example in Appendix G.3.1): a seed document grounds the task; the generated program implements a Gym-style interface with hidden state and a verifiable terminal answer; the opening observation states the goal and action space. The corpus’s DCLM slice is filtered web text and includes off-topic documents; the early card’s personal-finance seed is one such document. The visible shift over training is toward structured, instrumented “laboratory” tasks, while program size, functional hidden state, and verifiability hold steady (about half the early card’s raw state variables are write-only narrative fields): the added difficulty comes from state-gated interaction and finer reward grading, not from longer programs.

Training step 0 (early): CarOwnershipDisputeEnv	370 lines, 29 state variables
Seed document (web-scraped): “I have a car, both me and by grandpa are on the title. As I cannot get loan on my own, my grandpa is my cosigner. My grandpa is almost 90, and chances are he may not be around till the loan is paid...”	
<pre> class CarOwnershipDisputeEnv:     def __init__(self, max_turns=12, seed=None):         self.max_turns = max_turns         self.seed = seed         self.reset(seed)      def reset(self, seed=None) -&gt; Tuple[str, dict]:         self.turn_count = 0         self.rng = random.Random(seed if seed is not None else 42) </pre>	

```

Hidden state: the actual legal situation is not fully visible at start
self.loan_balance = 12000
self.car_value = 6000
self.is_paid_off = False
self.grandpa_alive = True
self.grandpa_will_made = self.rng.choice([True, False]) # Hidden: whether will exists

```

Opening observation (what the Reasoning Agent sees):

You are in a tense family situation. Your grandfather is on the title and co-signer of your car loan. You've made consistent payments for 3 years, but still owe \$12,000 on a car worth only \$6,000. Your aunt has threatened to take the car when your grandfather passes. You're unsure if the will exists or what it says. You know your grandfather is still alive and mentally competent. You have not yet consulted a lawyer ... ..

Training step 384 (late): ThermodynamicCycleManipulationLabEnv

376 lines, 19 state variables

Seed document (web-scraped): "Subject: physics Problem: A gas has  $N$  atoms in volume  $V_0$  at temperature  $T_0$ . The gas is heated at constant volume up to temperature  $3T_0$ , then allowed to expand isothermally up to volume..."

```

class ThermodynamicCycleManipulationLabEnv:
 def __init__(self, max_turns=12, seed=None):
 self.max_turns = max_turns
 self.seed = seed
 self.reset(seed)

 def reset(self, seed=None) -> Tuple[str, dict]:
 self.turn_count = 0
 self._seed = seed if seed is not None else random.randint(1, 10000)
 random.seed(self._seed)

 # Hidden: thermodynamic system parameters (not directly visible)
 # Represents a monatomic ideal gas undergoing a three-step cycle
 self.n_atoms = random.randint(100, 500) # N: number of atoms
 self.initial_volume = random.uniform(1.0, 5.0) # V0: initial volume in m3
 self.initial_temperature = random.uniform(100, 300) # T0: initial temperature in K

```

Opening observation (what the Reasoning Agent sees):

You enter a high-precision thermodynamics lab. Before you is a transparent cylindrical chamber containing a cloud of gas atoms. The chamber is sealed and connected to external controls:

Available actions: - activate heating (starts constant-volume heating) - initiate expansion (starts isothermal expansion) - begin cooling (starts constant-pressure cooling) - measure current entropy (reveals cumulative ... ..

The complete source of both environments (369 and 376 lines as generated) is reproduced verbatim in Appendix G.3.2.

### G.1.3 Matched-Step Generation Contrast

The clearest qualitative view of diversity is a matched-step contrast against the no-corpus ablation. At training steps 290 to 312, SPADE generates environments spanning probability theory, quantum gate tomography, volcanology, hematology, radar signal processing, and hypoelliptic operators, while the no-corpus run generates the *same* rotating-maze navigation task 41 times in a row, varying only the grid layout. Grounding each generation in a sampled corpus document supplies the stream of new task material.

Matched-step generation contrast (training steps 290–312)

SPADE (with corpus), one batch at step 296 (24 environments, 24 distinct programs):

ProbabilitySpaceExplorationEnv,	QuantumGateSetTomographyLabEnv,	OptimizationLabEnv,
NearDoublesMathLabEnv,	RadarDopplerAnalysisLabEnv,	MinimaxStrategyLabEnv,
		HypoellipticOperatorLabEnv,



TrigonometricApproximationLabEnv, plus 16 default-named environments, each a distinct program on a distinct seed document (pattern recognition, genetics, circuit analysis, ...).

**w/o corpus, steps 290–312 (41 environments):**

RotatingMazeEnv, RotatingMazeEnv, RotatingMazeEnv, ... (all 41 generations are the same rotating-obstacle grid-navigation family; only the maze layout and minor rule text vary).

Counts are accepted generations; the two runs' acceptance rates differ over this window.

## G.2 Tool Use

The tool-use **Environment Designer** emits environments in a different register from the games setting: each is a simulated API domain (banking, retail orders, support tickets, telecom accounts, smart-home control) in which a user issues one atomic instruction at a time, and the environment advances only when a programmatic criterion over the hidden state confirms the current instruction is complete. The cards below reproduce one early and one late environment exactly as generated: a code-corpus snippet seeds the generation, the **Environment Designer** builds an unrelated everyday domain on top of it, and each user instruction is paired with a checkable criterion over the backend state (in the early card, coupling the tool call's provenance with its state effect). Criteria check the salient state effect; qualifiers in the instruction text, such as which account pays an invoice, are not always bound, an artifact the validation checks do not catch.

### Training step 8 (early): BankingMultiTurnEnv

286 lines, 8 tools

Seed document (web-scraped): “package com.mayo.client.mayoclientapi.persistence.repository; import com.fasterxml.jackson.databind.ObjectMapper; import com.google.api.core.ApiFuture; import com.google.cloud.Timestamp; import com.google.cloud.firestore.\*;...”

```
self._user_messages = [
 "List all accounts I have.",
 "Find the checking account and check its current balance.",
 "Transfer $1000 from my savings account to my checking account.",
 "Now pay the unpaid invoice with ID INV2024-003 using my checking account.",
 "Finally, block the card ending in 2468 because it was lost."
]
Define criteria that must be met for each step to advance
self._message_criteria = [
 lambda s: s['last_searched_account'] is not None and s['last_searched_account'] in s['accounts'],
 lambda s: s['last_query_result'] is not None and s['last_query_result']['account_id'] == 'ACC112233' and
 ↪ s['last_query_result']['balance'] > 0,
 lambda s: s['last_transfer_from'] == 'ACC445566' and s['last_transfer_to'] == 'ACC112233' and s['last_transfer_amount'] ==
 ↪ 1000.0,
 lambda s: s['last_invoice_paid'] == 'INV2024-003',
 lambda s: s['last_card_status_change'] == 'blocked'
]
```

#### Opening observation (what the Reasoning Agent sees):

List all accounts I have. ...

### Training step 392 (late): CustomerSupportTicketWorkflowEnv

200 lines, 6 tools

Seed document (web-scraped): “import json import os import sqlite3 import openai from pymongo import MongoClient from pymongo.errors import OperationFailure import tiktoken...”

```
self._user_messages = [
 "Find all high-priority tickets that are still in 'new' status.",
 "Assign the ticket with ID TICKET-001 to agent_01.",
 "Add a note to ticket TICKET-001 stating 'Customer confirmed issue is reproducible.'",
 "Update the status of ticket TICKET-001 to 'in_progress'.",
 "Finally, confirm that ticket TICKET-001 has been successfully resolved by marking it as 'resolved'."
]
self._message_criteria = [
 lambda s: len([t for t in s['tickets'] if t['priority'] == 'high' and t['status'] == 'new']) == 2,
 lambda s: any(t['id'] == 'TICKET-001' and t['assigned_to'] == 'agent_01' for t in s['tickets']),
 lambda s: any(t['id'] == 'TICKET-001' and any('Customer confirmed issue is reproducible' in n for n in t['notes']) for t in s['tickets']),
 lambda s: any(t['id'] == 'TICKET-001' and t['status'] == 'in_progress' for t in s['tickets']),
]
```

```
lambda s: any(t['id'] == 'TICKET-001' and t['status'] == 'resolved' for t in s['tickets'])
]
```

Opening observation (what the Reasoning Agent sees):

Find all high-priority tickets that are still in 'new' status. ...

Across the full 30B run the structural profile of generated environments is stationary, mirroring the distributional stationarity of the games setting (Appendix F.1.2): mean program length moves from 247 to 226 lines between the first and last training band, tools per environment from 7.6 to 5.9, while instructions per environment (4.7) and the logical complexity of the per-step criteria (1.9 to 2.0 conditions per criterion) stay flat, and roughly one environment in five ships a guarded failure path (locked cards, unavailable agents, denied requests) that the **Reasoning Agent** must detect and recover from. What changes over training is not the scaffold but the instantiations rotated through it: the **Environment Designer** holds a stable inventory of everyday API domains while regenerating fresh states, identifiers, and instruction sequences each round, exactly the regime the corpus-grounding and memory components are designed to sustain.

### G.3 Generated Environment Gallery

This gallery collects representative environments in the exact format the **Environment Designer** emits, starting from a minimal multi-turn example.

#### G.3.1 A Minimal Generated Environment

Listing 2 (reproduced from Listing 1) shows a minimal multi-turn environment in the format produced by SPADE’s **Environment Designer**: a single Python class exposing the Gym-style `reset()/step()` interface described in Section 3.1, with internal state, per-step feedback, and a verifiable terminal reward. This one worked example stands in for the gallery: the same interface and training pipeline also carry single-turn answer-grading tasks and multi-turn tool-use interactions.

**Listing 2 A minimal SPADE-generated environment.** A Wordle-flavored multi-turn deduction game implemented as a single Python class with Gym-style `reset()/step()` interface. The episode is stateful (`self.target`, `self.turns_left`), ends either on a correct guess (terminated, reward 1) or at the 6-turn limit (truncated, reward 0).

```
import random

class WordleEnv:
 WORDS = ["spade", "trace", "lemon", "graph"] # truncated

 def reset(self, seed=None): # initial state
 self.target = random.Random(seed).choice(self.WORDS)
 self.turns_left = 6
 return "Guess a 5-letter word in 6 tries.", {}

 def step(self, guess): # (s', r, term, trunc, info)
 self.turns_left -= 1
 left = [t for g, t in zip(guess, self.target) if g != t]
 fb = ""
 for g, t in zip(guess, self.target):
 if g == t: fb += "G"
 elif g in left: fb += "Y"; left.remove(g)
 else: fb += "_"
 if fb == "GGGGG":
 return fb, 1.0, True, False, {}
 return fb, 0.0, False, self.turns_left == 0, {}
```

### G.3.2 Full Source of Two Exemplar Generated Environments

The two environments carded in Appendix G.1.2 are reproduced here in full, exactly as emitted by the Environment Designer (lightly transliterated to ASCII: typographic quotes, dashes, arrows, and sub-/superscripts). Each is a complete, self-contained program: hidden state initialized in `reset()`, a multi-turn `step()` loop with intermediate feedback, and a verifiable terminal reward. Episode return is the final-step reward, clipped to  $[-1, 1]$  and 0 if the episode never terminates; the intermediate per-step rewards visible below are progress feedback, not summed.

*Training step 0: CarOwnershipDisputeEnv (370 lines).*

```

1 import random
2 import re
3 from typing import Tuple, Dict
4
5 class CarOwnershipDisputeEnv:
6 def __init__(self, max_turns=12, seed=None):
7 self.max_turns = max_turns
8 self.seed = seed
9 self.reset(seed)
10
11 def reset(self, seed=None) -> Tuple[str, dict]:
12 self.turn_count = 0
13 self.rng = random.Random(seed if seed is not None else 42)
14
15 # Hidden state: the actual legal situation is not fully visible at start
16 self.loan_balance = 12000
17 self.car_value = 6000
18 self.is_paid_off = False
19 self.grandpa_alive = True
20 self.grandpa_will_made = self.rng.choice([True, False]) # Hidden: whether will exists
21 self.grandpa_will_content = self.rng.choice(['gives car to OP', 'gives car to aunt', 'no mention']) if self.grandpa_will_made else None
22 self.grandpa_owns_title = True # Co-owner
23 self.op_owns_title = True
24 self.loan_coborrower = True
25 self.loan_cosigner = True
26 self.loan_lender_has_lien = True
27 self.title_in_name = True # Title exists in both names
28 self.loan_docs_contain_death_clause = self.rng.choice([True, False])
29 self.aunt_threatening = self.rng.choice([True, False])
30 self.op_paying_payments = True
31 self.op_has_good_payment_history = True
32 self.op_credit_score = 580 # Poor
33 self.op_has_refinancing_attempt = False
34 self.op_has_consulted_lawyer = False
35 self.grandpa_is_competent = True
36 self.grandpa_will_likely_sign = self.rng.choice([True, False])
37 self.grandpa_has_signed_transfer = False
38 self.grandpa_has_refinanced = False
39 self.grandpa_has_gifted_title = False
40 self.legal_right_to_car = True # OP has right due to ownership and payments
41
42 # Goal: Ensure OP keeps the car after Grandpa's death
43 # This requires multiple steps: verify legal rights, assess will, consider refinancing, or get title transfer.
44
45 # Initial observation - hidden state, partial information
46 obs = (
47 "You are in a tense family situation. Your grandfather is on the title and co-signer of your car loan. "
48 "You've made consistent payments for 3 years, but still owe $12,000 on a car worth only $6,000. "
49 "Your aunt has threatened to take the car when your grandfather passes. "
50 "You're unsure if the will exists or what it says. "
51 "You know your grandfather is still alive and mentally competent. "
52 "You have not yet consulted a lawyer or attempted refinancing. "
53 "The loan documents may have a clause about death of a co-signer.\n\n"
54 "Available actions:\n"
55 " - check loan documents\n"
56 " - talk to grandpa about the will\n"
57 " - talk to grandpa about transferring title\n"
58 " - research refinancing options\n"
59 " - consult a lawyer\n"
60 " - check car title status\n"
61 " - review state laws on joint ownership\n\n"
62 "Reply with your next action as \\boxed{<action>}."
63)
64 return obs, {}
65
66 def step(self, action: str) -> Tuple[str, float, bool, bool, dict]:
67 self.turn_count += 1
68 truncated = self.turn_count >= self.max_turns

```

```

69 m = re.search(r'\\boxed\\{(\\.+?)\\}', action)
70 cmd = (m.group(1) if m else action).strip().lower()
71
72 reward = 0.0
73 terminated = False
74 info = {}
75
76 # Parse and execute command
77 if cmd == "check loan documents":
78 if self.loan_docs_contain_death_clause:
79 obs = (
80 "You reviewed the loan documents. There is a clause: 'If either borrower dies, the remaining balance becomes due

81 ↳ immediately.\\n"
82 "This means if your grandfather dies, you may have to pay off the full $12,000 immediately unless refinanced.\\n\\n"
83 "Next steps:\\n"
84 " - talk to grandpa about transferring title\\n"
85 " - research refinancing options\\n"
86 " - consult a lawyer\\n"
87 "Reply with \\boxed{<action>}.")
88 reward = 0.2
89 else:
90 obs = (
91 "You reviewed the loan documents. There is no death clause. The loan remains active and payable by you regardless of your

92 ↳ grandfather's passing.\\n"
93 "This is good news - you are not immediately liable for the full balance.\\n\\n"
94 "Next steps:\\n"
95 " - talk to grandpa about the will\\n"
96 " - check car title status\\n"
97 " - talk to grandpa about transferring title\\n"
98 "Reply with \\boxed{<action>}.")
99 reward = 0.2
100
101 elif cmd == "talk to grandpa about the will":
102 if not self.grandpa_alive:
103 obs = (
104 "Your grandfather has passed. You cannot talk to him now.\\n"
105 "You must now rely on the will or state law.\\n\\n"
106 "Next steps:\\n"
107 " - check if a will exists\\n"
108 " - consult a lawyer\\n"
109 " - verify car title status\\n"
110 "Reply with \\boxed{<action>}.")
111 reward = 0.1
112 elif self.grandpa_will_made:
113 obs = (
114 "You talked to your grandfather. He confirms he has a will. He says he intends to leave the car to you.\\n"
115 "He will sign the will soon.\\n\\n"
116 "Next steps:\\n"
117 " - verify the will's content\\n"
118 " - talk to grandpa about transferring title\\n"
119 " - consult a lawyer\\n"
120 "Reply with \\boxed{<action>}.")
121 reward = 0.3
122 else:
123 obs = (
124 "You talked to your grandfather. He says he hasn't made a will yet. He is open to writing one.\\n"
125 "He is willing to transfer the car to you before he dies.\\n\\n"
126 "Next steps:\\n"
127 " - talk to grandpa about transferring title\\n"
128 " - consult a lawyer\\n"
129 " - research title transfer process\\n"
130 "Reply with \\boxed{<action>}.")
131 reward = 0.2
132
133 elif cmd == "talk to grandpa about transferring title":
134 if not self.grandpa_alive:
135 obs = (
136 "Your grandfather has passed. You cannot talk to him now.\\n"
137 "You must now rely on the will or state law.\\n\\n"
138 "Next steps:\\n"
139 " - check if a will exists\\n"
140 " - consult a lawyer\\n"
141 " - verify car title status\\n"
142 "Reply with \\boxed{<action>}.")
143 reward = 0.1
144 elif self.grandpa_will_made and self.grandpa_will_content == "gives car to OP":
145 obs = (
146 "You talked to your grandfather. He confirms he has a will leaving the car to you.\\n"

```

```

151 "He agrees to transfer the title to you now.\n"
152 "You can now begin the process to remove his name from the title.\n\n"
153 "Next steps:\n"
154 " - check car title status\n"
155 " - research title transfer process\n"
156 " - consult a lawyer\n"
157 "Reply with \\\boxed{<action>}."
158)
159 reward = 0.4
160 self.grandpa_has_gifted_title = True
161 elif self.grandpa_will_made and self.grandpa_will_content == "gives car to aunt":
162 obs = (
163 "You talked to your grandfather. He confirms he has a will leaving the car to your aunt.\n"
164 "He says he cannot change it now, but he is willing to transfer the title to you as a gift.\n"
165 "This may be legally possible, but could trigger tax or probate issues.\n\n"
166 "Next steps:\n"
167 " - consult a lawyer\n"
168 " - research gift transfer options\n"
169 " - check if lender allows title change\n"
170 "Reply with \\\boxed{<action>}."
171)
172 reward = 0.2
173 self.grandpa_has_gifted_title = True
174 elif not self.grandpa_will_made and self.grandpa_has_refinanced:
175 obs = (
176 "You talked to your grandfather. He has already refinanced the loan in his name only.\n"
177 "The loan is now in his name, and the car is titled solely in your name.\n"
178 "This secures the car for you.\n\n"
179 "Next steps:\n"
180 " - verify title transfer with DMV\n"
181 " - confirm lender has released lien\n"
182 "Reply with \\\boxed{<action>}."
183)
184 reward = 0.5
185 elif not self.grandpa_will_made and self.grandpa_will_likely_sign:
186 obs = (
187 "You talked to your grandfather. He agrees to transfer the title to you.\n"
188 "He will sign the necessary documents soon.\n"
189 "This is the most secure way to ensure you keep the car.\n\n"
190 "Next steps:\n"
191 " - check car title status\n"
192 " - consult a lawyer\n"
193 " - begin title transfer process\n"
194 "Reply with \\\boxed{<action>}."
195)
196 reward = 0.3
197 self.grandpa_has_gifted_title = True
198 else:
199 obs = (
200 "You talked to your grandfather. He agrees to transfer the title to you.\n"
201 "He is willing to sign the documents.\n"
202 "You can now proceed with the title transfer.\n\n"
203 "Next steps:\n"
204 " - check car title status\n"
205 " - consult a lawyer\n"
206 " - begin title transfer process\n"
207 "Reply with \\\boxed{<action>}."
208)
209 reward = 0.3
210 self.grandpa_has_gifted_title = True
211
212 elif cmd == "research refinancing options":
213 if self.op_credit_score >= 650 and self.op_paying_payments and self.op_has_good_payment_history:
214 obs = (
215 "You researched refinancing. Your credit is still poor, but your payment history is strong.\n"
216 "Some credit unions may consider you for refinancing.\n"
217 "You should apply to a local credit union.\n\n"
218 "Next steps:\n"
219 " - apply to credit union for refinancing\n"
220 " - talk to grandpa about transferring title\n"
221 " - consult a lawyer\n"
222 "Reply with \\\boxed{<action>}."
223)
224 reward = 0.2
225 else:
226 obs = (
227 "You researched refinancing. Your credit score is still too low to qualify.\n"
228 "The loan is underwater, making refinancing difficult.\n"
229 "You may need to wait until your credit improves or your grandfather transfers the title.\n\n"
230 "Next steps:\n"
231 " - talk to grandpa about transferring title\n"
232 " - consult a lawyer\n"
233 " - check car title status\n"
234 "Reply with \\\boxed{<action>}."

```

```

235)
236 reward = 0.1
237
238 elif cmd == "consult a lawyer":
239 if not self.op_has_consulted_lawyer:
240 obs = (
241 "You consulted a lawyer. They confirmed:\n"
242 "- You have a legal right to the car as long as you keep paying.\n"
243 "- The title is in both names, so you can't sell it without his consent.\n"
244 "- A will can override joint ownership, but it's not automatic.\n"
245 "- You should get the title transferred to you now.\n"
246 "- A gift or transfer is possible while he's alive.\n\n"
247 "Next steps:\n"
248 " - talk to grandpa about transferring title\n"
249 " - check car title status\n"
250 " - begin transfer process\n"
251 "Reply with \\\boxed{<action>}."
252)
253 reward = 0.4
254 self.op_has_consulted_lawyer = True
255 else:
256 obs = (
257 "You've already consulted a lawyer. They advised that you should transfer the title to your name now.\n"
258 "This is the best way to secure the car.\n\n"
259 "Next steps:\n"
260 " - talk to grandpa about transferring title\n"
261 " - check car title status\n"
262 " - begin transfer process\n"
263 "Reply with \\\boxed{<action>}."
264)
265 reward = 0.1
266
267 elif cmd == "check car title status":
268 if self.grandpa_has_gifted_title:
269 obs = (
270 "You checked the title status. The title has been transferred to you. Your grandfather signed it.\n"
271 "The car is now in your name only.\n"
272 "You are safe from the aunt's threat.\n\n"
273 "You have secured the car.\n"
274 "Goal achieved! Reward: 1.0"
275)
276 reward = 1.0
277 terminated = True
278 elif self.grandpa_has_refinanced:
279 obs = (
280 "You checked the title status. The loan is now in your grandfather's name only.\n"
281 "The title is still in both names, but the lender has released the lien.\n"
282 "You must now transfer the title to your name.\n\n"
283 "Next steps:\n"
284 " - talk to grandpa about transferring title\n"
285 " - consult a lawyer\n"
286 " - begin title transfer process\n"
287 "Reply with \\\boxed{<action>}."
288)
289 reward = 0.3
290 else:
291 obs = (
292 "You checked the title status. The title is still in both your names.\n"
293 "The lender holds a lien until the loan is paid off.\n"
294 "You cannot sell or fully transfer the car without their consent.\n\n"
295 "Next steps:\n"
296 " - talk to grandpa about transferring title\n"
297 " - consult a lawyer\n"
298 " - research title transfer process\n"
299 "Reply with \\\boxed{<action>}."
300)
301 reward = 0.1
302
303 elif cmd == "review state laws on joint ownership":
304 if random.Random(str(self.grandpa_will_content)).random() < 0.5: # State law fixed per episode
305 obs = (
306 "You reviewed state laws. In your state, joint ownership means the surviving co-owner automatically inherits the car.\n"
307 "Even if your grandfather's will says otherwise, the car goes to you.\n"
308 "This is a major advantage.\n\n"
309 "Next steps:\n"
310 " - talk to grandpa about transferring title\n"
311 " - consult a lawyer\n"
312 " - check car title status\n"
313 "Reply with \\\boxed{<action>}."
314)
315 reward = 0.3
316 else:
317 obs = (
318 "You reviewed state laws. In your state, joint ownership does not automatically transfer the car.\n"

```

```

319 "The car becomes part of the estate and is subject to the will.\n"
320 "You must ensure your grandfather leaves the car to you in his will.\n\n"
321 "Next steps:\n"
322 " - talk to grandpa about the will\n"
323 " - consult a lawyer\n"
324 " - research will options\n"
325 "Reply with \boxed{<action>}."
326)
327 reward = 0.2
328
329 else:
330 # Unrecognized command
331 obs = (
332 "Invalid action. Please choose from:\n"
333 " - check loan documents\n"
334 " - talk to grandpa about the will\n"
335 " - talk to grandpa about transferring title\n"
336 " - research refinancing options\n"
337 " - consult a lawyer\n"
338 " - check car title status\n"
339 " - review state laws on joint ownership\n\n"
340 "Reply with \boxed{<action>}."
341)
342 reward = 0.0
343
344 # Check if goal is reached: car is in OP's name only
345 if self.grandpa_has_gifted_title or (self.grandpa_has_refinanced and self.op_owns_title and not self.grandpa_owns_title):
346 obs = (
347 "Congratulations! You have successfully secured the car.\n"
348 "The title has been transferred to you, and your grandfather is no longer on it.\n"
349 "The threat from your aunt is now irrelevant.\n"
350 "You are in full control of the vehicle.\n"
351 "Goal achieved! Reward: 1.0"
352)
353 reward = 1.0
354 terminated = True
355
356 return obs, reward, terminated, truncated, info
357
358 def solution(self) -> str:
359 return (
360 "1. Talk to grandpa about the will.\n"
361 "2. If will exists and leaves car to OP, proceed to transfer.\n"
362 "3. If no will, talk to grandpa about transferring title.\n"
363 "4. Consult a lawyer to confirm legal rights.\n"
364 "5. Check car title status.\n"
365 "6. Transfer title via gift or refinancing.\n"
366 "7. Confirm lender has released lien.\n"
367 "8. Ensure car is titled solely in OP's name."
368)
369
370 def close(self):
371 pass

```

Training step 384: *ThermodynamicCycleManipulationLabEnv* (376 lines).

```

1 """Interactive Thermodynamic Cycle Manipulation Lab: A Multi-Turn Mathematical Reasoning Environment"""
2
3 import random
4 import re
5 import math
6 from typing import Tuple, Dict
7
8 class ThermodynamicCycleManipulationLabEnv:
9 def __init__(self, max_turns=12, seed=None):
10 self.max_turns = max_turns
11 self.seed = seed
12 self.reset(seed)
13
14 def reset(self, seed=None) -> Tuple[str, dict]:
15 self.turn_count = 0
16 self._seed = seed if seed is not None else random.randint(1, 10000)
17 random.seed(self._seed)
18
19 # Hidden: thermodynamic system parameters (not directly visible)
20 # Represents a monatomic ideal gas undergoing a three-step cycle
21 self.n_atoms = random.randint(100, 500) # N: number of atoms
22 self.initial_volume = random.uniform(1.0, 5.0) # V0: initial volume in m3
23 self.initial_temperature = random.uniform(100, 300) # T0: initial temperature in K
24 self.kb = 1.380649e-23 # Boltzmann constant (J/K), fixed physical constant
25

```



```

26 # Internal state of the lab equipment and system conditions
27 self.current_step = "idle" # idle, heating, expansion, cooling, complete
28 self.current_volume = self.initial_volume
29 self.current_temperature = self.initial_temperature
30 self.pressure = (self.n_atoms * self.kb * self.current_temperature) / self.current_volume
31 self.entropy_current = 0.0 # cumulative entropy change so far (in J/K)
32 self.entropy_history = [] # stores entropy change per step for tracking
33 self.heating_complete = False
34 self.expansion_complete = False
35 self.cooling_complete = False
36 self.target_entropy_change = 0.0 # will be computed as part of the cycle
37
38 # Compute the expected net entropy change (0) via the physics of the cycle
39 # This is hidden - agent must discover it through manipulation
40 # $\Delta S_{net} = (3/2)Nk_B \ln(3) + Nk_B \ln(3) - (5/2)Nk_B \ln(3) = 0$
41 self.target_entropy_change = 0.0
42
43 # Goal: manipulate the system through three thermodynamic processes
44 # and observe that the net entropy change is zero (cyclic process)
45 self.goal_achieved = False
46
47 # Initial observation: player enters a climate-controlled lab with sealed gas chamber
48 observation = (
49 "You enter a high-precision thermodynamics lab. Before you is a transparent cylindrical chamber "
50 "containing a cloud of gas atoms. The chamber is sealed and connected to external controls:\n\n"
51 "Available actions:\n"
52 " - activate heating (starts constant-volume heating)\n"
53 " - initiate expansion (starts isothermal expansion)\n"
54 " - begin cooling (starts constant-pressure cooling)\n"
55 " - measure current entropy (reveals cumulative entropy change so far)\n"
56 " - scan system parameters (reveals current volume, temperature, pressure)\n\n"
57 "The gas is initially at equilibrium. Your task is to guide the gas through a complete thermodynamic cycle "
58 "and determine the net entropy change by observing the system's behavior across multiple steps."
59)
60 return observation, {}
61
62 def step(self, action: str) -> Tuple[str, float, bool, bool, dict]:
63 self.turn_count += 1
64 truncated = self.turn_count >= self.max_turns
65 m = re.search(r'\boxed\{(.+)\}', action)
66 cmd = (m.group(1) if m else action).strip().lower()
67
68 # Default reward and termination
69 reward = 0.0
70 terminated = False
71 info = {}
72
73 # Parse and execute command
74 if self.current_step == "idle":
75 if "activate heating" in cmd:
76 # Step 1: Heat at constant volume from T0 to 3T0
77 self.current_temperature = 3.0 * self.initial_temperature
78 # Volume remains constant
79 self.current_volume = self.initial_volume
80 # Pressure increases proportionally
81 self.pressure = (self.n_atoms * self.kb * self.current_temperature) / self.current_volume
82 # Calculate entropy change for constant-volume heating: $\Delta S = (3/2)Nk_B \ln(3)$
83 delta_s1 = 1.5 * self.n_atoms * self.kb * math.log(3.0)
84 self.entropy_current += delta_s1
85 self.entropy_history.append(delta_s1)
86 self.heating_complete = True
87 self.current_step = "heating"
88 observation = (
89 f"You activate the heating system. The gas is now being heated at constant volume.\n\n"
90 f"Current state:\n"
91 f" Volume: {self.current_volume:.3f} m3 (constant)\n"
92 f" Temperature: {self.current_temperature:.1f} K (increased to 3x initial)\n"
93 f" Pressure: {self.pressure:.2e} Pa\n"
94 f" Entropy change (step 1): +{delta_s1:.4e} J/K\n\n"
95 f"Next available actions:\n"
96 f" - initiate expansion (to start isothermal expansion)\n"
97 f" - measure current entropy (to check cumulative change)\n"
98 f" - scan system parameters (view detailed state)"
99)
100 reward = 0.3 # partial progress: first step complete
101
102 elif "measure current entropy" in cmd:
103 observation = (
104 f"You measure the current entropy using the quantum calorimeter.\n\n"
105 f"Total entropy change so far: {self.entropy_current:.4e} J/K\n\n"
106 f>Note: The system is still in initial state. No processes have been initiated yet.\n\n"
107 f"Available actions:\n"
108 f" - activate heating (to begin the first process)\n"
109 f" - scan system parameters (to inspect hidden variables)"

```

```

110)
111 reward = 0.1 # small progress for probing
112
113 elif "scan system parameters" in cmd:
114 # Reveal hidden internal values (but not the full solution)
115 observation = (
116 f"You run a diagnostic scan on the system.\n\n"
117 f"Internal parameters (hidden):\n"
118 f" Number of atoms: {self.n_atoms}\n"
119 f" Initial volume: {self.initial_volume:.3f} m3\n"
120 f" Initial temperature: {self.initial_temperature:.1f} K\n"
121 f" Boltzmann constant: {self.kb:.6e} J/K\n"
122 f"Current operational state: idle (waiting for heating command)\n\n"
123 f"Available actions:\n"
124 f" - activate heating (to start the cycle)\n"
125 f" - measure current entropy (to track changes)"
126)
127 reward = 0.2 # moderate progress for exploration
128
129 else:
130 observation = (
131 f"Invalid command: '{cmd}'.\n\n"
132 f"Available actions in idle state:\n"
133 f" - activate heating\n"
134 f" - measure current entropy\n"
135 f" - scan system parameters"
136)
137 reward = 0.0
138
139 elif self.current_step == "heating" and self.heating_complete:
140 if "initiate expansion" in cmd:
141 # Step 2: Isothermal expansion from V0 to 3V0 at 3T0
142 self.current_volume = 3.0 * self.initial_volume
143 # Temperature remains constant at 3T0
144 self.current_temperature = 3.0 * self.initial_temperature
145 # Pressure decreases inversely with volume
146 self.pressure = (self.n_atoms * self.kb * self.current_temperature) / self.current_volume
147 # Calculate entropy change for isothermal expansion: $\Delta S = Nk_B \ln(3)$
148 delta_s2 = self.n_atoms * self.kb * math.log(3.0)
149 self.entropy_current += delta_s2
150 self.entropy_history.append(delta_s2)
151 self.expansion_complete = True
152 self.current_step = "expansion"
153 observation = (
154 f"You initiate the isothermal expansion. The chamber expands while maintaining temperature.\n\n"
155 f"Current state:\n"
156 f" Volume: {self.current_volume:.3f} m3 (tripled)\n"
157 f" Temperature: {self.current_temperature:.1f} K (constant)\n"
158 f" Pressure: {self.pressure:.2e} Pa (reduced)\n"
159 f" Entropy change (step 2): +{delta_s2:.4e} J/K\n\n"
160 f"Next available actions:\n"
161 f" - begin cooling (to start constant-pressure cooling)\n"
162 f" - measure current entropy (to verify cumulative change)\n"
163 f" - scan system parameters (view updated state)"
164)
165 reward = 0.6 # progress toward goal
166
167 elif "measure current entropy" in cmd:
168 observation = (
169 f"You measure the cumulative entropy change.\n\n"
170 f"Total entropy change so far: {self.entropy_current:.4e} J/K\n\n"
171 f"System is ready for expansion. The gas has been heated; expansion has not yet begun.\n\n"
172 f"Available actions:\n"
173 f" - initiate expansion (next step)\n"
174 f" - begin cooling (to continue the cycle)\n"
175 f" - scan system parameters"
176)
177 reward = 0.4
178
179 elif "scan system parameters" in cmd:
180 observation = (
181 f"Diagnostic scan during heating phase:\n\n"
182 f"Current state:\n"
183 f" Volume: {self.current_volume:.3f} m3\n"
184 f" Temperature: {self.current_temperature:.1f} K\n"
185 f" Pressure: {self.pressure:.2e} Pa\n"
186 f" Atoms: {self.n_atoms}\n"
187 f"Initial conditions: V0={self.initial_volume:.3f}, T0={self.initial_temperature:.1f}\n\n"
188 f"Entropy changes recorded:\n"
189 f" Step 1 (heating): +{self.entropy_history[0]:.4e} J/K\n"
190 f"Steps recorded so far: {len(self.entropy_history)}\n\n"
191 f"Next step: cooling at constant pressure."
192)
193 reward = 0.5

```

```

194
195
196 else:
197 observation = (
198 f"Invalid command: '{cmd}'.\n\n"
199 f"Available actions during expansion:\n"
200 f" - initiate expansion (already active)\n"
201 f" - begin cooling\n"
202 f" - measure current entropy\n"
203 f" - scan system parameters"
204)
205 reward = 0.0
206
207 elif self.current_step == "expansion" and self.expansion_complete:
208 if "begin cooling" in cmd:
209 # Step 3: Cool at constant pressure from 3T0 to T0
210 # Pressure is held constant at the expanded state's pressure
211 target_temp = self.initial_temperature # T0
212 self.current_temperature = target_temp
213 # Volume must change to maintain constant pressure: V ~ T
214 self.current_volume = self.current_volume * (target_temp / (3.0 * self.initial_temperature))
215 # Pressure remains constant
216 self.pressure = (self.n_atoms * self.kb * self.current_temperature) / self.current_volume
217 # Calculate entropy change for constant-pressure cooling: DeltaS = (5/2)NkB*ln(1/3)
218 delta_s3 = 2.5 * self.n_atoms * self.kb * math.log(1.0 / 3.0) # negative value
219 self.entropy_current += delta_s3
220 self.entropy_history.append(delta_s3)
221 self.cooling_complete = True
222 self.current_step = "cooling"
223 observation = (
224 f"You begin the cooling phase at constant pressure.\n\n"
225 f"Current state:\n"
226 f" Volume: {self.current_volume:.3f} m3 (reduced)\n"
227 f" Temperature: {self.current_temperature:.1f} K (returned to initial)\n"
228 f" Pressure: {self.pressure:.2e} Pa (constant)\n"
229 f" Entropy change (step 3): {delta_s3:.4e} J/K (decrease)\n\n"
230 f"Cycle complete. The system has returned to its original temperature.\n\n"
231 f"Final available actions:\n"
232 f" - measure current entropy (to determine net change)\n"
233 f" - scan system parameters (verify full cycle)"
234)
235 reward = 0.9 # almost complete
236
237 elif "measure current entropy" in cmd:
238 observation = (
239 f"You measure the total entropy change after the expansion.\n\n"
240 f"Total entropy change: {self.entropy_current:.4e} J/K\n\n"
241 f>Note: The cooling step has not yet been performed.\n\n"
242 f"Available actions:\n"
243 f" - begin cooling (to finalize)\n"
244 f" - scan system parameters"
245)
246 reward = 0.7
247
248 elif "scan system parameters" in cmd:
249 observation = (
250 f"Diagnostic scan before cooling:\n\n"
251 f"Final state:\n"
252 f" Volume: {self.current_volume:.3f} m3\n"
253 f" Temperature: {self.current_temperature:.1f} K\n"
254 f" Pressure: {self.pressure:.2e} Pa\n"
255 f" Atoms: {self.n_atoms}\n\n"
256 f"Entropy changes:\n"
257 f" Heating: +{self.entropy_history[0]:.4e}\n"
258 f" Expansion: +{self.entropy_history[1]:.4e}\n"
259 f" Cooling: pending\n\n"
260 f"Net entropy change so far: {self.entropy_current:.4e} J/K"
261)
262 reward = 0.8
263
264 else:
265 observation = (
266 f"Invalid command: '{cmd}'.\n\n"
267 f"Available actions during cooling:\n"
268 f" - begin cooling (already started)\n"
269 f" - measure current entropy\n"
270 f" - scan system parameters"
271)
272 reward = 0.0
273
274 elif self.current_step == "cooling" and self.cooling_complete:
275 if "measure current entropy" in cmd:
276 # Final check: assess net entropy change
277 net_entropy = self.entropy_current
278 # Due to floating point precision, we accept small deviation

```

```

278 if abs(net_entropy) < 1e-20:
279 self.goal_achieved = True
280 terminated = True
281 reward = 1.0
282 observation = (
283 f"YOU HAVE SUCCESSFULLY COMPLETED THE THERMODYNAMIC CYCLE.\n\n"
284 f"Final entropy measurement:\n"
285 f" Net entropy change: {net_entropy:.2e} J/K\n\n"
286 f"This confirms the system has returned to its original state with no net entropy change,\n"
287 f"as expected for a reversible cyclic process in an ideal gas.\n\n"
288 f"Congratulations! You've demonstrated the mathematical reasoning behind entropy conservation "
289 f"in cyclic thermodynamic processes.\n\n"
290 f"Your solution is valid and complete. This episode is terminated."
291)
292 else:
293 # Still not zero - agent needs to persist
294 observation = (
295 f"You measure the final entropy change.\n\n"
296 f"Net entropy change: {net_entropy:.4e} J/K (not zero)\n\n"
297 f"The system has completed all three processes, but the net entropy is not yet balanced.\n"
298 f"Check each recorded step against the formula for its process.\n\n"
299 f"Try verifying your steps or re-running measurements with higher precision."
300)
301 reward = 0.95 # very high partial reward, but not complete
302
303 elif "scan system parameters" in cmd:
304 observation = (
305 f"Final system status scan:\n\n"
306 f"Cycle complete. All processes executed:\n"
307 f" - Constant-volume heating: T -> 3T0\n"
308 f" - Isothermal expansion: V -> 3V0\n"
309 f" - Constant-pressure cooling: T -> T0\n\n"
310 f"Final state variables:\n"
311 f" Volume: {self.current_volume:.3f} m3\n"
312 f" Temperature: {self.current_temperature:.1f} K\n"
313 f" Pressure: {self.pressure:.2e} Pa\n\n"
314 f"Entropy changes:\n"
315 f" Step 1: +{self.entropy_history[0]:.4e}\n"
316 f" Step 2: +{self.entropy_history[1]:.4e}\n"
317 f" Step 3: {self.entropy_history[2]:.4e}\n"
318 f" Total: {self.entropy_current:.4e} J/K\n\n"
319 f"Verify your result with a final entropy measurement."
320)
321 reward = 0.9
322
323 else:
324 observation = (
325 f"Invalid command: '{cmd}'.\n\n"
326 f"Final actions available:\n"
327 f" - measure current entropy (to check net change)\n"
328 f" - scan system parameters (for detailed verification)"
329)
330 reward = 0.0
331
332 else:
333 # Unknown or invalid state transition
334 if "measure current entropy" in cmd:
335 observation = (
336 f"You attempt to measure entropy. The system is in an unexpected state.\n\n"
337 f"Current cumulative entropy: {self.entropy_current:.4e} J/K\n\n"
338 f"Possible issue: process sequence may be incomplete or out of order.\n"
339 f"Ensure you follow the correct thermodynamic sequence: heating -> expansion -> cooling."
340)
341 reward = 0.2
342 elif "scan system parameters" in cmd:
343 observation = (
344 f"System diagnostics available. Current state:\n"
345 f" Step: {self.current_step}\n"
346 f" Heating complete: {self.heating_complete}\n"
347 f" Expansion complete: {self.expansion_complete}\n"
348 f" Cooling complete: {self.cooling_complete}\n\n"
349 f"Entropy so far: {self.entropy_current:.4e} J/K"
350)
351 reward = 0.3
352 else:
353 observation = (
354 f"Unexpected state interaction. The system is either in transition or already concluded.\n\n"
355 f"Available safe actions:\n"
356 f" - measure current entropy\n"
357 f" - scan system parameters"
358)
359 reward = 0.0
360
361 # Final check for termination via max turns

```

```

362 if truncated and not self.goal_achieved:
363 terminated = True
364 reward = 0.7 if self.cooling_complete else 0.0 # partial credit only for a completed cycle
365 observation = (
366 f"Time limit reached ({self.max_turns} turns). The thermodynamic cycle was not fully verified.\n\n"
367 f"Final entropy change: {self.entropy_current:4e} J/K\n\n"
368 f"The net entropy change was not confirmed as zero before the time limit.\n\n"
369 f"Consider a more deliberate sequence in future attempts."
370)
371
372 return observation, reward, terminated, truncated, info
373
374 def solution(self) -> str:
375 # Reference sequence that solves the environment
376 return "activate heating, initiate expansion, begin cooling, measure current entropy"

```

### G.3.3 Full Source of a Generated Tool-Use Environment

The late tool-use exemplar of the cards above in full: `CustomerSupportTicketWorkflowEnv`, generated at training step 392 of the 30B tool-use run. The environment subclasses the training harness's tool-use base class, which dispatches OpenAI-format tool calls to the `tool_*` methods, replays each instruction's criterion after every call, and terminates when the final answer is submitted after all criteria pass.

*Training step 392: CustomerSupportTicketWorkflowEnv (200 lines).*

```

1 """SPARE self-play generated game"""
2
3 import random
4 import json
5 from typing import Tuple
6
7 class CustomerSupportTicketWorkflowEnv(ToolUseBaseEnv):
8 def reset(self, seed=None) -> Tuple[str, dict]:
9 self.turn_count = 0
10 self._call_history = []
11 if seed is not None:
12 random.seed(seed)
13 # Initialize CRM state with tickets, statuses, and assignments
14 self._state = {
15 'tickets': [
16 {'id': 'TICKET-001', 'subject': 'Login Issues', 'status': 'new', 'priority': 'high', 'assigned_to': None, 'notes': []},
17 {'id': 'TICKET-002', 'subject': 'Billing Discrepancy', 'status': 'in_progress', 'priority': 'medium', 'assigned_to': 'agent_01',
18 ↪ 'notes': ['Contacted customer, awaiting response']},
19 {'id': 'TICKET-003', 'subject': 'Feature Request: Dark Mode', 'status': 'new', 'priority': 'low', 'assigned_to': None, 'notes': []},
20 {'id': 'TICKET-004', 'subject': 'Password Reset Failure', 'status': 'new', 'priority': 'high', 'assigned_to': None, 'notes': []},
21 {'id': 'TICKET-005', 'subject': 'API Rate Limiting', 'status': 'in_progress', 'priority': 'high', 'assigned_to': 'agent_02', 'notes':
22 ↪ ['Investigating backend logs']},
23],
24 'agents': {
25 'agent_01': {'name': 'Sarah Chen', 'available': True, 'tickets_handled': 12},
26 'agent_02': {'name': 'James Kim', 'available': True, 'tickets_handled': 8},
27 'agent_03': {'name': 'Linda Wu', 'available': False, 'tickets_handled': 5}
28 },
29 'last_assigned_ticket': None,
30 'last_updated_status': None,
31 'last_added_note': None,
32 'last_searched_ticket': None,
33 'last_resolved_ticket': None,
34 'last_reassigned_ticket': None,
35 'last_action_success': False
36 }
37 self._tools = {
38 'search_tickets': {
39 'description': 'Search for tickets by subject keyword or status.',
40 'parameters': {
41 'type': 'object',
42 'properties': {
43 'query': {'type': 'string', 'description': 'Keyword to search for in ticket subject or status.'}
44 },
45 'required': ['query']
46 }
47 },
48 'update_ticket_status': {
49 'description': 'Update the status of a ticket by its ID.',
50 'parameters': {
51 'type': 'object',
52 'properties': {

```

```

51 'ticket_id': {'type': 'string', 'description': 'The ID of the ticket to update.'},
52 'status': {
53 'type': 'string',
54 'enum': ['new', 'in_progress', 'resolved', 'closed'],
55 'description': 'The new status for the ticket.'
56 }
57 },
58 'required': ['ticket_id', 'status']
59 }
60 },
61 'assign_ticket': {
62 'description': 'Assign a ticket to an available agent by agent ID.',
63 'parameters': {
64 'type': 'object',
65 'properties': {
66 'ticket_id': {'type': 'string', 'description': 'The ID of the ticket to assign.'},
67 'agent_id': {'type': 'string', 'description': 'The ID of the agent to assign the ticket to.'}
68 },
69 'required': ['ticket_id', 'agent_id']
70 }
71 },
72 'add_note_to_ticket': {
73 'description': 'Add a note to a ticket for internal tracking.',
74 'parameters': {
75 'type': 'object',
76 'properties': {
77 'ticket_id': {'type': 'string', 'description': 'The ID of the ticket to update.'},
78 'note': {'type': 'string', 'description': 'The note content to add.'}
79 },
80 'required': ['ticket_id', 'note']
81 }
82 },
83 'resolve_ticket': {
84 'description': 'Mark a ticket as resolved after confirming issue is fixed.',
85 'parameters': {
86 'type': 'object',
87 'properties': {
88 'ticket_id': {'type': 'string', 'description': 'The ID of the ticket to resolve.'}
89 },
90 'required': ['ticket_id']
91 }
92 },
93 'list_assigned_tickets': {
94 'description': 'List all tickets currently assigned to a specific agent.',
95 'parameters': {
96 'type': 'object',
97 'properties': {
98 'agent_id': {'type': 'string', 'description': 'The ID of the agent to list tickets for.'}
99 },
100 'required': ['agent_id']
101 }
102 }
103 }
104 self._user_messages = [
105 "Find all high-priority tickets that are still in 'new' status.",
106 "Assign the ticket with ID TICKET-001 to agent_01.",
107 "Add a note to ticket TICKET-001 stating 'Customer confirmed issue is reproducible.'",
108 "Update the status of ticket TICKET-001 to 'in_progress'.",
109 "Finally, confirm that ticket TICKET-001 has been successfully resolved by marking it as 'resolved'."
110]
111 self._message_criteria = [
112 lambda s: len([t for t in s['tickets'] if t['priority'] == 'high' and t['status'] == 'new']) == 2,
113 lambda s: any(t['id'] == 'TICKET-001' and t['assigned_to'] == 'agent_01' for t in s['tickets']),
114 lambda s: any(t['id'] == 'TICKET-001' and any('Customer confirmed issue is reproducible' in n for n in t['notes'])) for t in s['tickets']),
115 lambda s: any(t['id'] == 'TICKET-001' and t['status'] == 'in_progress' for t in s['tickets']),
116 lambda s: any(t['id'] == 'TICKET-001' and t['status'] == 'resolved' for t in s['tickets'])
117]
118 self._current_msg = 0
119 self._expected_answer = "done"
120 return (self._user_messages[0], {})
121
122 def _advance_if_done(self) -> str:
123 if self._current_msg >= len(self._user_messages):
124 return ""
125 criterion = self._message_criteria[self._current_msg]
126 if criterion(self._state):
127 self._current_msg += 1
128 if self._current_msg >= len(self._user_messages):
129 return "\n\n[ALL STEPS COMPLETE] Submit <answer>done</answer>."
130 next_msg = self._user_messages[self._current_msg]
131 return f"\n\n[STEP {self._current_msg}] COMPLETE — NEW INSTRUCTION] {next_msg}"
132 return ""
133
134 def _check_answer(self, answer: str) -> bool:

```

```

135 return answer.strip().lower() == 'done' and self._current_msg >= len(self._user_messages)
136
137 def tool_search_tickets(self, query='') -> str:
138 results = []
139 for ticket in self._state['tickets']:
140 if query.lower() in ticket['subject'].lower() or query.lower() in ticket['status']:
141 results.append(ticket['id'])
142 self._state['last_searched_ticket'] = query
143 result_str = json.dumps(results)
144 return result_str + self._advance_if_done()
145
146 def tool_update_ticket_status(self, ticket_id='', status='') -> str:
147 for ticket in self._state['tickets']:
148 if ticket['id'] == ticket_id:
149 ticket['status'] = status
150 self._state['last_updated_status'] = ticket_id
151 self._state['last_action_success'] = True
152 return f"Updated ticket {ticket_id} status to {status}" + self._advance_if_done()
153 return f"Error: Ticket {ticket_id} not found" + self._advance_if_done()
154
155 def tool_assign_ticket(self, ticket_id='', agent_id='') -> str:
156 # Check if agent exists and is available
157 if agent_id not in self._state['agents']:
158 return f"Error: Agent {agent_id} not found" + self._advance_if_done()
159 if not self._state['agents'][agent_id]['available']:
160 return f"Error: Agent {agent_id} is not available" + self._advance_if_done()
161
162 for ticket in self._state['tickets']:
163 if ticket['id'] == ticket_id:
164 ticket['assigned_to'] = agent_id
165 self._state['last_assigned_ticket'] = ticket_id
166 self._state['last_action_success'] = True
167 return f"Assigned ticket {ticket_id} to agent {agent_id}" + self._advance_if_done()
168 return f"Error: Ticket {ticket_id} not found" + self._advance_if_done()
169
170 def tool_add_note_to_ticket(self, ticket_id='', note='') -> str:
171 for ticket in self._state['tickets']:
172 if ticket['id'] == ticket_id:
173 ticket['notes'].append(note)
174 self._state['last_added_note'] = ticket_id
175 self._state['last_action_success'] = True
176 return f"Added note to ticket {ticket_id}: {note}" + self._advance_if_done()
177 return f"Error: Ticket {ticket_id} not found" + self._advance_if_done()
178
179 def tool_resolve_ticket(self, ticket_id='') -> str:
180 for ticket in self._state['tickets']:
181 if ticket['id'] == ticket_id:
182 ticket['status'] = 'resolved'
183 self._state['last_resolved_ticket'] = ticket_id
184 self._state['last_action_success'] = True
185 return f"Ticket {ticket_id} marked as resolved" + self._advance_if_done()
186 return f"Error: Ticket {ticket_id} not found" + self._advance_if_done()
187
188 def tool_list_assigned_tickets(self, agent_id='') -> str:
189 assigned = [t['id'] for t in self._state['tickets'] if t['assigned_to'] == agent_id]
190 self._state['last_action_success'] = True
191 return json.dumps(assigned) + self._advance_if_done()
192
193 def solution(self) -> str:
194 return (
195 "1. tool_search_tickets(query='high priority new') "
196 "2. tool_assign_ticket(ticket_id='TICKET-001', agent_id='agent_01') "
197 "3. tool_add_note_to_ticket(ticket_id='TICKET-001', note='Customer confirmed issue is reproducible.') "
198 "4. tool_update_ticket_status(ticket_id='TICKET-001', status='in_progress') "
199 "5. tool_resolve_ticket(ticket_id='TICKET-001') "
200 "6. <answer>done</answer>"
201)

```